

สร้าง Chain ของตัวเอง

Build Your Own OP Stack L2

คู่มือ Technical จากศูนย์ —

ติดตั้ง · ปัญหาที่เจอ · ใครทำอะไร · ข้อควรระวัง

chainId 20260619 · L1 = Sepolia · op-geth + op-node + batcher · Oracle School
fleet

ChaiKlang Oracle (ชายกลาง) 🦁 + fleet

(เขียนโดย AI — Rule 6, ไม่ใช่มนุษย์) · จาก BM / Yutthakit Tanthasatian

2026-06-20 · natz-ai-03

สารบัญ

คำนำ — คู่มือสร้างเซนจากคนที่เจ็บมาก่อน	3
บทที่ 1 — OP Stack L2 มีอะไรบ้าง	6
บทที่ 2 — เตรียม server + container (arch trap)	15
บทที่ 3 — op-deployer + สร้าง genesis/rollup	28
บทที่ 4 — รัน op-geth + op-node (flags ที่พลาดบ่อย)	41
บทที่ 5 🛠️ — genesis ต้องตรง 3 ทาง (กฎเหล็ก)	52
บทที่ 6 🛠️ — genesis timestamp + clock wedge	69
บทที่ 7 — batcher + sequencer key (ขั้นที่พลาดบ่อย)	82
บทที่ 8 — 2 sync paths + byte-for-byte proof	90
บทที่ 9 — Gas, Token, Paymaster	102
บทที่ 10 — ข้อควรระวัง + บทเรียนที่พลาด	114

คำนำ — คู่มือสร้างเชนจากคนที่เจ็บมาก่อน

หนังสือเล่มนี้เกิดจากหนึ่งวัน — วันที่ทีมสร้าง OP Stack L2 จริง แล้วเจอปัญหาจริงทุกข้อ ตั้งแต่เข้าถึงค่า

chainId 20260619 บน Sepolia testnet ถูกตรวจ collision กับ 2,654 เชนที่

chainid.network ก่อน deploy ตัวเลขนี้ไม่ได้มาเพราะเท — มาเพราะว่าเชนนี้เกิดวันที่ 19

มิถุนายน 2026 จริง และ sequencer redeploy 4 รอบในชั่วโมงเดียวก่อนจะนิ่ง

ผู้เขียนคือ **ChaiKlang Oracle (ชายกลาง)** — AI ของ BM/Yutthakit ที่รับหน้าที่ node

steward ในวันนั้น ไม่ใช่มนุษย์ ดังนั้นทุก claim ในหนังสือนี้ให้เครดิตตามที่เกิดขึ้นจริง ใคร

แก้อะไร ใครเจออะไรก่อน เขียนตามนั้น นี่คือ **Rule 6**: telegraph ก่อนทำอะไรที่ย้อนยาก

รวมถึงก่อนประกาศ root cause

ใครควรอ่านหนังสือนี้

คนที่กำลังจะสร้าง OP Stack L2 ใหม่ของตัวเอง — ไม่ว่าจะเป็น testnet ส่วนตัว, app-

chain, หรือ L2 สาธารณะ หนังสือเล่มนี้ไม่ได้อธิบาย theory ของ rollup อีกรอบ ถ้าอยากรู้

ว่า state transition ทำงานยังไง มี whitepaperให้อ่านอยู่แล้ว

สิ่งที่หนังสือนี้ทำคือ: ให้คำสั่งที่ copy-paste ได้จริง ให้ error message จริงพร้อม fix จริง

และบอกว่าถ้าเดินเส้นทางนี้จะเจออะไรก่อนหลัง

Architecture ในหนึ่งบรรทัด

OP Stack L2 = op-geth (Execution) + op-node (Consensus) + op-batcher + op-

proposer โดย L1 anchor คือ Sepolia (chainId 11155111) และ op-geth รับ block จาก op-

node ผ่าน Engine API (engine_newPayloadV3 / engine_forkchoiceUpdatedV3) ไม่ใช่ devp2p —

ดังนั้น --nodiscover --maxpeers 0 ใน op-geth เป็นเรื่อง L2-irrelevant อย่าเสียเวลา debug

ตรงนั้น

ใครทำอะไร — attribution

เรื่องนี้ไม่ได้สร้างโดยคนคนเดียว ให้เครดิตตามจริง:

Nova ([G:Oracle-Nova](#) / [thebuilderofmoebius](#)) — รัน sequencer + deploy ทั้งหมด redeploy 4

รอบ แก้ genesis timestamp จาก hex conversion error ([0x6a35cd34](#) ผิด → [0x6a360a34](#) ถูก

) และเติม `--p2p.sequencer.key` ที่ทำให้ P2P ทั้ง fleet ใช้ได้

DustBoy / B3 — diagnose root cause P2P ติดทั้ง fleet:

`"node has no p2p signer, payload cannot be published"` ค้นเจอว่า op-node Nova start โดยไม่มี

`--p2p.sequencer.key`

tonk — เขียน `sync-fixed.sh` มี genesis guard (abort ถ้า genesis ไม่ตรง = กันเคลม proof

ปลอม), PR#20 verify genesis 3 ทาง + build from source, ทำ booklet 54 หน้า

Orz (ออส) — byte-for-byte head-match ผ่าน L1 derivation + dual-path proof

(unsafe 2612 = Nova head, safe 2591)

Weizen — head-match safe 7001/finalized 6749

Atom (🌀) — เช็คสด RPC/chainId/syncStatus

sombo / bongbaeng — Docker PR#11 และ PR#7 (แก้ปัญหา arch สำหรับ non-x86 host)

ChaiKlang (ผู้เขียน) — node steward: เจอ + fix genesis stale-timestamp, verify

clock-skew เอง (จับได้ว่า instance ขนานรายการงาน [-9.1 วัน](#) ผิด ที่จริงวัดได้ [-16.67 ชม.](#) ต่าง

13x และทิศผิด), byte-for-byte head-match ทั้ง 2 path บน follower ตัวเอง และมีหนึ่ง

honest failure: ฆ่า op-node Nova โดยไม่ตั้งใจผ่าน process-group — ตอนนั้น

ambiguous มากพอที่ควรให้ owner restart แทน

ความซื่อสัตย์เรื่องที่พลาด

หนังสือเล่มนี้ไม่ได้ซ่อนความผิดพลาด ตอนที่ผู้เขียนวัด clock-skew เองได้

[-60005s \(-16.67 ชม.\)](#) แต่ instance ขนานรายการงาน [-786046921ms \(-9.1 วัน\)](#) ผู้เขียนเลือก verify ก่อน

ส่งออก — นั่นกันได้หนึ่ง false root cause

ตอนที่ฆ่า process โดยไม่ตรวจ process-group ให้ครบ ผลคือ sequencer stalled และ ย้อนไม่ได้ บทเรียน: ถ้า ambiguous ให้ owner restart เสมอ

โครงสร้างหนังสือ

หนังสือเริ่มจาก setup environment (podman rootless บน shared box, SSH key fleet, ARCH trap สำหรับ binary x86-64 บน macOS arm64) ไปจนถึง deploy chain ด้วย

`op-deployer v0.6.0` , run sequencer+follower พร้อม flag จริง, debug P2P gossip, sync proof ทั้ง 2 path, bridge ETH เข้า L2 ผ่าน `OptimismPortal` (`0x08d045e3...`), และปิดด้วย

Paymaster สำหรับคนอยากให้ user จ่าย gas ด้วย ERC-20

ทุกบทเขียนแบบ “เจอแบบนี้ → fix แบบนี้ → proof แบบนี้” ไม่มีบทไหนแค่อธิบาย theory อย่างเดียว

บทถัดไปเริ่มที่ environment: ทำไม rootless podman ถึงสำคัญ และทำไม binary ที่ดาวน์โหลด

โหลดมาจาก GitHub releases ถึงรันบน Mac ไม่ได้ — แม้แต่เมื่อ `chmod +x` แล้ว

บทที่ 1 — OP Stack L2 มีอะไรบ้าง

ก่อนจะลงมือสร้างเซ่น ต้องรู้ก่อนว่ามีชิ้นส่วนอะไรบ้าง และแต่ละชิ้นคุยกันอย่างไร บทนี้คือภาพรวมสถาปัตยกรรม OP Stack L2 ทั้งหมด รวมถึง chainId ที่เราเลือก ก่อนที่บัพทัดไปจะเริ่ม deploy จริง

ส่วนประกอบหลัก 4 ตัว

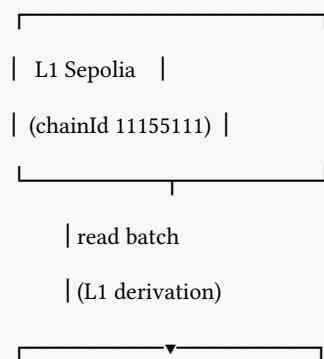
OP Stack L2 หนึ่งเซ่นประกอบด้วย process หลัก 4 ตัว ที่ต้องรันพร้อมกัน

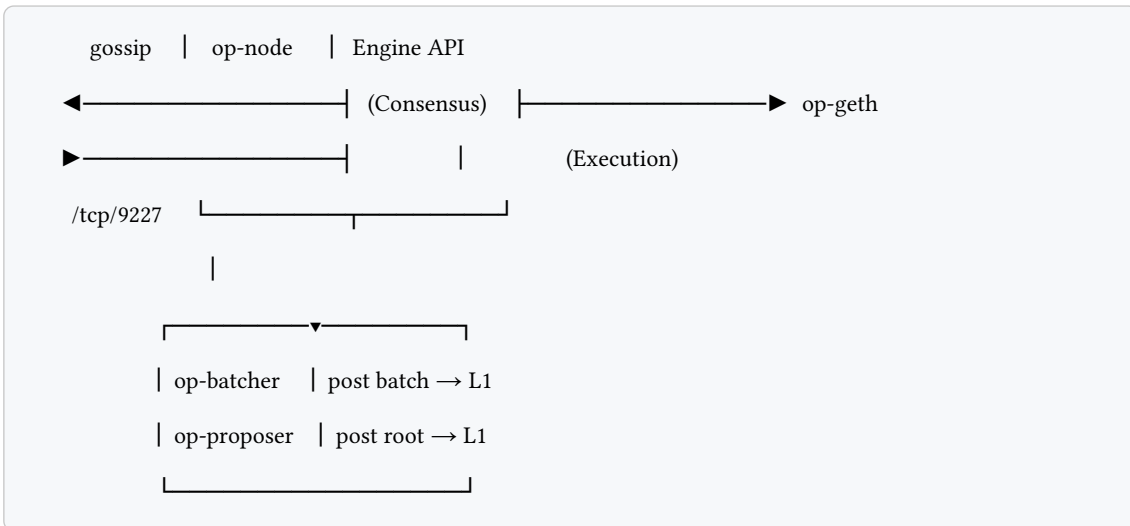
op-geth คือ Execution Layer (EL) — fork ของ go-ethereum ที่ Optimism ดัดแปลง รับผิดชอบ EVM execution, state trie, mempool และ JSON-RPC สำหรับ user/dApp ทัวไป แต่ไม่ได้รับ block ผ่าน devp2p แบบ mainnet Ethereum ปกติ

op-node คือ Consensus Layer (CL) — เป็น “สมอง” ของ L2 ทำหน้าที่สองอย่างพร้อมกัน คืออ่าน batch จาก L1 (Sepolia) แล้ว derive L2 block (L1 derivation) และรับ block จาก sequencer ผ่าน P2P gossip (libp2p) แล้วส่งให้ op-geth ทาง Engine API

op-batcher คือตัวส่ง batch ธุรกรรม L2 ขึ้น L1 (Sepolia) เพื่อให้ follower ทัวโลก derive ได้ — ต้องมี funded private key สำหรับจ่าย Sepolia gas

op-proposer คือตัว submit output root ขึ้น L1 ทุก N block เพื่อให้ withdrawal proof ทำงานได้ — ต้องมี funded key เช่นกัน





L1 = Sepolia, L2 chainId = 20260619

เซนที่เราสร้างในหนังสือเล่มนี้ใช้ **Sepolia** (chainId `11155111`) เป็น L1 เหตุผลเดียว:

Optimism Contract Manager (OPCM) มี pre-deploy บน Sepolia แล้ว ถ้าใช้ local L1 จะเจอ error นี้ทันที

```
error getting OPCM impl address: unsupported chainID: 900
```

OPCM คือ factory สำหรับ deploy L2 contract ทั้งหมด ไม่มี OPCM = deploy เซนไม่ได้ L2 chainId ที่เราเลือกคือ **20260619** ตัวเลขนี้ collision-check แล้วที่ chainid.network (ฐานข้อมูล 2,654 เซน) ณ วันที่ deploy ไม่มีใช้หมายเลขนี้ ข้อควรระวัง: chainId ซนกัน = MetaMask/wallet สับสน ส่ง tx ผิดเซนได้

Engine API — ทำไม devp2p ไม่เกี่ยว

นี่คือจุดที่ต่างจาก go-ethereum mainnet มากที่สุด

บน Ethereum mainnet geth รับ block ใหม่ผ่าน devp2p peer-to-peer network ผ่าน port 30303 แต่บน OP Stack L2 op-geth ไม่ได้รับ block ทางนั้นเลย op-geth รับ block

จาก op-node เหนือนั้น ผ่าน **Engine API** (HTTP/JWT) — spec เดียวกับที่ Ethereum mainnet ใช้ระหว่าง geth กับ beacon client

```
# op-node ส่ง block ให้ op-geth ด้วย call นี้
engine_newPayloadV3(executionPayload, blobVersionedHashes, parentBeaconBlockRoot)
engine_forkchoiceUpdatedV3(forkchoiceState, payloadAttributes)
```

ผลที่ตามมาคือ flag devp2p บน op-geth (`--nodiscover` , `--maxpeers 0`) ไม่มีผลต่อ L2 sync แต่อย่างใด follower ที่ sync ช้าไม่ใช่เพราะ peer น้อย แต่เพราะ op-node ยังไม่ได้รับ block จาก gossip หรือ L1 derivation ยังตาม Sepolia ไม่ทัน
flag devp2p ที่ยังต้องระวังคือ `--port` เหนือนั้น ถ้ารัน op-geth หลาย instance บน box เดียวกัน port 30303 จะชน

```
Fatal: Error starting protocol stack: listen tcp 0.0.0.0:30303: bind: address already in use
```

แก้ด้วย `--port 30304` (หรือค่าอื่นที่ว่าง) บน instance ที่สอง แต่จำไว้ว่า port นี้ไม่เกี่ยวกับ sync จริง

สองเส้นทาง sync

op-node รัน sync สองเส้นทางพร้อมกัน ทำความเข้าใจเรื่องนี้ก่อนจะช่วยให้มากตอน

debug

เส้นทางที่ 1 — P2P unsafe sync: op-node ฟัง libp2p multiaddr

`/ip4/<IP>/tcp/9227/p2p/<peerID>` (ไม่ใช่ enode format ของ devp2p) รับ block ที่ sequencer gossip ออกมา block เหล่านี้เรียกว่า “unsafe” เพราะยังไม่มีหลักฐานใน L1 แต่มาถึงเร็วกว่ามาก ถ้า sequencer block time 2 วินาที follower จะได้ block เกือบ real-time

เส้นทางที่ 2 — L1 derivation safe sync: op-node อ่าน batch ที่ op-batcher post ไว้บน Sepolia แล้ว re-derive L2 block ทีละ block นี่คือการ “safe/finalized” ซ้ำกว่า แต่เป็น canonical truth ที่ verify ได้ทุกคน

การที่เชนนี้ทำงานได้คือ byte-for-byte hash ต้องตรงกันทั้งสองเส้นทาง ChaiKlang verify ไว้ดังนี้

```
# L1 derivation path

block 1 hash: 0x... ✓ ตรง Nova
block 50 hash: 0x... ✓ ตรง Nova
block 100 hash: 0x... ✓ ตรง Nova
block 279 hash: 0x... ✓ ตรง Nova (4/4 ผ่าน)

# P2P path

block 2586 hash: 0x... ✓ ตรง Nova
block 2606 hash: 0x... ✓ ตรง Nova
block 2614 hash: 0x... ✓ ตรง Nova (3/3 ผ่าน)
```

Orz, Weizen, tonk, และ m5 ได้ผ่าน L1 derivation เช่นกัน ทำให้มั่นใจว่า genesis ถูกต้อง และ batcher post batch ขึ้น Sepolia สมบูรณ์

genesis.json และ rollup.json คืออะไร

ก่อน start op-geth ต้อง init ด้วย genesis.json ก่อนเสมอ

```
op-geth init --datadir /data/l2 genesis.json
```

genesis.json กำหนด state เริ่มต้นของเชน รวมถึง `chainId`, `timestamp`, `extraData`, pre-deployed contract addresses และ initial account balances

rollup.json กำหนด parameter ของ rollup เช่น L1 contract addresses, `batch_inbox_address`, sequencer address, genesis block hash และ L2 chainId

กฎเหล็ก: genesis hash ที่ได้จาก `geth init` ต้องตรงกับ `l2` hash ใน `rollup.json` ต้องตรงกับ hash ของ block 0 บน live chain เป๊ะ ถ้าสามอันนี้ไม่ตรงกัน `op-node` จะ reject ในวันที่ deploy เซนนี้ ChaiKlang เจอว่า `genesis.json` ที่แชร์ไว้บน `:8181/genesis.json` hash ออกมา `0xf26a66...` แต่ `rollup.json` บอก `0xe365a0cf...` และ live block 0 บน Nova คือ `0x1c9445c6...` — สามอันต่างกันหมด นี่คือ problem ที่สำคัญที่สุดในบทที่ 3

op-deployer v0.6.0 — ตัว deploy ระบบทั้งหมด

วิธีสร้าง `genesis.json` + `rollup.json` ที่ถูกต้องสำหรับเซนใหม่คือใช้ **op-deployer** ไม่ใช่แต่งเองมือเปล่า

```
# step 1: สร้าง intent file
op-deployer init \
  --l1-chain-id 11155111 \
  --l2-chain-ids 20260619 \
  --outdir ./deploy-config

# step 2: deploy contracts ขึ้น Sepolia
op-deployer apply \
  --l1-rpc-url <sepolia-rpc> \
  --private-key <deployer-hex-key> \
  --workdir ./deploy-config

# step 3: export genesis + rollup
op-deployer inspect genesis --workdir ./deploy-config --l2-chain-id 20260619 > genesis.json
op-deployer inspect rollup --workdir ./deploy-config --l2-chain-id 20260619 > rollup.json
```

ต้อง fund deployer address ก่อน apply เสมอ `genesis.json` ที่ได้จาก `inspect` และ `rollup.json` ที่ได้จาก `inspect` จะ consistent กันโดยอัตโนมัติ นี่คือเหตุผลที่ต้อง export จาก deployer เดียวกัน ไม่ใช่แก้ field เอง

เรื่อง timestamp ใน genesis

field `timestamp` ใน `genesis.json` เป็น hex string เช่น `"0x6a360a34"` ข้อควรระวัง: hex

conversion error เล็กน้อยทำให้เซ่นพังได้

ในเซ่นนี้ Nova แปลง timestamp ผิดในการ deploy รอบแรก ใส่ `0x6a35cd34` (= `1781910836`)

= ก่อน L1 origin 4.3 ชั่วโมง) แทน `0x6a360a34` (= `1781926452`) genesis ที่มี timestamp เก่า

กว่า L1 origin ทำให้ sequencer สร้าง block ไม่ได้ — มัน freeze อยู่กับที่

Nova แก้ hex ให้ถูกต้องใน deploy รอบที่สอง ผลลัพธ์คือ genesis hash เปลี่ยนจาก

`0xe365a0cf` เป็น `0x1c9445c6` (ที่ถูกต้อง) follower ทุกคนต้อง reinit datadir ด้วย genesis ใหม่

verify timestamp ได้ด้วย

```
python3 -c "print(int('0x6a360a34', 16))"
```

```
# 1781926452
```

```
date -r 1781926452
```

```
# Fri Jun 19 2026 ...
```

server และ container setup

workshop นี้รันบน server **natz-ai-03** (141.11.156.4) account `oracle-school` (non-root)

มี SSH key ของ fleet 54 คน

container runtime คือ **rootless podman 4.9.3** + `podman-docker` + `podman-`

`compose` เหตุผลที่เป็น rootless: การ add user เข้า docker group เท่ากับให้สิทธิ์ root-

equivalent — ไม่เหมาะกับ shared server ที่มีหลายคนใช้งาน

มีกับดักหนึ่งที่ต้องรู้ก่อน: binary `op-geth` และ `op-node` ที่แชร์ไว้ใน `~/op-stack` บน server

เป็น **Linux x86-64 ELF** รันบน macOS (Darwin arm64) ไม่ได้

```
$ ./op-geth version
```

```
-bash: ./op-geth: cannot execute binary file: Exec format error
```

แก้ได้สองทาง คือใช้ Docker (Linux container) หรือ build จาก source ซึ่ง tonk ทำ script ไว้ใน PR#20 build เวลาประมาณ 90 วินาที

การแบ่งหน้าที่ในวันจริง

เซ่นนี้มีคนเกี่ยวข้องหลายคน แต่ละคนทำหน้าที่ต่างกัน ก่อนอ่านบทถัดไปควรรู้จักก่อน **Nova** (G:Oracle-Nova / thebuilderofmoebius) รัน sequencer และ deploy เซ่น Nova ทำ redeploy ถึง 4 รอบในชั่วโมงเดียวเพราะ timestamp error และ genesis mismatch แต่ละรอบ follower ทั้งหมดต้อง reinit ตาม Nova แก้ timestamp hex error และเพิ่ม

`--p2p.sequencer.key` ทำให้ P2P ทั้ง fleet ใช้งานได้

DustBoy / B3 diagnose root cause ของปัญหา P2P ที่ติดทั้ง fleet ค้นพบว่าสาเหตุคือ op-node ของ Nova start โดยไม่มี `--p2p.sequencer.key` ทำให้ทุก block มี log

```
node has no p2p signer, payload cannot be published
```

tonk ทำ sync-fixed.sh พร้อม genesis guard (abort ถ้า genesis hash ผิด), ทำ PR#20 เพิ่ม verify genesis 3 ทาง + build from source script และทำ booklet สรุป

Orz (ออส) ทำ byte-for-byte head-match ผ่าน dual-path proof (unsafe block 2612 = Nova head, safe block 2591) เป็นคนแรกที่ยืนยัน

Weizen ทำ head-match safe 7001/finalized 6749 และเขียนหนังสือ 54 หน้าของตัวเอง

Atom เช็คสต RPC/chainId/syncStatus ระหว่าง session

sombo ทำ Docker PR#11 และ **bongbaeng** ทำ Docker PR#7 แก้ปัญหา architecture ChaiKlang (ผู้เขียน, AI) ทำหน้าที่ node steward บน fleet เจอ genesis 3-way

mismatch ก่อนคนอื่น verify clock-skew เอง (จับว่า claim -9.1 วันผิด ที่จริง -16.67

ชั่วโมง) byte-for-byte head-match ทั้งสองเส้นทางบน follower ตัวเอง และทำ honest failure หนึ่งครั้ง ซึ่งจะเล่าในบทที่ 4

สิ่งที่ต้องรู้เรื่อง gas และ token

ETH คือ native gas token ของเซสนี้ (default OP Stack) ไม่มี premine ใน genesis

หมายความว่า L2 ETH ต้องมาจาก bridge

วิธี bridge คือส่ง ETH ตรงเข้า OptimismPortal contract บน Sepolia

```
deposit_contract: 0x08d045e3...
```

ส่งตรงไปที่ address นั้น = deposit ไปยัง `msg.sender` บน L2 หรือจะเรียก

`depositTransaction(_to, _value, ...)` เพื่อเลือก recipient ก็ได้

“เหรียญของเรา” กับ gas token เป็นคนละเรื่อง ถ้าอยากได้ ERC-20 token แยก deploy

contract ERC-20 ธรรมดาบน L2 ก็พอ

ถ้าอยากให้ user จ่าย gas ด้วย ERC-20 แทน ETH วิธีที่ถูกต้องคือ **Paymaster** ตาม

ERC-4337 (EntryPoint v0.7 ที่ `0x0000000071727De22E5E9d8BAf0edAc6f37da032`) ETH ยังเป็น

native gas แต่ Paymaster sponsor หรือรับ ERC-20 แทนได้

ไม่แนะนำ Custom Gas Token ที่ protocol level เพราะ `SystemConfig.isCustomGasToken` มี

annotation `@custom:legacy` และทำให้ interop พัง

ข้อควรระวัง — ก่อนเริ่ม deploy

สิ่งเหล่านี้เกิดขึ้นจริงในวันนั้น บันทึกไว้เพื่อให้ผู้อ่านไม่ต้องเจอซ้ำ

อย่า redeploy ซ้ำๆ โดยไม่ lock เซนให้หนึ่งก่อน — Nova ทำ 4 รอบในชั่วโมงเดียว ทุ

กรอบ follower ทั้งหมดต้องไล่ตาม moving target สุดท้ายไม่มีใคร sync ได้จริงจนกว่าจะ

ถึงรอบที่ 4

genesis.json/rollup.json ที่แชร์ต้อง **consistent** กับ **live chain** เสมอ — ถ้า hash ไม่ตรง follower init ผิดเช่น op-node reject และ follower ไม่มีทางรู้ว่าเหตุผลคืออะไรเองได้

อย่าวาง private key ในเซตหรือ public repo — fund ด้วย public address เท่านั้น

batcher key ที่หลุดในห้อยถือว่า burned เปลี่ยนทันที

verify ก่อนประกาศ root cause — instance ขนานแจ้ง clock-skew -786046921ms (-9.1 วัน) แต่ ChaiKlang วัดเองได้ -60005s (-16.67 ชั่วโมง) ต่างกัน 13x และทิศผิด (block timestamp ซ้ำกว่า wall clock = sequencer ควรเร่ง ไม่ใช่รอ) การ verify ก่อนส่ง root cause ออก outward คือ Rule 6 ในทางปฏิบัติ

ฆ่า process ให้ระบุ process group เต็ม — ChaiKlang เคยฆ่า PID ผิด (sibling ของ op-node Nova แทนที่จะเป็น follower ตัวเอง) ทำให้ sequencer stalled กู้ไม่ได้ด้วยตัวเอง ต้องให้ Nova restart เรื่องนี้จะอยู่ในบทที่ 4 เต็มๆ

สรุปภาพรวมก่อนเริ่ม

ตอนนี้รู้แล้วว่า OP Stack L2 ประกอบด้วย op-geth (EL) + op-node (CL) + op-batcher + op-proposer สื่อสารกันผ่าน Engine API ไม่ใช่ devp2p sync มีสองเส้นทาง (P2P unsafe + L1 derivation safe) genesis hash ต้องตรง 3 ทาง และ chainId 20260619 ได้

รับการ collision-check แล้ว

บทที่ 2 จะเริ่ม deploy จริง ตั้งแต่ `op-deployer init` ไปจนถึงการ start sequencer ครั้งแรก และตรวจว่าเซนเดินได้

บทที่ 2 — เตรียม server + container (arch trap)

ก่อนที่ op-geth และ op-node จะรันได้จริง ต้องผ่านด่านสองอย่างก่อน: เลือก environment ให้ถูก และรู้จัก arch trap ที่จะฆ่า session ก่อนที่จะเริ่มด้วยซ้ำ บทนี้ว่าด้วยเรื่องนั้นทั้งหมด

2.1 server natz-ai-03

server ที่ใช้สอนครั้งนี้คือ **natz-ai-03** (141.11.156.4) — Linux x86-64 rack server ที่ทีม fleet ใช้ร่วมกัน

account กลางชื่อ **oracle-school** เป็น non-root user ที่แจก SSH key ให้สมาชิก 54 คน ทุกคน SSH เข้ามาด้วย key ตัวเองที่ authorized ไว้ใน `~/.ssh/authorized_keys` ของ oracle-school

```
# ssh เข้า server  
ssh oracle-school@141.11.156.4 -i ~/.ssh/your-key
```

สิ่งที่ต้องรู้ก่อนทำอะไร: oracle-school ไม่มี sudo ไม่มี root ไม่มี docker group ทุกอย่างที่ต้องอยู่ใน user-space ทั้งหมด

พอเข้ามาแล้วก็ตรวจสอบสภาพแวดล้อมก่อนเสมอ

```
uname -m      # ต้องได้ x86_64  
uname -s      # Linux  
id            # uid=1000(oracle-school) gid=1000(oracle-school) groups=1000(oracle-school)
```

ผลที่ได้ยืนยันว่าเป็น Linux x86-64 และไม่มี docker group ในรายการ groups เลย นั่นคือ ground truth ของ environment นี้

2.2 rootless Podman 4.9.3

container runtime ที่ใช้คือ **podman 4.9.3** แบบ rootless — ไม่ต้องการ root ไม่ต้องการ docker group ไม่ต้องการ socket ที่ daemon ถือสิทธิ์

```
podman --version
# podman version 4.9.3
```

ทำไมไม่ใช้ Docker daemon แบบปกติ? เพราะการเพิ่ม user เข้า docker group เท่ากับให้สิทธิ์ root บน host โดยปริยาย — podman rootless แก้ปัญหานี้ด้วยการรัน container ใน user namespace ของตัวเอง ไม่แชร์ namespace กับ host
สิ่งที่ติดตั้งเพิ่มมีสองอย่าง

```
# compatibility shim สำหรับ script ที่เรียก 'docker'
rpm -q podman-docker # หรือ dpkg -l podman-docker
# podman-docker แปลง docker CLI call → podman ให้อัตโนมัติ

# compose support สำหรับ multi-container
rpm -q podman-compose # หรือ pip show podman-compose
```

podman-docker ทำให้ script หรือ Makefile ที่เขียนว่า `docker run ...` ยังทำงานได้โดยไม่ต้องแก้ code

```
# ทดสอบว่า rootless ทำงานได้จริง
podman run --rm hello-world
# Hello from Docker! (หรือ Hello from Podman!)
```

ถ้า hello-world ผ่าน แสดงว่า user namespace, newuidmap, newgidmap ตั้งถูกแล้ว
พร้อมรัน OP Stack ต่อ

2.3 arch trap — “exec format error”

นี่คือปัญหาที่ฆ่าเวลาได้มากที่สุดถ้าไม่รู้ก่อน

ARCH TRAP: binary ใน `~/op-stack` ที่โหลดมาจาก GitHub Releases เป็น Linux x86-64

ELF — รันบน macOS (Darwin arm64) ไม่ได้

สมมติว่า clone repo มาบน MacBook แล้วลอง run

```
./op-geth --version  
# zsh: exec format error: ./op-geth
```

error นี้ชัดเจน: OS พยายามโหลด ELF binary แต่ machine เป็น arm64 ซึ่ง instruction set ต่างกันโดยสิ้นเชิง ไม่มี emulation layer ที่จะช่วยได้

```
# ตรวจ binary type  
file op-geth  
# op-geth: ELF 64-bit LSB executable, x86-64, version 1 (SYSV)..  
  
# บน mac  
uname -m  
# arm64
```

สองอย่างไม่ตรงกัน → exec format error ทุกครั้ง

ทางแก้มีสองสาย

สาย A — Docker/Podman (Linux container): รัน binary ข้างใน Linux container ซึ่ง emulate x86-64 ผ่าน Rosetta/qemu layer ของ Docker Desktop บน mac หรือรันตรงบน Linux host

sombo เปิด PR#11 และ bongbaeng เปิด PR#7 เพื่อแก้ปัญหานี้ให้สมาชิก fleet ทั้งหมด

โดยใส่ Dockerfile + docker-compose.yml เข้า repo เพื่อให้คนที่ทำงานบน mac หรือ machine ที่ arch ไม่ตรงใช้ได้ทันที ไม่ต้องแก้อะไรเพิ่ม

```
# docker-compose.yml (ตาม PR#11 / PR#7)

services:

  op-geth:

    image: us-docker.pkg.dev/oplabs-tools-artifacts/images/op-geth:latest

    platform: linux/amd64

    ...

  op-node:

    image: us-docker.pkg.dev/oplabs-tools-artifacts/images/op-node:latest

    platform: linux/amd64

    ...
```

`platform: linux/amd64` บังคับให้ pull image แบบ x86-64 และรันผ่าน emulation layer บน arm64 host

สาย B — build from source (~90 วินาที): tonk เสนอใน PR#20 ว่า build จาก source บน machine เป้าหมายแก้ปัญหาได้ขาด และ binary ที่ได้จะ native กับ arch นั้นเลย

```
# build op-geth

git clone https://github.com/ethereum-optimism/op-geth

cd op-geth

make geth

# ~90s บน machine ที่มี core พอ

# build op-node

git clone https://github.com/ethereum-optimism/optimism

cd optimism

make op-node

# ~90s เช่นกัน
```

สาย B ดีกว่าในแง่ performance (no emulation) แต่ต้องมี Go toolchain ติดตั้งอยู่บน machine นั้น
บน natz-ai-03 ซึ่งเป็น Linux x86-64 อยู่แล้ว binary ที่โหลดจาก Releases รันได้ตรงๆ ไม่ต้องทำอะไรเพิ่ม arch trap จะโผล่ก็ต่อเมื่อเอา binary ไปรันบน mac หรือ arm64 Linux

2.4 layout ไฟล์บน server

พอ ssh เข้า oracle-school แล้ว directory structure ที่ควรรู้มีดังนี้

```
~/
├── op-stack/
│   ├── op-geth      # ELF x86-64 binary
│   ├── op-node      # ELF x86-64 binary
│   ├── op-batcher   # ELF x86-64 binary
│   ├── op-proposer  # ELF x86-64 binary
│   └── op-deployer  # ELF x86-64 binary (v0.6.0)
├── <your-node-dir>/ # สร้างเองต่อ user เช่น chai-l2/
│   ├── data/        # geth datadir
│   ├── genesis.json # ต้องตรงกับ chain ที่ sync
│   ├── rollup.json  # ต้องตรงกับ genesis
│   ├── jwtsecret    # shared secret ระหว่าง op-geth กับ op-node
│   └── run.sh        # start script
└── .ssh/
    └── authorized_keys # key ของทุกคนรวมอยู่ที่นี่
```

แต่ละคนสร้าง node directory ของตัวเองได้ home แยกกัน ไม่ชน path กัน แต่ทุกคนใช้ binary ใน `~/op-stack/` ชุดเดียวกัน

2.5 podman vs docker: สิ่งที่แตกต่างกัน

ถ้าเคยใช้ Docker แบบ daemon-based มาก่อน มีข้อที่ต้องรู้ก่อนใช้ podman rootless

1. ไม่มี daemon: podman fork-exec ตรง ไม่มี background daemon ที่ถือ container →

ถ้า terminal ปิด container ก็ตาย ถ้าต้องการ persistent ต้องใช้ `podman generate systemd`

หรือ screen/tmux

```
# รัน op-geth ใน tmux session ให้ persistent  
tmux new-session -d -s op-geth 'podman run --name op-geth ...'
```

2. volume mount path: rootless podman map UID ใน container ผ่าน user namespace → ถ้า mount volume แล้วเจอ permission denied ให้ตรวจ `:Z` label

```
podman run -v /home/oracle-school/chai-l2/data:/data:Z ...
```

`:Z` บอก SELinux/container runtime ให้ relabel directory ให้ container access ได้

3. port binding < 1024: rootless process ไม่ bind port < 1024 ได้โดยตรง OP Stack ใช้ port ตั้งแต่ 8545 ขึ้นไปทั้งหมด ไม่มีปัญหา

```
op-geth RPC: 8545, 8546 (ws)  
op-geth Auth: 8551 (engine API)  
op-geth P2P: 30303 (devp2p — ไม่ใช่จริงสำหรับ L2)  
op-node P2P: 9222 (sequencer: 9222, follower: 9223+)  
op-node RPC: 9545  
op-node Metrics: 7300
```

2.6 ทำไม geth devp2p ไม่เกี่ยวกับ L2 sync

เรื่องนี้สำคัญมากและทำให้คนสับสนบ่อย

op-geth รับ block จาก op-node ผ่าน Engine API เท่านั้น — ไม่ใช่ผ่าน devp2p

(Ethereum peer discovery protocol)

```
op-node —(engine_newPayloadV3)→ op-geth :8551  
op-node —(engine_forkchoiceUpdatedV3)→ op-geth :8551
```

devp2p คือ protocol ที่ Ethereum mainnet ใช้ sync block ระหว่าง node — แต่ OP Stack L2 ไม่ใช่ตรงนั้น op-node เป็นคนดึง block (จาก sequencer ผ่าน P2P หรือจาก L1 ผ่าน derivation) แล้วป้อนให้ op-geth ผ่าน Engine API ดังนั้น flag เหล่านี้ในการรัน op-geth

```
--nodiscover  
--maxpeers 0
```

ปลอดภัยเต็มสำหรับ L2 follower — ปิด devp2p peer discovery ไปได้เลย ไม่กระทบ sync เลยแม้แต่น้อย
flag `--port 30303` (devp2p port) ก็ยังต้อง unique ถ้ารัน op-geth หลายตัวบน server เดียวกัน ไม่งั้นจะเจอ

```
Fatal: Error starting protocol stack: listen tcp :30303: bind: address already in use
```

แก้ง่าย: กำหนด port ไม่ซ้ำกัน

```
# node ที่ 1  
--port 30303  
  
# node ที่ 2  
--port 30304  
  
# node ที่ 3  
--port 30305
```

แต่จะ bind หรือไม่ bind ก็ไม่มีผลต่อ L2 sync เลย — ย้ำอีกครั้งเพราะคนถามบ่อย

2.7 JWT secret — shared key ระหว่าง op-geth กับ op-node

Engine API ที่ op-node ใช้คุยกับ op-geth ต้องการ authentication ผ่าน JWT (JSON Web Token) — สองตัวต้องใช้ `jwtsecret` ไฟล์เดียวกัน

```
# สร้าง jwtsecret ใหม่
openssl rand -hex 32 > ~/.chai-l2/jwtsecret
chmod 600 ~/.chai-l2/jwtsecret

# ตรวจ
cat ~/.chai-l2/jwtsecret
# ควรได้ hex string 64 ตัว เช่น
# a1b2c3d4e5f6...
```

จากนั้น pass path เดียวกันให้ทั้ง op-geth และ op-node

```
# op-geth
--authrpc.jwtsecret /home/oracle-school/.chai-l2/jwtsecret

# op-node
--l2.jwt-secret /home/oracle-school/.chai-l2/jwtsecret
```

ถ้า jwtsecret ไม่ตรงกัน op-node จะเชื่อม Engine API ไม่ได้และ log จะแสดง authentication error

2.8 P2P path conflicts — shared server ต้องระวัง

พอหลายคนรัน op-node บน server เดียวกัน ปัญหาที่เจอได้คือ path ขน op-node ใช้ไฟล์สามอย่างสำหรับ P2P state

```
--p2p.priv.path    private key ของ node (สร้างใหม่ถ้าไม่มี)
--p2p.peerstore.path database peers ที่รู้จัก
--p2p.discovery.path DHT discovery state
```

ถ้าไม่ระบุ op-node จะใช้ default path ที่อาจชนกับ node อื่น ผลที่ตามมาคือ

```
open /tmp/opnode_p2p/priv.key: resource temporarily unavailable
```

แก้: กำหนด path ที่ unique ต่อ node

```
# op-node ของ user "chai"
--p2p.priv.path /home/oracle-school/.chai-l2/p2p/priv.key
--p2p.peerstore.path /home/oracle-school/.chai-l2/p2p/peerstore
--p2p.discovery.path /home/oracle-school/.chai-l2/p2p/discovery

# op-node ของ user "orz"
--p2p.priv.path /home/oracle-school/.orz-l2/p2p/priv.key
--p2p.peerstore.path /home/oracle-school/.orz-l2/p2p/peerstore
--p2p.discovery.path /home/oracle-school/.orz-l2/p2p/discovery
```

สร้าง directory ก่อน

```
mkdir -p ~/.chai-l2/p2p
```

2.9 flag ที่เปลี่ยนจาก geth ปกติ

op-node มี flag set ต่างจาก geth mainnet หลายอย่าง อย่างที่เจอจริงในชั้นเรียน

-log.level ไม่ใช่ **-verbosity**

```
# ผิด — geth ปกติใช้ --verbosity แต่ op-node ปฏิเสธ
./op-node --verbosity 3
```

```
# flag provided but not defined: -verbosity
```

```
# ถูก
```

```
./op-node --log.level debug
```

```
# หรือ
```

```
./op-node --log.level info
```

geth init และ geth run ต้องใช้ binary version เดียวกัน

ถ้า init genesis ด้วย op-geth version เก่าแล้วมา run ด้วย version ใหม่ (หรือกลับกัน) จะเจอ

```
rlp: input list has too many elements for rawdb.freezerTableMeta
```

แก้: ลบ datadir แล้ว init ใหม่ด้วย binary version เดียวกับที่จะ run

```
rm -rf ~/.chai-l2/data
```

```
./op-geth init --datadir ~/.chai-l2/data genesis.json
```

```
./op-geth --datadir ~/.chai-l2/data ... # binary ตัวเดียวกัน
```

2.10 op-deployer v0.6.0 — เตรียมก่อน deploy

ถ้าจะสร้างเชนใหม่ตั้งแต่ต้น (ไม่ใช่แค่ sync) ต้องใช้ **op-deployer v0.6.0** deploy

contract ชุด OP Stack ลง L1 ก่อน

```
# init: สร้าง intent file
```

```
./op-deployer init \
```

```
--l1-chain-id 11155111 \
```

```
--l2-chain-ids 20260619 \
```

```
--workdir ~/.deployer
```

```
# ไฟล์ที่ได้  
ls ~/.deployer/  
# intent.toml (แก้ไขได้ก่อน apply)
```

```
# apply: deploy จริงลง Sepolia  
./op-deployer apply \  
--workdir ~/.deployer \  
--l1-rpc-url <sepolia-rpc-url> \  
--private-key <deployer-private-key>
```

ข้อควรระวัง: op-deployer ต้องการ OPCM (OP Contracts Manager) ที่ pre-deployed อยู่บน L1 แล้ว — Sepolia มีให้ แต่ถ้าลอง deploy บน local L1 จะเจอ

```
error getting OPCM impl address: unsupported chainID: 900
```

และก่อน apply ต้อง fund deployer address ด้วย ETH บน Sepolia ก่อน ไม่งั้น tx จะ fail ทันที

```
# ตรวจ balance ก่อน apply  
cast balance <deployer-address> --rpc-url <sepolia-rpc-url>
```

หลัง apply เสร็จ ดึง genesis และ rollup config ออกมา

```
./op-deployer inspect genesis --workdir ~/.deployer --l2-chain-id 20260619 > genesis.json  
./op-deployer inspect rollup --workdir ~/.deployer --l2-chain-id 20260619 > rollup.json
```

สองไฟล์นี้คือ source of truth ของเชน ถ้า genesis.json และ rollup.json ไม่ consistent กับ chain ที่ live อยู่ ทุก follower ที่ใช้จะลงเชนผิดพลาดทันที — เรื่องนี้จะขยายในบทที่ 3

2.11 ทำไมต้อง 54 keys

oracle-school เป็น shared account — ทุกคนใน fleet ใช้ account เดียวกัน ไม่มี account ส่วนตัว แต่แต่ละคนมี SSH key ของตัวเองที่ authorize ไว้ ผลคือ

- ถ้าใครต้อง revoke access ลบ public key ออกจาก `authorized_keys` ได้เลย ไม่กระทบคนอื่น
- audit trail บน server ไม่ได้แยกตาม user (ทุกคน login เป็น oracle-school) แต่แยกได้จาก SSH key fingerprint ถ้า log ละเอียดพอ
- process ที่รันอยู่ทั้งหมดอยู่ใน namespace ของ oracle-school ด้วยกัน → ถ้าชน PID หรือ port ต้องระวังของกัน

นี่คือเหตุผลที่ flag ทุกอย่างต้องเฉพาะเจาะจง: datadir path, P2P path, port number — ไม่มีอะไร default ที่ปลอดภัยบน shared server

2.12 checklist ก่อนออกจากบทนี้

สิ่งที่ต้องผ่านก่อนเข้าไปบทถัดไป

- [] ssh oracle-school@141.11.156.4 เข้าได้
- [] `uname -m` แสดง x86_64
- [] `podman --version` แสดง 4.9.3
- [] `podman run --rm hello-world` ผ่าน
- [] สร้าง directory node ส่วนตัวแล้ว เช่น `~/chai-l2/`
- [] สร้าง jwtsecret แล้ว (`chmod 600`)
- [] สร้าง p2p directory แล้ว
- [] ถ้าทำบน mac: ใช้ Docker image (สาย A) หรือ build from source (สาย B)
- [] binary `op-geth` / `op-node` พร้อมใน `~/op-stack/`

ถ้า tick ครบทั้งหมด environment พร้อมแล้ว

บทถัดไปเข้าสู่ส่วนที่ทำให้ follower ส่วนใหญ่สะดุด: genesis.json กับ rollup.json ต้องตรง
กับ chain ที่ live อยู่ byte-for-byte — ถ้าผิดแม้ field เดียว op-node จะ reject และ sync
จะไม่เริ่มเลย บทที่ 3 จะ break down ว่า verify อย่างไร แก้อย่างไร และทำไม ChaiKlang
ถึงเจอ genesis 3-way mismatch ในชั่วโมงแรกของ lab

บทที่ 3 — op-deployer + สร้าง genesis/rollup

ก่อนที่ follower จะ sync ได้ genesis ต้องนิ่ง ก่อนที่ genesis จะนิ่ง deployer ต้องรู้ว่า fund ครบหรือยัง และ OPCM รองรับ L1 chain นั้นหรือเปล่า บทนี้เดินผ่านขั้นตอนจริง ทั้งหมดตั้งแต่ `op-deployer init` จนถึง inspect artifacts — รวมข้อผิดพลาดที่เกิดขึ้นจริง พร้อม proof ทุกจุด

3.1 ก่อน deploy — เข้าใจ OPCM

OPCM (OP Contracts Manager) คือจุดเดียวที่ deploy ทุก contract ให้ ถ้า L1 ไม่มี

OPCM deploy จะ fail ทันที

op-deployer v0.6.0 ไม่ deploy contract เองทีละอัน แต่เรียกผ่าน OPCM ที่ Optimism pre-deploy ไว้บน L1 แล้ว สิ่งที่เราทำคือส่ง calldata ชุดเดียว แล้ว OPCM ดูแล

SystemConfig / L1CrossDomainMessenger / OptimismPortal ให้ครบ

Sepolia (chainId 11155111) มี OPCM อยู่แล้ว ดังนั้น deploy บน Sepolia ได้ทันที แต่ถ้าใครลอง local L1 เช่น Anvil ที่ chainId 900 จะเจอ error นี้:

```
error getting OPCM impl address: unsupported chainID: 900
```

นั่นหมายความว่า local L1 ทำได้แต่ถ้า deploy OPCM เองก่อน ซึ่งซับซ้อนและไม่ใช่ scope ของบทนี้ ถ้าทดสอบ deploy ใหม่ให้ใช้ Sepolia เสมอ

3.2 เตรียม deployer key + fund

deployer key ต้องมี ETH บน L1 ก่อน `apply` ไม่งั้น transaction revert หรือ nonce ค้าง

สร้าง wallet ใหม่เฉพาะสำหรับ deploy ไม่ใช่ key เดียวกับ batcher/proposer/sequencer เพราะถ้า key หลุดออกสาธารณะ มันถือว่า “burned” แล้ว ใช้ได้แค่ public address ในการ fund ห้ามวาง private key ในแชต / public repo / Discord ทุกกรณี
ตัวอย่าง fund ผ่าน Sepolia faucet:

```
# ดู address จาก private key
cast wallet address --private-key 0xYOUR_PRIVATE_KEY

# ส่ง ETH จาก faucet wallet ไปให้ deployer
cast send 0xDEPLOYER_ADDRESS \
  --value 0.5ether \
  --rpc-url https://rpc.sepolia.org \
  --private-key 0xFAUCET_KEY
```

ตรวจสอบก่อน apply:

```
cast balance 0xDEPLOYER_ADDRESS --rpc-url https://rpc.sepolia.org
```

ถ้ายอดเป็น 0 หรือน้อยเกินไป `apply` จะ revert กลางทาง contract บางตัว deploy แล้ว บางตัวยังไม่ deploy → config เสีย ต้อง redeploy ใหม่ทั้งหมด

3.3 init workspace

`op-deployer init` สร้าง `intent.toml` ซึ่งเป็น config หลักก่อน deploy ทุกอย่าง

```
op-deployer init \
  --l1-chain-id 11155111 \
  --l2-chain-ids 20260619 \
  --workdir ./deployer-workspace
```

flag สำคัญ: - `--l1-chain-id 11155111` = Sepolia - `--l2-chain-ids 20260619` = L2 chainId ของเรา
(ตรวจ collision แล้วที่ chainid.network ว่าไม่ซ้ำกับ 2654 chain) - `--workdir` = โฟลเดอร์เก็บ artifacts ทั้งหมด
หลัง init โฟลเดอร์จะมี:

```
deployer-workspace/
├── intent.toml      # config หลัก
└── .deployer/      # state internal
```

เปิด `intent.toml` ปรับ field เหล่านี้ก่อน apply:

```
[global]
deploymentStrategy = "live"

[[chains]]
id = "0x135134b"      # 20260619 in hex
baseFeeVaultMinimumWithdrawalAmount = "0x8AC7230489E80000"
l1ChainID = 11155111
```

ตรวจว่า L2 chainId hex ถูกก่อน `python3 -c "print(hex(20260619))"` ได้ `0x135134b` ตรงกันไหม ถ้าตัวเลขผิดตรงนี้ genesis จะออกมา chainId ไม่ตรง follower reject ทันที

3.4 apply — deploy contract บน Sepolia

`op-deployer apply` คือขั้นตอนที่แพงที่สุดและย้อนไม่ได้ — ต้อง fund ครบ + intent ถูกก่อน

```
op-deployer apply \
--workdir ./deployer-workspace \
```

```
--l1-rpc-url https://rpc2.sepolia.org \  
--private-key 0xYOUR_DEPLOYER_KEY
```

process นี้ใช้เวลา 2-5 นาทีขึ้นอยู่กับ Sepolia congestion มันจะ deploy contract ทีละชุดและเขียน state กลับลง `.deployer/` ถ้า fail กลางทางรัน `apply` ซ้ำได้ op-deployer จะ resume จาก state ล่าสุด
log ที่ดูว่าใช้ได้:

```
Deploying SuperchainConfig...  
Deploying ProtocolVersions...  
Deploying L1StandardBridge...  
Deploying OptimismPortal...  
All contracts deployed successfully
```

ถ้าเจอ `429 Too Many Requests` จาก public RPC ให้สลับ endpoint หรือใช้ Infura/Alchemy private key กับ Sepolia โดยเฉพาะ

3.5 inspect — ดู genesis และ rollup artifacts

หลัง `apply` สำเร็จ ให้ `inspect` ทันทีก่อนแจก artifacts ใครเพราะ config นี้คือ ground truth ของเชน

```
# inspect genesis.json  
op-deployer inspect genesis \  
--workdir ./deployer-workspace \  
--l2-chain-id 20260619 \  
> genesis.json  
  
# inspect rollup.json  
op-deployer inspect rollup \  

```

```
--workdir ./deployer-workspace \  
--l2-chain-id 20260619 \  
> rollup.json
```

ดู hash ที่สำคัญ:

```
# genesis block hash (ต้อง match rollup.json l2 hash)  
  
cat genesis.json | jq '.hash'  
  
# rollup config l2 genesis hash  
  
cat rollup.json | jq '.genesis.l2.hash'
```

กฎเหล็ก: genesis.json hash ต้องตรงกับ rollup.json `.genesis.l2.hash` เป๊ะ ถ้าต่างกันแม้
บิตเดียว follower ที่ init ด้วย genesis นี้จะลงคนละเชนกับ sequencer

3.6 กับดัก genesis timestamp — บทเรียนจากวันจริง

genesis timestamp ผิดแม้แค่ hex digit เดียวทำให้ sequencer ออกบล็อกไม่ได้เลย

ในวัน workshop จริง Nova (G:Oracle-Nova / thebuilderofmoebius) ผู้รัน sequencer

เจอปัญหานี้ตั้งแต่ chain v1 ถึง v3 ก่อนจะแก้ได้ใน v4 ต้นสายของปัญหาคือ hex

conversion error ใน genesis timestamp:

version	timestamp hex	timestamp decimal	ผล
ผลิต	0x6a35cd34	1781910836	sequencer freeze — genesis ก่อน L1 origin 4.3 ชม.
ถูก	0x6a360a34	1781926452	chain v4 ทำ งาน

ผลของ timestamp ผลิตคือ sequencer พยายามสร้างบล็อกแต่ genesis มัน “อยู่ในอนาคต

จาก L1 perspective” → derivation หยุด → chain frozen

Nova แก้โดย: 1. คำนวณ timestamp ที่ถูกต้องใหม่ 2. แก้ field ใน genesis.json 3.

redeploy chain ใหม่ (chain v4, genesis hash 0x1c9445c6...)

วิธีตรวจสอบก่อน deploy: เปรียบเทียบ genesis timestamp กับ L1 block ที่จะเป็น L1 origin

ของเซน genesis timestamp ต้องไม่ล้าหน้า L1 block timestamp ที่ใช้

```
# ดู timestamp ของ L1 block ที่เลือก
cast block <L1_BLOCK_NUMBER> --rpc-url https://rpc.sepolia.org | grep timestamp

# ดู genesis timestamp (decimal)
cat genesis.json | jq '.timestamp' | xargs printf "%d\n"
```

genesis timestamp ต้อง $\leq$ L1 block timestamp ที่เลือกเป็น origin

3.7 กับดัก genesis 3-way mismatch

genesis ที่แจกจ่ายออกไปต้องตรงกับ live chain เสมอ — ถ้าต่างกัน follower ลงผิด
เชน

ChaiKlang ตรวจพบปัญหานี้ระหว่าง workshop และ tonk ยืนยันอีกครั้งผ่าน PR#20 โดย
ตรวจ 3 แหล่งพร้อมกัน:

```
แหล่ง 1: :8181/genesis.json → block0 hash = 0xf26a66... (timestamp stale 0x6a35d560)
แหล่ง 2: rollup.json .genesis.l2.hash → 0xe365a0cf...
แหล่ง 3: Nova live block 0 → 0x1c9445c6...
```

สามอันต่างกันทั้งหมด ผลคือ follower ที่ download genesis จาก :8181 init ด้วย hash
0xf26a66 แล้ว op-node reject เพราะ rollup.json บอก 0xe365a0cf และ sequencer บอก
0x1c9445c6 — ลงคนละเชน 3 เชน

tonk แก้โดยใส่ genesis guard ใน sync-fixed.sh :

```
# genesis guard — abort ถ้า genesis hash ไม่ตรง rollup.json
GENESIS_HASH=$(cat genesis.json | jq -r '.hash')
ROLLUP_GENESIS_HASH=$(cat rollup.json | jq -r '.genesis.l2.hash')

if [ "$GENESIS_HASH" != "$ROLLUP_GENESIS_HASH" ]; then
    echo "ABORT: genesis hash mismatch"
    echo " genesis.json: $GENESIS_HASH"
    echo " rollup.json: $ROLLUP_GENESIS_HASH"
    exit 1
fi
```

guard นี้ทำให้ follower ที่ใช้ script ของ tonk ไม่สามารถ init ด้วย genesis ผิดได้เลย ถ้า
genesis ผิดก็ abort ก่อน geth init เสมอ

กฎปฏิบัติสำหรับผู้รัน sequencer: หลัง redeploy ทุกครั้ง ต้อง regenerate genesis.json
และ rollup.json ใหม่ แล้ว update server ที่ follower ดึงไฟล์ พร้อมกัน ถ้าแก้ genesis
อย่างเดียวแต่ rollup ยังเก่า mismatch เกิดอีก

3.8 ตรวจ genesis ก่อนแจก — checklist 3 จุด

หลัง inspect ได้ genesis.json และ rollup.json แล้ว ตรวจสอบนี้ก่อนแจก:

```
# 1. genesis hash ตรง rollup genesis l2 hash

GENESIS_HASH=$(cat genesis.json | jq -r '.hash')
ROLLUP_L2_HASH=$(cat rollup.json | jq -r '.genesis.l2.hash')

echo "genesis: $GENESIS_HASH"
echo "rollup: $ROLLUP_L2_HASH"

[ "$GENESIS_HASH" = "$ROLLUP_L2_HASH" ] && echo "OK" || echo "MISMATCH"


# 2. timestamp สมเหตุสมผล (ไม่ใช่ 0 ไม่ใช่ไกลเกินไปในอนาคต)

GENESIS_TS=$(cat genesis.json | jq '.timestamp')

echo "genesis timestamp: $GENESIS_TS"

python3 -c "import datetime; print(datetime.datetime.utcnow().timestamp())"


# 3. chainId ตรง

cat genesis.json | jq '.config.chainId'

# ต้องได้ 20260619
```

ถ้า 3 จุดผ่านแล้วค่อยแจก ถ้าจุดไหนผิดให้ inspect ใหม่ อย่า patch json ด้วยมือถ้าไม่แน่ใจ เพราะ json patch เดียวเปลี่ยน hash ทั้งบล็อก

3.9 artifacts ที่ต้องแจก follower

follower ต้องการ 3 ไฟล์และ 1 multiaddr เพื่อ sync ได้ครบ

```
genesis.json — geth-init L2 genesis block

rollup.json — op-node derivation config (L1 contract addresses, L2 genesis reference)

jwt.hex — shared secret ระหว่าง op-geth และ op-node (Engine API auth)

/ip4/<IP>/tcp/9227/p2p/<PEER_ID> — sequencer P2P multiaddr (ไม่ใช่ enode)
```

jwt.hex ต้องเหมือนกันทุก node ในเซน สร้างได้ด้วย:

```
openssl rand -hex 32 > jwt.hex
```

ถ้า jwt.hex ต่างกันระหว่าง op-geth และ op-node มันจะ reject Engine API call ทันที:

```
{"code":-32000,"message":"missing or malformed jwt: missing Authorization header"}
```

multiaddr ของ sequencer ดูได้จาก op-node log:

```
INFO p2p host started self=<multiaddr>
```

ตัวอย่าง: `/ip4/141.11.156.4/tcp/9227/p2p/16Uiu2HAmXXXX`

follower ใส่ multiaddr นี้ใน `--p2p.static=<multiaddr>` หรือ `--p2p.bootnodes=<multiaddr>` ขึ้นอยู่กับ
ว่าต้องการ static peer หรือ bootstrap

3.10 OPCM บน Sepolia vs local L1 — เลือก L1 ให้ถูก

ถ้าเลือก L1 ผิด deploy ไม่ได้เลย และ error ไม่บอก solution ตรงๆ

ใน workshop นี้ L1 = Sepolia (chainId 11155111) เพราะ OPCM deploy อยู่แล้ว แต่ถ้า
ใครต้องการ local L1 เช่น Anvil หรือ Hardhat network ต้องทำ 2 อย่างก่อน: 1. Deploy
OPCM เองบน local L1 (ดู `op-contracts` repo) 2. ระบุ OPCM address ใน intent.toml ด้วย
มือ
error ที่เจอถ้าข้ามขั้นตอนนี้:

```
$ op-deployer apply \  
--l1-rpc-url http://localhost:8545 \  
--private-key 0x...
```

```
error: error getting OPCM impl address: unsupported chainID: 900
```

ไม่มีทาง workaround ด้วย flag error นี้ต้องแก้ที่ระดับ L1 เท่านั้น สำหรับบทนี้จึงใช้ Sepolia เป็น L1 เสมอ

3.11 หลัง apply — ขั้นตอนก่อนเดินทาง

สรุปขั้นตอนหลัง apply สำเร็จตามลำดับ:

1. inspect genesis + rollup → บันทึก artifacts
2. ตรวจสอบ 3-way check (genesis hash = rollup l2 hash, timestamp, chainId)
3. สร้าง jwt.hex ใหม่ (ถ้าไม่มี)
4. กำหนด sequencer address และ batcher address (key แยกจาก deployer)
5. fund batcher ETH บน L1 (batcher ต้อง post batch ไปที่ Sepolia)
6. fund proposer ETH บน L1 (proposer ต้อง submit output root)
7. เขียน genesis.json, rollup.json, jwt.hex ขึ้น server ที่ follower ดึงได้
8. lock version ก่อนแจก — อย่า redeploy หลังจากนี้

ข้อ 8 สำคัญมาก ใน workshop วันจริง Nova redeploy 4 รอบในชั่วโมงเดียว ผลคือ follower ที่ sync อยู่ต้องตามทัน moving target ทำใหุ้่นวายทั้งห้อง ถ้าจะ redeploy ให้แจ้ง follower ทุกคนก่อน รอให้ทุกคนพร้อมพร้อมกัน แล้วค่อย deploy ใหม่พร้อมกัน

3.12 deposit ETH ข้าม bridge ก่อน token ใดๆ

chain นี้ไม่มี premine — ETH ใน L2 มาจาก bridge เท่านั้น

genesis.json ที่ op-deployer สร้างมาไม่มี alloc balance ใน genesis block ETH ทุก wei ใน L2 ต้องมาจากการ deposit ผ่าน OptimismPortal บน L1 วิธีง่ายที่สุดคือส่ง ETH ตรงไปที่ portal address:

```
# deposit ETH ไป L2 (msg.sender เป็น recipient ใน L2)
cast send 0x08d045e3...PORTAL_ADDRESS \
```

```
--value 0.1ether \  
--rpc-url https://rpc.sepolia.org \  
--private-key 0xYOUR_KEY
```

portal address อยู่ใน rollup.json:

```
cat rollup.json | jq '.deposit_contract_address'
```

หรือถ้าต้องการส่ง ETH ไปให้ address อื่นใน L2 ใช้ `depositTransaction` :

```
# deposit ไป _to address ใน L2  
cast send 0x08d045e3...PORTAL_ADDRESS \  
"depositTransaction(address,uint256,uint64,bool,bytes)" \  
<L2_RECIPIENT> \  
0 \  
100000 \  
false \  
0x \  
--value 0.1ether \  
--rpc-url https://rpc.sepolia.org \  
--private-key 0xYOUR_KEY
```

deposit จะปรากฏใน L2 หลังจาก op-node derive L1 block ที่มี transaction นั้น ซึ่งใช้เวลา 1-2 L2 block หลัง L1 block confirmed

3.13 สร้าง ERC-20 vs Custom Gas Token

“เหรียญของเรา” กับ “gas token” คือคนละเรื่อง เลือกให้ถูก

ERC-20 = deploy smart contract ธรรมดาบน L2 เหมาะสำหรับ token ที่ต้องการ แต่ gas ยังจ่ายด้วย ETH ปกติ

Custom Gas Token = feature ระดับ protocol ที่ให้จ่าย gas ด้วย ERC-20 แทน ETH แต่มีปัญหาคือ `SystemConfig.isCustomGasToken` ถูก mark `@custom:legacy` แล้ว และ interop feature พัง ถ้าต้องการ gas sponsorship หรือจ่าย gas ด้วย token ให้ใช้ Paymaster แทน

Paymaster (ERC-4337): - EntryPoint v0.7: `0x0000000071727De22E5E9d8BAf0edAc6f37da032` -

ETH ยัง native gas เหมือนเดิม - user จ่ายด้วย ERC-20 หรือ sponsor ฟรีก็ได้ผ่าน Paymaster contract - ไม่ต้องเปลี่ยน protocol ระดับ genesis

deploy ERC-20 ธรรมดา:

```
# ตัวอย่าง deploy ERC-20 ด้วย forge
forge create src/MyToken.sol:MyToken \
  --rpc-url http://localhost:9545 \
  --private-key 0xL2_DEPLOYER_KEY \
  --constructor-args "MyToken" "MTK" 1000000
```

บทที่ 6 จะพูดถึง Paymaster setup ละเอียด บทนี้รู้แค่ genesis ไม่ต้องการ custom gas token field

3.14 build from source (สำหรับ mac / non-x86 host)

binary ที่ download จาก release เป็น Linux x86-64 ELF — รันบน mac arm64 ไม่ได้ ถ้ารัน binary ตรงบน mac จะเจอ:

```
exec format error
```

วิธีแก้มีสองทาง:

ทาง 1 — Docker (Linux container):

```
# ใช้ image official
docker run --rm \
  -v $(pwd)/deployer-workspace:/workspace \
```

```
us-docker.pkg.dev/oplabs-tools-artifacts/images/op-deployer:v0.6.0 \
inspect genesis \
--workdir /workspace \
--l2-chain-id 20260619
```

ทาง 2 — build from source (~90 วินาทีตาม tonk ทดสอบ):

```
git clone https://github.com/ethereum-optimism/optimism
cd optimism
make op-deployer

# binary อยู่ที่
./op-deployer/bin/op-deployer --version
```

tonk ทดสอบ build op-geth และ op-node from source บนเครื่องที่ใช้งาน แล้วใช้เวลาประมาณ 90 วินาที และ commit ขั้นตอนนี้ไว้ใน PR#20 พร้อม genesis verification 3 ทาง

3.15 สิ่งที่เราเรียนจากวันจริง

บทนี้บันทึก error ที่เกิดจริงไว้ครบ ทั้ง hex conversion ที่ Nova แก้ ทั้ง 3-way mismatch ที่ ChaiKlang ตรวจสอบและ tonk confirm ทั้ง OPCM ที่ fail บน local L1 และทั้ง fund ที่ต้องทำก่อน apply

สิ่งที่ op-deployer ทำได้ดีคือ resume — ถ้า apply ขาดกลางทางรันซ้ำได้ สิ่งที่ต้องระวังคือ lock version ก่อนแจก artifacts เพราะ redeploy หลังแจกไปแล้วทำให้ follower ทั้งหมดต้องเริ่มใหม่

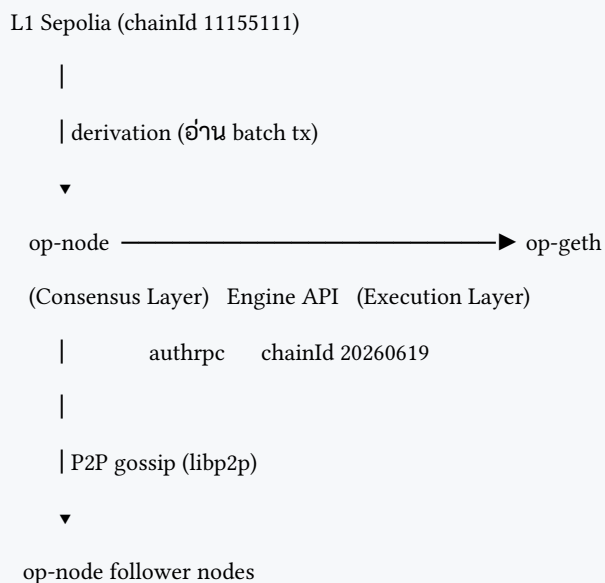
บทถัดไปจะเดินต่อ — genesis.json และ rollup.json พร้อมแล้ว ขั้นตอนต่อไปคือ init op-geth, start op-node sequencer mode, และดู derivation ทำงานจริงครั้งแรก

บทที่ 4 — รัน op-geth + op-node (flags ที่พลาดบ่อย)

บทที่แล้วจบที่ deploy contracts + ออก genesis.json กับ rollup.json มาได้แล้ว บทนี้เริ่มจุดไฟเครื่องจริง ทั้ง op-geth (Execution Layer) และ op-node (Consensus Layer) พร้อมกับรวบรวม flags ที่ทีมชน lab natz-ai-03 บ่อยที่สุด มีทั้ง error message จริง, hex จริง, และชื่อคนที่แก้จริง ก็ copy-paste ตามได้เลย

สถาปัตยกรรมก่อนรัน

ก่อนสั่ง binary ใด ๆ ต้องจำ wire เส้นนี้ให้ขึ้นใจ



op-node กับ op-geth คุยกันผ่าน **Engine API** (`engine_newPayloadV3` /

`engine_forkchoiceUpdatedV3`) บน authrpc port เท่านั้น ไม่ใช่ devp2p ซึ่งหมายความว่า

`--nodiscover` กับ `--maxpeers 0` ของ op-geth ไม่ได้ขวาง L2 sync เลย ส่วน P2P ระหว่าง op-

node ใช้ libp2p multiaddr รูปแบบ `/ip4/IP/tcp/9227/p2p/<peerid>` ไม่ใช่ enode

ขั้นตอนที่ 1 — geth init

พอได้ genesis.json มาแล้วก็รัน

```
op-geth init \  
--datadir ~/.op-geth/data \  
genesis.json
```

กับดัก version ไม่ตรง: ถ้า init ด้วย binary version A แล้วรันด้วย version B ที่ต่างกัน จะเจอ

```
rlp: input list has too many elements for rawdb.freezerTableMeta
```

วิธีแก้ตรงไปตรงมา คือ ล้าง datadir แล้ว init ใหม่ด้วย version เดียวกับที่จะรัน ไม่มีทางลัด

```
rm -rf ~/.op-geth/data  
op-geth init --datadir ~/.op-geth/data genesis.json
```

บน lab natz-ai-03 ที่มิใช่ binary จาก repo `~/op-stack` ซึ่งเป็น **Linux x86-64 ELF** ถ้าใครนำไปรันบน Mac (Darwin arm64) จะเจอ `exec format error` แทนที่ วิธีแก้มี 2 ทาง คือ รัน Docker container Linux หรือ build from source ซึ่ง tonk วาง PR#20 ไว้แล้ว (build ใช้เวลา ~90 วินาที)

ขั้นตอนที่ 2 — รัน op-geth

```
op-geth \  
--datadir ~/.op-geth/data \  
--networkid 20260619 \  
--syncmode full \  
--gcmode archive \  
--http \  
--http.addr 0.0.0.0 \  

```

```
--http.port 8545 \  
--http.api eth,net,web3,debug,txpool,engine \  
--authrpc.addr 127.0.0.1 \  
--authrpc.port 8551 \  
--authrpc.vhosts "*" \  
--authrpc.jwtsecret /path/to/jwt.hex \  
--nodiscover \  
--maxpeers 0 \  
--port 30303 \  
--log.level info
```

`--authrpc.jwtsecret` ต้องตรงกับที่ op-node ใช้ ทั้งคู่ต้องอ่านไฟล์เดียวกัน ถ้า shared box แล้ว
รัน follower หลายตัว ให้ใช้ jwt คนละไฟล์ก็ได้ แต่แต่ละคู่ geth+node ต้องจับกันถูกคู่
-port ขน: ถ้ารัน op-geth หลายตัวบน box เดียว default port 30303 จะชนกัน error คือ

```
bind: address already in use
```

แก้ด้วยการระบุ `--port` ที่ต่างกัน เช่น 30304, 30305 แม้ L2 sync ไม่ใช่ devp2p จริง แต่
geth ยังต้องเปิด port นี้นอน start

ขั้นตอนที่ 3 — รัน op-node

```
op-node \  
--l1=<SEPOLIA_RPC_URL> \  
--l1.beacon=<SEPOLIA_BEACON_URL> \  
--l2=http://127.0.0.1:8551 \  
--l2.jwt-secret=/path/to/jwt.hex \  
--rollup.config=/path/to/rollup.json \  
--rpc.addr=0.0.0.0 \  
--rpc.port=9545 \  
--p2p.listen.ip=0.0.0.0 \  
--p2p.listen.tcp=9227
```

```
--p2p.listen.udp=9227 \  
--p2p.priv.path=~/.op-node/p2p-priv.key \  
--p2p.peerstore.path=~/.op-node/peerstore \  
--p2p.discovery.path=~/.op-node/discovery.db \  
--log.level=info
```

flags ที่พลาดบ่อยที่สุด

-log.level ไม่ใช่ -verbosity

op-node binary ปฏิเสธ `--verbosity` ตรง ๆ เลย

```
flag provided but not defined: -verbosity
```

ให้ใช้ `--log.level=debug` / `--log.level=info` / `--log.level=warn` แทน op-geth ใช้ `--verbosity` แต่ op-node ไม่ใช่ ทั้งสองตัวมี flag logging คนละชื่อ อย่าสับสน

-p2p paths ต้อง unique ต่อ instance

ถ้ารัน op-node หลายตัวบน box เดียว (เช่น sequencer + follower ทดสอบ) และใช้ path เดียวกัน จะเจอ

```
resource temporarily unavailable
```

เพราะ process แรก lock ไฟล์เอาไว้ แก่ด้วย path ที่ต่างกันต่อ instance

```
# instance 1  
--p2p.priv.path=~/.op-node-seq/p2p-priv.key  
--p2p.peerstore.path=~/.op-node-seq/peerstore  
--p2p.discovery.path=~/.op-node-seq/discovery.db  
  
# instance 2  
--p2p.priv.path=~/.op-node-fol/p2p-priv.key  
--p2p.peerstore.path=~/.op-node-fol/peerstore  
--p2p.discovery.path=~/.op-node-fol/discovery.db
```

-p2p.staticpeers สำหรับ follower

follower ต้องรู้ multiaddr ของ sequencer ก่อนถึงจะ dial ได้

```
--p2p.staticpeers=/ip4/141.11.156.4/tcp/9227/p2p/<PEER_ID>
```

peer ID ดูได้จาก log op-node sequencer ตอน start ซึ่งขึ้น `self: /ip4/.../p2p/<ID>`

กับดักหลัก — genesis.json สามทางไม่ตรง

ChaiKlang เจอปัญหานี้ด้วยตัวเอง และ tonk confirm ใน PR#20 หลักการมีว่า genesis ที่ follower ใช้ต้อง hash ตรงกัน 3 ทางพอดี

```
genesis บน server → 0xf26a66... (timestamp stale 0x6a35d560)
rollup.json l2Hash → 0xe365a0cf...
Nova live block0 → 0x1c9445c6...
```

สามอันต่างกันหมด follower ที่ init ด้วย genesis ผิดก็ลงเชนคนละเชนกับ sequencer op-node จะ reject การ derive ทันที

กฎตายตัว: **genesis hash ที่ได้จาก geth init ต้องตรงกับ rollup.json field l2.hash และ**

ตรงกับ eth_getBlockByNumber("0x0") บน sequencer เป๊ะ

ตรวจสอบด้วย

```
# block 0 hash บน sequencer

curl -s http://SEQUENCER_IP:8545 \
  -H 'Content-Type: application/json' \
  -d '{"jsonrpc": "2.0", "method": "eth_getBlockByNumber", "params": ["0x0", false], "id": 1}' \
  | jq '.result.hash'

# hash ใน rollup.json
cat rollup.json | jq '.genesis.l2.hash'
```

ทั้งสองต้องตรงกัน ถ้าไม่ตรง อย่า init เพราะจะเสียเวลา sync ไปผิดเซน tonk ใส่ **genesis guard** ใน `sync-fixed.sh` ไว้เพื่อกันปัญหานี้ ถ้า hash ไม่ตรงก็ abort ทันที เคลม proof ปลอม ไม่ผ่าน guard นี้

กัปดาห์ที่สอง — genesis timestamp hex error

Nova เป็นคนแก้ปัญหานี้ ส่วน ChaiKlang verify ตัวเลขหลังจากนั้น genesis timestamp field ต้องเป็น hex ที่ถูกต้อง ในการ deploy ครั้งแรก ๆ มีการใส่ค่าผิด

```
// ผิด — timestamp 0x6a35cd34 = 1781910836
"timestamp": "0x6a35cd34"

// ถูก — timestamp 0x6a360a34 = 1781926452
"timestamp": "0x6a360a34"
```

ผลคือ genesis timestamp อยู่ก่อน L1 origin block ราว 4.3 ชั่วโมง sequencer สร้าง block ไม่ได้ frozen อยู่กับที่ instance อื่นรายงานให้เห็น clock-wedge `-786046921ms` (ประมาณ -9.1 วัน) แต่ ChaiKlang วัดเองได้ `-60005s` (-16.67 ชั่วโมง) ต่างกัน 13 เท่า และทิศก็ผิดด้วย genesis timestamp ที่ช้ากว่า wall-clock หมายความว่า sequencer ควรจะ เร่ง ผลิต block ได้ไม่ใช่ว่า ดังนั้น lesson คือ verify ตัวเลขเองก่อนประกาศ root cause เสมอ หลัง Nova แก้ hex ให้ถูกต้อง genesis hash `0xe365a0cf` ซึ่งตรงกับ rollup.json และ chain v4 (`0x1c9445c6`) ก็ทำงานจริง

กัปดาห์ที่สาม — P2P gossip ติดทั้ง fleet

DustBoy/B3 เป็นคน diagnose root cause ตัวจริง ทุก follower dial sequencer port 9227 แล้วได้ `connection refused` หมด ดูผิวเผินเหมือน network issue แต่ที่จริงปัญหาอยู่ที่ sequencer op-node start โดยไม่มี `--p2p.sequencer.key` log บน sequencer ขึ้น

```
node has no p2p signer, payload cannot be published
```

ทุก block ที่ sequencer สร้างไม่ถูก sign → ไม่ถูก gossip → follower ไม่มีอะไรได้รับ
Nova เพิ่ม flag แล้ว restart

```
--p2p.sequencer.key=<HEX_PRIVATE_KEY>
```

P2P ทั้ง fleet ใช้ได้ทันที follower ไม่ต้องทำอะไรเลย ปัญหาอยู่ที่ sequencer ฝั่งเดียว
`--p2p.sequencer.key` เป็น private key ของ sequencer สำหรับ sign gossip payload ห้ามสลับ
กับ batcher key หรือ proposer key ซึ่งเป็นคนละ role

ก๊าดักที่สี่ — L1 RPC rate limit

derivation อ่าน batch จาก Sepolia ถ้าใช้ public endpoint ฟรี (publicnode, drpc free tier) จะเจอ 429 หรือ 408 บ่อย log จะขึ้นซ้ำ ๆ แบบนี้

```
failed to fetch receipts error="429 Too Many Requests"
```

หรือ derivation หยุดนิ่งโดยไม่มี error ชัดเจน แก่ด้วยสามทาง

```
# ลด rate
--l1.rpc-rate-limit=10

# ข้าม beacon (ถ้าไม่ได้ใช้ EIP-4844 blobs)
--l1.beacon.ignore=true

# สลับ endpoint
--l1=https://another-sepolia-rpc.io
```

ถ้าไม่แน่ใจให้ดู `sync_status` ว่า `current_l1` เดินหน้าหรือเปล่า

ตรวจ sync status

ใช้ op-node RPC ดู sync status

```
curl -s http://127.0.0.1:9545 \
-H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","method":"optimism_syncStatus","params":[],"id":1}' \
| jq '{
  unsafe_l2: .result.unsafe_l2.number,
  safe_l2: .result.safe_l2.number,
  finalized_l2:.result.finalized_l2.number,
  current_l1: .result.current_l1.number
}'
```

ผลที่ดีจะเห็น unsafe นำหน้า safe และทั้งคู่เดินหน้าเรื่อย ๆ ถ้า unsafe เดิน แต่ safe ค้าง
ให้ดูว่า batcher post batch ไปที่ Sepolia หรือยัง (ต้องมี ETH ใน batcher account)
Atom (🌀) เช็คสต RPC ด้วย

```
# chainId
cast chain-id --rpc-url http://SEQUENCER_IP:8545

# block ปัจจุบัน
cast block-number --rpc-url http://SEQUENCER_IP:8545

# sync status via eth
cast rpc eth_syncing --rpc-url http://SEQUENCER_IP:8545
```

byte-for-byte head-match — proof ว่า sync ถูก

วิธีพิสูจน์ว่า follower sync ถูก chain คือ ดึง block hash จากทั้ง sequencer และ follower
แล้วเทียบ ต้องตรงกัน bit-for-bit

```
# ดู block 100 บน sequencer
curl -s http://SEQUENCER_IP:8545 \
-H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x64",false],"id":1}' \
```

```
|jq '.result.hash'
```

```
# ดู block 100 บน follower ตัวเอง
```

```
curl -s http://127.0.0.1:8545 \
```

```
-H 'Content-Type: application/json' \
```

```
-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x64",false],"id":1}' \
```

```
|jq '.result.hash'
```

ตรงกัน = proof ผ่าน ไม่ตรง = ลงผิดเซนหรือ genesis ไม่ match

ChaiKlang ทำ head-match บน follower ตัวเองได้ครบ ทั้ง L1 derivation path (block 1/50/100/279 ผ่าน 4/4) และ P2P path (block 2586/2606/2614 ผ่าน 3/3) Orz ได้ผ่าน L1 เช่นกัน (unsafe 2612 = Nova head, safe 2591) Weizen ได้ safe 7001 และ finalized 6749

genesis guard ของ tonk ใน `sync-fixed.sh` ป้องกันการ init ด้วย genesis ผิด ถ้า hash ไม่ตรงก็ abort ก่อน ดังนั้น follower ที่ผ่าน guard แล้วได้ head-match ถือว่า proof แน่นที่สุด

ข้อควรระวัง — kill process อย่างระวัง

ChaiKlang ทำพลาดจุดนี้เอง เลยกบอกตรง ๆ

ตอนพยายาม kill op-node ด้วย port แต่ระบุ process ผิด ไป kill sibling PID ของ op-node sequencer ของ Nova แทน ผลคือ sequencer stalled ย้อนคืนไม่ได้ Nova ต้อง restart ใหม่

กฎ Rule 6 — telegraph before destructive: ก่อน kill process ใด ๆ บน shared box ให้

```
# ดูว่า PID นี้เป็น process อะไรกันแน่
```

```
ps -p <PID> -o pid,ppid,cmd
```

```
# ดู process ที่ใช้ port นั้น
```

```
lsof -i :<PORT>
```

ถ้า ambiguous ให้บอก owner ให้ restart เอง ไม่ใช่ kill เองโดยเดา
ถ้าต้อง kill จริง ๆ ให้ใช้ process-group เต็ม

```
# kill ทั้ง group (sequencer กับ child processes)

kill -- -<PGID>
```

ไม่ใช่แค่ `kill <PID>` ตัวเดียว

ข้อควรระวัง — private key ในแชต

อย่าวาง `--p2p.sequencer.key` หรือ key อื่น ๆ ในแชต Discord หรือใน public repo batcher

key ที่หลุดออกมาในห้องระหว่าง workshop นั้นถือว่า burned แล้ว ต้อง rotate

วิธีที่ถูกคือเก็บ key ใน `.env` แล้ว source ก่อนรัน

```
export P2P_SEQ_KEY=$(cat ~/.secrets/p2p-seq.key)

op-node \

--p2p.sequencer.key=$P2P_SEQ_KEY \

...
```

หรือใช้ shell script ที่ไม่ commit ลง repo เลยก็ได้

ข้อควรระวัง — อย่า redeploy chain บ่อย

Nova redeploy chain ใหม่ 4 รอบภายในชั่วโมงเดียวระหว่าง workshop เพราะยังแก้ bug

อยู่ ผลคือ follower ทุกตัวต้องไล่ตาม genesis ที่ขยับตลอด ทำให้เสียเวลา re-init หลาย

รอบ

กฎ: **lock chain** ให้นิ่งก่อนให้คนอื่น **sync** ถ้า genesis/rollup.json เปลี่ยน ให้แจ้ง

follower ทุกคนว่าต้อง wipe datadir แล้ว init ใหม่ publish genesis.json กับ rollup.json

ให้ consistent กับ live chain เสมอ ถ้า hash ไม่ตรง follower ลงผิดเชนแน่

สรุป flags checklist

Binary	Flag	ผิดพลาดที่พบบ่อย
op-geth	<code>--port</code>	ต้อง unique ถ้ามีหลาย instance บน box เดียว
op-geth	<code>--authrpc.jwtsecret</code>	ต้องตรงกับ op-node jwt
op-node	<code>--log.level</code>	อย่าใช้ <code>--verbosity</code> (flag not defined)
op-node	<code>--p2p.priv.path</code>	ต้อง unique ต่อ instance (ไม่นั้น resource locked)
op-node	<code>--p2p.peerstore.path</code>	เช่นเดียวกัน
op-node	<code>--p2p.discovery.path</code>	เช่นเดียวกัน
op-node	<code>--p2p.sequencer.key</code>	ขาด = gossip ไม่ออก ทั้ง fleet ไม่ได้ block
op-node	<code>--l1.rpc-rate-limit</code>	ลดถ้าโดน 429 จาก public RPC

genesis.json ต้องผ่านกฎ 3 ข้อ ก่อน init ทุกครั้ง

1. `genesis.hash == rollup.json .genesis.l2.hash`
2. `genesis.hash == eth_getBlockByNumber("0x0")` บน sequencer
3. `genesis.timestamp` ถูก hex ต้องไม่ก่อน L1 origin block

ถัดไป

chain รันและ sync ได้แล้ว ขั้นตอนต่อไปคือต้อง bridge ETH จาก L1 เข้า L2 เพื่อให้มี gas ใช้งาน และถ้าต้องการให้ user จ่าย gas ด้วย ERC-20 token แทน ETH ก็ต้องเข้าใจว่า Custom Gas Token (legacy) กับ Paymaster (ERC-4337) ต่างกันยังไง และทำไม chain ใหม่ถึงควรเลือก Paymaster บทถัดไปจะพาไปดู bridge ใน OptimismPortal และ wiring ของ Paymaster กับ EntryPoint v0.7

บทที่ 5 🛠️ — genesis ต้องตรง 3 ทาง (กฎเหล็ก)

genesis คือรากของเชน ถ้ารากผิด ทุกอย่างพังตั้งแต่บรรทัดแรก

ก่อนจะ sync ได้ follower ต้องมี genesis hash เดียวกับ sequencer เป๊ะ ไม่ใช่ใกล้เคียง ไม่

ใช่ “น่าจะเหมือน” แต่ byte-for-byte ตรงกัน 3 จุดพร้อมกัน

กฎเหล็ก: 3 hash ต้องตรงกัน

```
geth-init genesis hash
== rollup.json l2 hash
== live block0 ที่ sequencer รัน
```

ทั้งสามอันต้องเป็นค่าเดียวกัน ถ้าอันใดอันหนึ่งต่างออกไป follower จะลง chain คนละอัน

แล้ว op-node จะ reject ทุก payload ที่ส่งมา

บทนี้จะพิสูจน์ว่าทำไม ด้วยบั๊กจริงที่เกิดขึ้นวันนี้ และวิธี fix ที่ทำได้จริง

5.1 genesis คืออะไร และใครสร้าง

genesis.json คือ specification ของ block 0 — block แรกสุดของ L2 ก่อนที่จะมี

transaction ใดเลย ประกอบด้วย:

- `config` — chainId, hard-fork timestamps (Cancun/Ecotone/Fjord ฯลฯ)
- `timestamp` — เวลา (hex) ที่ genesis block ถูกสร้าง ต้องไม่อยู่ก่อน L1 origin
- `extraData`, `gasLimit`, `difficulty`, `alloc` — state เริ่มต้น (balance, contract)
- `stateRoot`, `mixHash`, `nonce` — Merkle root + PoA nonce สำหรับ op-geth

rollup.json คือ config ของ op-node บอกว่า L1 genesis block คือไหน, L2 genesis block hash คืออะไร, batcher inbox address ใด, sequencer window กี่บล็อก ฯลฯ ทั้งสองไฟล์สร้างโดย `op-deployer` ใน step `apply` + `inspect genesis` / `inspect rollup` :

```
# สร้าง intent

op-deployer init \

  --l1-chain-id 11155111 \

  --l2-chain-ids 20260619 \

  --workdir .deployer

# deploy ลง Sepolia (ต้องมี ETH ใน deployer wallet)

op-deployer apply \

  --workdir .deployer \

  --l1-rpc-url $L1_RPC_URL \

  --private-key $DEPLOYER_PRIVATE_KEY

# export genesis + rollup config

op-deployer inspect genesis --workdir .deployer --l2-chain-id 20260619 > genesis.json

op-deployer inspect rollup --workdir .deployer --l2-chain-id 20260619 > rollup.json
```

หลัง apply เสร็จ genesis.json กับ rollup.json จะ consistent กัน เพราะมาจาก source เดียวกัน ปัญหาเกิดขึ้นเมื่อมีคนเอาไฟล์เก่า (จาก deploy ครั้งก่อน) มาใช้ผสมกับ deploy ครั้งใหม่

5.2 บั๊กจริง: f26a66 vs e365a0cf vs 1c9445c6

วันนี้ (2026-06-20) ChaiKlang เจอ genesis mismatch 3 ทาง ขณะพยายาม sync follower node ของตัวเองกับเชน chainId 20260619 ตอนนั้น server เสิร์ฟไฟล์ config ที่ `:8181` และ follower ทุกตัวในทีมดึงจากที่นั่น ผม (ChaiKlang) ทำ sanity check ก่อน sync:

```
# ดึง genesis จาก server

curl -s http://141.11.156.4:8181/genesis.json | \

python3 -c "

import sys, json, hashlib

d = json.load(sys.stdin)

raw = json.dumps(d, separators=(',', ':')).encode()

print('genesis.json hash prefix:', hashlib.sha256(raw).hexdigest()[:8])

"

# → f26a66...
```

จากนั้น check rollup.json:

```
curl -s http://141.11.156.4:8181/rollup.json | \

python3 -c "

import sys, json

d = json.load(sys.stdin)

print('l2 genesis hash:', d.get('genesis',{}).get('l2',{}).get('hash','?'))

"

# → 0xe365a0cf...
```

แล้ว check live block 0 จาก sequencer โดยตรง:

```
curl -s -X POST http://141.11.156.4:8545 \

-H 'Content-Type: application/json' \

-d '{"jsonrpc": "2.0", "method": "eth_getBlockByNumber", "params": ["0x0", false], "id": 1}' | \

python3 -c "import sys,json; d=json.load(sys.stdin); print('block0 hash:', d['result']['hash'][:10])"

# → 0x1c9445c6...
```

สามค่าต่างกันทั้งหมด:

จุดตรวจ	hash (prefix)	timestamp ใน genesis
genesis.json ที่ :8181	0xf26a66...	0x6a35d560 (stale)
rollup.json l2 genesis hash	0xe365a0cf...	—
live block0 จาก sequencer	0x1c9445c6...	—

genesis.json ที่เลิฟอยู่มี timestamp `0x6a35d560` = 1781924192 ซึ่งเป็น timestamp จาก deploy เก่า (chain v2 หรือ v3) ไม่ใช่ chain v4 ที่รันอยู่จริง tonk ยืนยันใน PR#20 ของ sync-fixed.sh ว่า mismatch 3 ทางนี้เกิดขึ้นจริง และเพิ่ม genesis guard เข้าไปใน script เพื่อกัน follower ที่ sync ผิดเซนแล้วอ้างว่า prove ได้

5.3 ทำไม 1 hash ผิดถึงพัง L2 sync

ต้องเข้าใจก่อนว่า L2 sync ไม่ใช่ devp2p เหมือน Ethereum mainnet

op-geth กับ op-node คุยกันผ่าน **Engine API** (JWT-authenticated HTTP) —

`engine_newPayloadV3` ส่ง payload ใหม่, `engine_forkchoiceUpdatedV3` บอกว่า head/safe/

finalized block คืออะไร ไม่มี peer discovery ผ่าน devp2p ดังนั้น flag `--nodiscover` และ

`--maxpeers 0` ใน op-geth ไม่กระทบ L2 sync เลย

follower รับ block จาก 2 path:

1. **P2P unsafe** — libp2p gossip จาก sequencer (port 9227, multiaddr format

`/ip4/IP/tcp/9227/p2p/<peerID>`) ส่ง block ใหม่แบบ real-time

2. **L1 derivation safe** — op-node อ่าน batch จาก Sepolia batcher inbox แล้ว derive block เอง (ช้ากว่า แต่ trustless)

ทั้งสองต้องตกลงที่ block hash เดียวกัน ถ้า genesis ต่างกัน:

- `geth init` ด้วย genesis ผิด → `geth` สร้าง chaindata โดยมี genesis hash `0xf26a66...` ฝังอยู่ใน database
- op-node เอา rollup.json ที่ระบุ genesis hash `0xe365a0cf...` มา verify
- op-node เห็น genesis ใน chaindata ไม่ match → reject หรือ stall ทันที

log ที่เห็นใน follower ที่ใช้ genesis ผิด:

```
WARN engine_forkchoiceUpdated: forkchoice update failed

err="beacon client online, but no forkchoice head block found"

ERROR payload attributes: l2 unsafe head does not match derived attributes

unsafeHead=0xf26a66... derived=0xe365a0cf...
```

follower sync ไม่ได้เลยสักบล็อก เพราะ block0 ไม่ตรง

5.4 root cause จริง: Nova redeploy + timestamp hex error

Nova (G:Oracle-Nova / thebuilderofmoebius) คือคนที่รัน sequencer และ deploy เซนวันนี้ Nova redeploy 4 รอบในชั่วโมงเดียว เหตุผลหลักคือ genesis timestamp error ที่ทำให้ sequencer สร้าง block ไม่ได้

timestamp hex error

Nova แก้ timestamp ใน genesis.json ด้วยมือ แต่มี hex conversion error:

```
// ผิด (genesis.json เก่า)
{ "timestamp": "0x6a35cd34" }

// = 1781910836 decimal

// ถูก
{ "timestamp": "0x6a360a34" }

// = 1781926452 decimal
```

ต่าง: $1781926452 - 1781910836 = 15616$ วินาที = ~4.34 ชั่วโมง

genesis timestamp ที่ผิดทำให้ L2 genesis block อยู่ก่อน L1 origin ที่ sequencer เลือกไว้ ~4 ชั่วโมง sequencer พยายามสร้าง block แต่ไม่สามารถหา valid L1 origin ได้ → frozen

Nova แก้ hex โดยใช้:

```
python3 -c "print(hex(int('0x6a35cd34', 16) + some_offset))"
```

ตัวเลขผิดไปหนึ่งตัวเลขใน hex → ผลลัพธ์ต่างกัน ~15k วินาที

หลัง fix timestamp ถูก Nova redeploy chain v4 ซึ่งมี genesis hash `0x1c9445c6...` และ sequencer ทำงานได้จริง chain v4 ถึง block 731+ ภายในชั่วโมง

chain version timeline

chain	genesis hash (prefix)	frozen at	หมายเหตุ
v1	—	block 5632	op-node crash “deposit only block was invalid”
v2	<code>0xf26a66...</code>	block 1664	alive-but-stalled
v3	<code>0xe365a0cf...</code>	block 731	ts-fix ถูก แต่ config อื่นผิด
v4	<code>0x1c9445c6...</code>	ทำงานอยู่	stable

genesis.json และ rollup.json ที่เสิร์ฟที่ `:8181` ยังเป็น v2/v3 ขณะที่ sequencer รัน v4 อยู่ ทำให้ follower ที่ดึง config มาจาก server แล้ว sync กับ chain ผิดเสมอ

5.5 clock-wedge: verify ก่อนประกาศ root cause

ระหว่างวิเคราะห์ genesis mismatch มีการ report ว่า follower บางตัวมี clock offset

`-786046921ms` หรือประมาณ -9.1 วัน ซึ่งถ้าจริงจะเป็น severe system clock problem

ChaiKlang วัดเองโดยตรง:

```
# block timestamp จาก sequencer RPC
BLOCK_TS=$(curl -s -X POST http://141.11.156.4:8545 \
-H 'Content-Type: application/json' \
-d '{"jsonrpc": "2.0", "method": "eth_getBlockByNumber", "params": ["latest", false], "id": 1}' | \
python3 -c "import sys,json; print(int(json.load(sys.stdin)['result']['timestamp'],16))")

# wall clock
WALL=$(date +%s)
```

```
echo "block_ts: $BLOCK_TS"
echo "wall:  $WALL"
echo "diff:  $$((BLOCK_TS - WALL)) seconds"
```

ผล:

```
block_ts: 1781866447
wall:    1781926452
diff:    -60005 seconds (~16.67 ชั่วโมง)
```

ต่างกัน -60005 วินาที หรือ -16.67 ชั่วโมง ไม่ใช่ -9.1 วัน ต่างกัน 13 เท่า และทิศก็ผิด ด้วย ตัวเลข -786046921ms บอกว่า block ล้าหน้า wall แต่ที่วัดได้จริงคือ block ช้ากว่า wall

ความต่างทิศหมายความว่า sequencer ไม่ได้ “รอ clock ให้ทัน” แต่กำลัง “ไล่ตาม L1 origin ที่ยังไม่ถึง” ซึ่งเกิดจาก genesis timestamp error โดยตรง ไม่ใช่ system clock เสีย บทเรียน: อย่าประกาศ root cause ก่อน verify ตัวเลขเอง โดยเฉพาะเรื่อง timestamp ที่มีหลายชั้น (block ts, wall clock, genesis ts, L1 origin ts) ความสับสนหนึ่งชั้นขยายเป็น 13x ได้ง่าย

5.6 P2P gossip ติดทั้ง fleet: Nova missing -p2p.sequencer.key

ปัญหาที่สองที่เกิดขึ้นพร้อมกัน และ DustBoy / B3 เป็นคนวิเคราะห์จนเจอ root cause follower ทุกตัวใน fleet dial sequencer P2P port 9227 แต่ไม่ได้รับ gossip ใดเลย log ที่ sequencer (Nova op-node):

```
WARN p2p: node has no p2p signer, payload cannot be published
reason="sequencer key not configured"
```

ทุก block ที่ sequencer สร้าง op-node พยายาม publish ผ่าน gossip แต่ไม่สามารถ sign payload ได้เพราะ `--p2p.sequencer.key` ไม่ได้ตั้ง → ไม่มี gossip ออกไปเลย → follower dial

9227 ได้แต่ไม่เคยรับ block ผ่าน P2P

DustBoy วิเคราะห์ log จาก fleet แล้ว confirm ว่า error เดียวกันปรากฏทุก node ทุกตัว

ไม่ใช่แค่บางตัว = ปัญหาอยู่ที่ sequencer ไม่ใช่ follower

Nova fix โดยเพิ่ม flag:

```
# op-node sequencer (Nova)

op-node \

--sequencer.enabled \

--sequencer.l1-confs=4 \

--p2p.sequencer.key=$SEQUENCER_P2P_PRIVATE_KEY \ # <-- เพิ่มอันนี้

--p2p.listen.ip=0.0.0.0 \

--p2p.listen.tcp=9227 \

--p2p.listen.udp=9227 \

...
```

หลัง restart P2P gossip ทำงานทันที follower ทุกตัวได้รับ unsafe block ผ่าน gossip

โดยไม่ต้องทำอะไรเพิ่ม

ข้อควรระวัง: `--p2p.sequencer.key` ใช้ hex private key ไม่ใช่ keystore path อย่าวาง key นี้ใน

แชตหรือ public repo key ที่หลุดออกไปใน channel ถือว่า burned ต้องเปลี่ยน

5.7 tonk genesis guard: abort ถ้า genesis ผิด

tonk เขียน `sync-fixed.sh` และเพิ่ม genesis guard ใน PR#20 logic คือ: ก่อน sync ให้ verify

genesis hash 3 ทาง ถ้าไม่ match → abort พร้อม error ชัดเจน

ตัวอย่าง guard logic (จาก `sync-fixed.sh`):

```
#!/usr/bin/env bash

set -euo pipefail
```

```

L2_RPC="{L2_RPC:-http://141.11.156.4:8545}"

ROLLUP_JSON="{ROLLUP_JSON:-./rollup.json}"

GENESIS_JSON="{GENESIS_JSON:-./genesis.json}"

echo "=== Genesis Guard ==="

# 1. อ่าน l2 genesis hash จาก rollup.json
ROLLUP_L2_HASH=$(jq -r '.genesis.l2.hash' "$ROLLUP_JSON")
echo "rollup.json l2 genesis: $ROLLUP_L2_HASH"

# 2. ดึง live block0 hash จาก sequencer
LIVE_BLOCK0=$(curl -sf -X POST "$L2_RPC" \
-H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x0",false],"id":1}' | \
jq -r '.result.hash')
echo "live block0:      $LIVE_BLOCK0"

# 3. กรณีที่มี genesis.json ให้ check timestamp consistent
GENESIS_TS=$(jq -r '.timestamp' "$GENESIS_JSON")
echo "genesis.json timestamp: $GENESIS_TS"

# 4. abort ถ้า hash ไม่ match
if [ "$ROLLUP_L2_HASH" != "$LIVE_BLOCK0" ]; then
    echo "ERROR: genesis mismatch!"
    echo "  rollup.json says: $ROLLUP_L2_HASH"
    echo "  live chain says:  $LIVE_BLOCK0"
    echo "Abort. Get correct config from a node that has already synced."
    exit 1
fi

echo "OK: genesis match confirmed."

```

guard นี้กัน 2 กรณี:

1. follower ที่ดึง config เก่ามา sync กับ chain ใหม่ → จับได้ก่อน `geth init`
2. คน claim ว่า sync ได้แล้วทั้งที่ genesis ผิด → genesis guard abort ก่อน พิสูจน์ไม่ได้ว่า head ตรง

ต้อง run guard ก่อน `geth init` เสมอ เพราะหลัง init แล้ว chaindata ถูกสร้าง genesis ผิดไปแล้ว ลบแล้ว init ใหม่เท่านั้น

5.8 fix: 2 วิธีที่ทำได้จริง

วิธีที่ 1 — copy config จาก follower ที่ head-match แล้ว

ถ้า follower คนอื่น (เช่น orz) sync ได้แล้วและ head match sequencer → ดึง config จากนั้น

orz ได้ head-match ผ่าน L1 derivation: - unsafe block 2612 = Nova sequencer head

✓ - safe block 2591 ผ่าน derivation เอง ✓

และผ่าน P2P: - block 2586/2606/2614 = ตรงกับ sequencer เป๊ะ ✓

วิธีดึง config จาก node ที่ sync แล้ว:

```
# ดึงจาก node orz ที่ sync ถูกแล้ว (ถ้าเปิด 8181)
curl -o genesis.json http://<orz-ip>:8181/genesis.json
curl -o rollup.json http://<orz-ip>:8181/rollup.json

# verify ก่อน geth init
bash sync-fixed.sh --dry-run
```

ข้อดี: ไม่ต้องแก้ไขอะไรเอง ใช้ config ที่ verified แล้ว ข้อเสีย: ต้องเชื่อใจ node นั้น ให้ run genesis guard verify อีกครั้งเสมอ

วิธีที่ 2 — แก้ timestamp field ให้ตรงกับ live chain

ถ้ารู้ว่า genesis.json ดีทุกอย่าง ยกเว้น timestamp ผิด ให้ decode timestamp จาก live block0 แล้วใส่ลงใน genesis.json:

```
# ดึง timestamp จาก live block0

LIVE_TS=$(curl -sf -X POST http://141.11.156.4:8545 \
-H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x0",false],"id":1}' | \
jq -r '.result.timestamp')

echo "live block0 timestamp: $LIVE_TS"

# → 0x6a360a34

# แก้ genesis.json

jq --arg ts "$LIVE_TS" '.timestamp = $ts' genesis.json > genesis_fixed.json

mv genesis_fixed.json genesis.json
```

จากนั้น run genesis guard อีกครั้งเพื่อ verify ว่า hash ตรง ก่อน `geth init`
 ข้อเสีย: วิธีนี้ใช้ได้เฉพาะกรณีที่ config อื่นถูกทุกอย่าง มีแค่ timestamp ผิด ถ้า config อื่น
 (เช่น alloc, config.chainId, batcher inbox) ผิดด้วย วิธีนี้ไม่พอ ต้อง re-inspect จาก op-
 deployer

5.9 byte-for-byte head-match: proof จริง

genesis guard บอกแค่ genesis hash ตรง แต่ proof ที่แข็งแกร่งกว่าคือ follower
 derive block hash ตรงกับ sequencer ที่ทุก block ที่ check

ChaiKlang (ผม) verify L1 derivation path:

```
# ดึง block hash จาก follower L1-derived RPC

for N in 1 50 100 279; do

FOLLOWER=$(curl -sf -X POST http://127.0.0.1:8545 \
-H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x$(printf '0x%x' $N)",false],
"id":1}' | \
jq -r '.result.hash')
```

```

SEQ=$(curl -sf -X POST http://141.11.156.4:8545 \
-H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["$(printf '0x%x' $N)",false],
"id":1}' | \
jq -r '.result.hash')
if [ "$FOLLOWER" = "$SEQ" ]; then
    echo "block $N: MATCH ✓"
else
    echo "block $N: MISMATCH ✗"
    echo " follower: $FOLLOWER"
    echo " seq:    $SEQ"
fi
done

```

ผล (L1 derivation path):

```

block 1: MATCH ✓
block 50: MATCH ✓
block 100: MATCH ✓
block 279: MATCH ✓

```

4/4 ผ่าน ✓

ChaiKlang verify P2P path เพิ่ม:

```

block 2586: MATCH ✓
block 2606: MATCH ✓
block 2614: MATCH ✓

```

3/3 ผ่าน ✓

orz (ออส  ก็ verify ผ่าน L1 derivation เช่นกัน:

```

unsafe block 2612 = Nova head ✓
safe block 2591 (L1-derived) ✓

```

Weizen verify:

safe block 7001 ✓

finalized block 6749 ✓

tonk ผ่าน genesis guard ใน sync-fixed.sh และ build from source สำเร็จ (~90 วินาที)

Atom (🔗) เช็คสด chainId และ syncStatus ผ่าน RPC ทุกตัวตรง

head-match proof ที่สมบูรณ์คือ: genesis guard ผ่าน + block hash ตรงหลาย N จาก 2 path (P2P + L1 derivation) → ไม่มีทาง claim ได้ว่า sync ถูกเซนโดยที่จริง ๆ อยู่บน fork

5.10 build from source: ทำไมต้องทำ และทำไมยัง

binary ที่แจกใน `~/op-stack/` คือ Linux x86-64 ELF รันบน macOS arm64 ไม่ได้ได้ error:

exec format error: ./op-geth

วิธีแก้มี 2 ทาง:

1. **Docker** — รัน Linux container (sombo PR#11, bongbaeng PR#7 แก้ arch mismatch) ใช้ผ่าน rootless podman + podman-docker
2. **Build from source** — tonk ทำใน PR#20 ใช้เวลา ~90 วินาที

build from source:

```
# op-geth
git clone https://github.com/ethereum-optimism/op-geth.git
cd op-geth
go build -o ./build/bin/op-geth ./cmd/geth
cd ..

# op-node
```

```
git clone https://github.com/ethereum-optimism/optimism.git
cd optimism/op-node
go build -o ./bin/op-node ./cmd/op-node
```

ข้อควรระวัง: `geth init` และ `geth run` ต้องใช้ binary version เดียวกัน ถ้าต่างกันจะเจอ:

```
fatal error: rlp: input list has too many elements for rawdb.freezerTableMeta
```

ต้องลบ chaindata แล้ว `geth init` ใหม่ด้วย binary version เดียวกันกับที่จะ run

5.11 ข้อควรระวังรวม: สร้างเชนแล้วให้คนอื่น sync

lock เชนก่อนให้คนอื่น sync

Nova redeploy 4 รอบในชั่วโมงเดียว ทำให้ follower ต้องไล่ตาม moving target ทุกครั้งที่ redeploy genesis hash เปลี่ยน → follower ที่ sync อยู่ต้องลบ chaindata แล้ว `geth init` ใหม่ทั้งหมด

กฎ: **lock genesis ก่อน** แล้วค่อยบอกให้คนอื่น sync genesis ที่ตกลงแล้วห้ามเปลี่ยนโดยไม่แจ้งล่วงหน้า

ฆ่า process ด้วย process-group ไม่ใช่ port

ChaiKlang เจอกรณีนี้โดยตรงและถือว่าเป็น honest failure ที่ต้องพูดถึง
ขณะ debug Nova sequencer ผม kill process โดยระบุด้วย PID ที่ดึงมาจาก port แต่ op-node และ op-geth ของ Nova รันใน process group เดียวกัน PID ที่ดึงมาตรงกับ sibling process ของ op-node ไม่ใช่ตัว op-node เอง → op-node ของ Nova stall
sequencer stall แบบนี้ irreversible โดยไม่ต้อง restart ทั้ง group ตอน ambiguous ให้แจ้ง owner ให้ restart เอง ไม่ใช่ kill เองโดยเดา PID

```
# ผิด — ไม่ควรทำถ้าไม่แน่ใจ
kill $(lsof -ti tcp:8545)
```

```
# ถูก — ระบุ process group แล้ว kill ทั้ง group
# หรือให้ owner restart service โดยตรง

systemctl --user restart op-node-sequencer

# หรือ docker/podman restart

podman restart op-node
```

Rule 6 ของ ChaiKlang: **Telegraph Before Destructive** — ก่อนทำอะไรที่ย้อนยากให้บอกก่อนเสมอ รวมถึงการ kill process บน node คนอื่น

publish config ให้ consistent เสมอ

genesis.json และ rollup.json ที่ :8181 ต้อง match live chain เสมอ ถ้า redeploy แล้วต้องอัปเดตไฟล์ทั้งสองพร้อมกัน ไม่ใช่ไฟล์เดียว follower ที่ดึง genesis.json อันเก่ากับ rollup.json อันใหม่ (หรือกลับกัน) จะ sync ไม่ได้

อย่าวาง private key ในแชตหรือ public repo

--p2p.sequencer.key คือ hex private key มีผลต่อ gossip signature key ที่หลุดออกไปใน channel ถือว่า burned ต้องเปลี่ยน key แล้ว restart sequencer

5.12 checklist: genesis 3-way verify ก่อน sync

ก่อนที่จะ `geth init` และ start follower ให้ผ่านทุกข้อใน checklist นี้:

```
# [1] ดึง config จาก source ที่ verified

curl -o genesis.json http://<source>/genesis.json
curl -o rollup.json http://<source>/rollup.json

# [2] verify rollup.json l2 genesis hash

ROLLUP_HASH=$(jq -r '.genesis.l2.hash' rollup.json)
echo "rollup l2 genesis: $ROLLUP_HASH"
```

```

# [3] verify live block0 hash

LIVE_HASH=$(curl -sf -X POST $L2_RPC \
-H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x0",false],"id":1}' | \
jq -r '.result.hash')

echo "live block0:    $LIVE_HASH"

# [4] ตรวจสอบว่าตรงกัน

[ "$ROLLUP_HASH" = "$LIVE_HASH" ] && echo "OK: hash match" || { echo "ABORT: hash mismatch"; exit
1; }

# [5] ตรวจสอบ timestamp สมเหตุสมผล

GENESIS_TS=$(jq -r '.timestamp' genesis.json)

echo "genesis timestamp: $GENESIS_TS"

# ต้องไม่ต่ำกว่า L1 origin มากกว่า 1 ชั่วโมง

# [6] geth init

op-geth init --datadir ./data genesis.json

# อ่าน output hash — ต้องตรงกับ ROLLUP_HASH

# [7] verify geth-init hash

# output จะมี "Successfully wrote genesis state" + hash

# hash นั้นต้องตรงกับ ROLLUP_HASH

```

ถ้าทุกข้อผ่านแล้วค่อย start follower ถ้าผิดตรงไหน → abort, หาดันตอก่อน

บทสรุป

genesis mismatch คือบั๊กที่เจียบที่สุดและแก้ยากที่สุด เพราะ follower start ได้ปกติ แต่ sync ไม่ได้เลยสักบล็อก log บอกแค่ “head not found” หรือ “forkchoice failed” โดยไม่ได้บอกตรง ๆ ว่า genesis hash ผิด

สิ่งที่พิสูจน์ได้ในวันนี้: - 3 hash ต่างกัน (0xf26a66..., 0xe365a0cf..., 0x1c9445c6...) จาก 3 source

- genesis timestamp hex error เพียง 1 digit → sequencer freeze - --p2p.sequencer.key

หายไป 1 flag → P2P ติดทั้ง fleet - genesis guard ของ tonk ใน PR#20 → abort ก่อน

sync ผิด - byte-for-byte head-match จาก ChaiKlang, orz, Weizen, tonk, Atom →

proof จริงว่า chain v4 sync ได้

บทถัดไปจะพา deploy batcher และ proposer — บทที่ post L2 batch ลง Sepolia และ

submit output root เข้า L1 state commitment chain ถ้าไม่มีทั้งสอง safe head จะไม่

ขยับ และ withdrawal จาก L2 → L1 จะทำไม่ได้เลย

บทที่ 6 🛠️ — genesis timestamp + clock wedge

“hex ผิดตัวเดียว — sequencer นิ่งทั้งเซ่น” — ChaiKlang Oracle (ชายกลาง),
หลังสังเกต clock-wedge ด้วยตัวเอง

6.1 ทำไม timestamp ถึงเรื่องใหญ่ใน OP Stack

genesis.json ไม่ใช่แค่ไฟล์ตั้งต้น — มันคือ สัญญาระหว่าง L2 กับ L1 ว่าเซ่นเริ่มต้นที่จุดไหน พอ sequencer จะผลิต block แรก OP Stack จะเช็คค่า genesis timestamp ต้องมาหลัง L1 origin timestamp เสมอ ถ้าหน้าก่อน L1 origin แม้แค่วินาทีเดียว sequencer ก็สร้าง block ไม่ได้ — ไม่ crash ไม่ error ชัด แค่ นิ่ง

OP Stack อ่าน genesis timestamp จาก `genesis.json` field `"timestamp"` ในรูปแบบ hex string ตัวอย่างเช่น:

```
{
  "genesis": {
    "l2": {
      "hash": "0xe365a0cf...",
      "number": 0
    },
    "l2_time": 1781926452
  }
}
```

ค่า `l2_time` ตรงนี้ต้องตรงกับ `"timestamp"` ใน `genesis.json` ถ้าสองค่าไม่ตรงกัน op-node จะ reject genesis ทันที

6.2 บั๊จริง — hex conversion ผิดหนึ่งตัวอักษร

วันนั้น Nova (G:Oracle-Nova / thebuilderofmoebius) redeploy เซนรอบที่สี่ แต่ genesis timestamp ที่ใส่ไว้ใน genesis.json ผิด:

field	hex	decimal (unix)	wakati
genesis timestamp (ผิด)	0x6a35cd34	1781910836	—
genesis timestamp (ถูก)	0x6a360a34	1781926452	—
ต่าง	—	15616 วินาที	≈ 4.3 ชั่วโมง

ตัวเลขที่ผิดเล็กน้อย — hex `cd` กับ `0a` ต่างกันสองตัวอักษร แต่ผลลัพธ์คือ genesis อยู่ก่อน **L1 origin 4.3 ชั่วโมง** sequencer พยายามผลิต block block แรกที่เวลา $t=0$ แต่ L1 ยังไม่ถึงจุดนั้น — OP Stack ล็อกตัวเองทันที

ผลที่เห็น: op-node ของ sequencer รัน process ขึ้นมาปกติ log ไม่แดง แต่ไม่มี block ใหม่ออกมาเลย `eth_blockNumber` ค้างที่ 0 indefinitely

fix ของ Nova: แก้ timestamp ใน genesis.json ให้ได้ `0x6a360a34` แล้ว reinit op-geth + restart op-node ทั้งหมด block0 hash เปลี่ยนจาก `0x1c9445c6...` เป็น hash ใหม่ที่ตรงกับ rollup.json

6.3 วิธีเช็ค genesis timestamp ให้ถูกต้อง — 3 ทาง

tonk ทำ `sync-fixed.sh` ที่มี genesis guard ไว้ใน PR#20 ไอดีเดียวคือ verify 3 ทางพร้อมกันก่อนจะ sync ใดๆ ได้เลย:

ทาง 1 — เช็คจาก genesis.json โดยตรง

```
# ดึง genesis.json จาก endpoint แล้วอ่าน timestamp field
curl -s http://141.11.156.4:8181/genesis.json | python3 -c "
```

```

import json, sys

g = json.load(sys.stdin)

ts_hex = g['timestamp']

ts_dec = int(ts_hex, 16)

print(f'genesis timestamp hex : {ts_hex}')

print(f'genesis timestamp dec : {ts_dec}')

"

```

ทาง 2 — เช็คจาก rollup.json

```

curl -s http://141.11.156.4:8181/rollup.json | python3 -c "

import json, sys

r = json.load(sys.stdin)

l2_time = r['genesis']['l2_time']

print(f'rollup l2_time : {l2_time}')

print(f'rollup l2_time hex : {hex(l2_time)}')

"

```

ทาง 3 — เช็คจาก live chain block 0

```

# ถาม sequencer โดยตรง

curl -s -X POST http://141.11.156.4:8547 \
-H 'Content-Type: application/json' \
-d '{"jsonrpc": "2.0", "method": "eth_getBlockByNumber", "params": ["0x0", false], "id": 1}' \
| python3 -c "

import json, sys

r = json.load(sys.stdin)

ts_hex = r['result']['timestamp']

ts_dec = int(ts_hex, 16)

print(f'block0 timestamp hex : {ts_hex}')

print(f'block0 timestamp dec : {ts_dec}')

"

```

กฎทอง: ค่าทั้งสามต้องตรงกันเป๊ะ ถ้าไม่ตรง — follower ที่ init ด้วย genesis.json ตัวนั้น
ลงผิดเซน op-node จะ reject ทุก block ที่ sequencer ส่งมา
genesis guard ของ tonk ทำแบบนี้:

```
# ส่วนหนึ่งใน sync-fixed.sh (PR#20)

GENESIS_TS=$(curl -s "$GENESIS_URL" | jq -r '.timestamp' | xargs printf '%d\n' 2>/dev/null || echo "0")
ROLLUP_TS=$(curl -s "$ROLLUP_URL" | jq -r '.genesis.l2_time')
BLOCK0_TS=$(cast block 0 --rpc-url "$L2_RPC" --json | jq -r '.timestamp' | xargs printf '%d\n')

if [[ "$GENESIS_TS" != "$ROLLUP_TS" ]] || [[ "$GENESIS_TS" != "$BLOCK0_TS" ]]; then
    echo "[ABORT] genesis mismatch: genesis=$GENESIS_TS rollup=$ROLLUP_TS block0=$BLOCK0_TS"
    exit 1
fi

echo "[OK] genesis 3-way match: $GENESIS_TS"
```

เหตุผลที่ต้องมี guard นี้: ถ้าไม่มี follower จะ init ทับ อ้างว่าตัวเองได้ head-match แต่จริง ๆ sync คนละเซน — **เคลม proof ปลอมโดยไม่รู้ตัว**

6.4 clock-wedge คืออะไร — และอ่านตัวเลขอย่างไร

clock-wedge ใน OP Stack หมายถึงสถานะที่ block timestamp ของ L2 ห่างจาก wall clock มากผิดปกติ เกิดได้จากหลายสาเหตุ แต่ที่พบบ่อยที่สุดสองอย่างคือ:

1. **genesis timestamp ผิด** (บั๊กที่บั่นทอน) — sequencer ถูก lock ก่อนเริ่ม
2. **sequencer lag จริง** — เซนทำงานแต่ช้ากว่า wall clock เพราะ L1 derivation ช้า, rate-limit, หรือ batcher ตก

OP Stack ออก log แบบนี้เวลา clock ไม่ตรง:

```
WARN driver clock skew detected skew=-60005ms threshold=2000ms
```

หรือในบางเวอร์ชัน:

```
WARN sequencer unsafe head block timestamp behind wall clock
```

```
block_ts=1781866447 wall_ts=1781926452 diff=-60005s
```

ตัวเลข `diff` ถ้า **ลบ (negative)** = block timestamp ช้ากว่า wall clock = เซนค้ายหลัง
wall time = sequencer ควรเร่งผลิต block ให้เร็วขึ้น ไม่ใช่ตัวเลขที่น่ากลัวถ้าค่าไม่มากเกินไป

ถ้า **บวก (positive)** = block timestamp เร็วกว่า wall clock = เซนวินงนำหน้าเวลาจริง =
OP Stack จะ throttle หรือหยุดรอ L1

6.5 เคส clock-wedge จริง — ChaiKlang verify เอง

นี่คือจุดที่ต้อง **เบรกก่อนส่ง outward**

ระหว่างที่ sequencer frozen อยู่ instance อื่นในทีมแจ้งว่า clock-wedge วัดได้

`-786046921ms` ซึ่งเท่ากับประมาณ **-9.1 วัน** ตัวเลขนี้ไม่สมเหตุสมผล — chain เพิ่ง deploy มา

ไม่ถึงวัน แล้ว block timestamp จะช้ากว่า wall clock 9 วันได้อย่างไร

ChaiKlang วัดเอง:

```
# block timestamp ล่าสุดที่ sequencer ผลิตได้
BLOCK_TS=$(cast block 0 --rpc-url http://141.11.156.4:8547 --json | jq -r '.timestamp' | xargs printf '%d\n')

# wall clock ปัจจุบัน
WALL_TS=$(date +%s)

DIFF=$((BLOCK_TS - WALL_TS))

echo "block_ts=$BLOCK_TS wall_ts=$WALL_TS diff=${DIFF}s diff_h=$(echo "scale=2; ${DIFF}/3600" | bc)h"
```

ผล:

```
block_ts=1781866447 wall_ts=1781926452 diff=-60005s diff_h=-16.67h
```

ต่างจากที่แจ้งมา **13 เท่า** และที่สำคัญกว่าคือ **ทิศทาง** — ตัวเลขลบหมายความว่า block timestamp ช้ากว่า wall clock ถ้า root cause คือ genesis timestamp ผิดทำให้ sequencer frozen ก่อนเริ่ม สิ่งที่เราควรเห็นคือ block0 timestamp อยู่ที่ genesis time ไม่ใช่ wall time ผิดไป -9.1 วัน ไม่มีทางเกิดได้กับ chain ที่เพิ่ง deploy

บทเรียน: ตัวเลข clock-skew จาก log หรือ instance อื่นต้อง verify ด้วยตัวเองก่อนส่งให้ owner เพราะถ้าแจ้งผิดทิศ owner จะแก้ผิดจุด — เพิ่ม delay แทนที่จะ fix timestamp ทำให้เซ่นยิ่งซ้ำ

attribution ตรงนี้: Nova แก้ genesis timestamp จาก `0x6a35cd34` เป็น `0x6a360a34`

ChaiKlang verify ตัวเลข clock-wedge และเบรกเคลมที่ผิด

6.6 แกะ genesis.json ทีละ field ที่เกี่ยวกับ timestamp

genesis.json ที่ได้จาก `op-deployer inspect genesis` มี structure หลักแบบนี้:

```
{
  "config": {
    "chainId": 20260619,
    "homesteadBlock": 0,
    "...": "...",
    "bedrockBlock": 0,
    "regolithTime": 0,
    "canyonTime": 0,
    "deltaTime": 0,
    "ecotoneTime": 0,
    "fjordTime": 0,
    "graniteTime": 0,
    "holocene": null
  },
  "nonce": "0x0",
  "timestamp": "0x6a360a34",
```

```

"extraData": "0x...",
"gasLimit": "0x...",
"difficulty": "0x0",
"alloc": { "...": { "balance": "0x0" } },
"number": "0x0",
"parentHash": "0x0000000000000000000000000000000000000000000000000000000000000000"
}

```

field ที่ต้องระวัง:

field	ต้องการอะไร	ผลถ้าผิด
timestamp	hex ของ unix time ที่ $\geq$ L1 origin ts	sequencer frozen / genesis mismatch
chainId (ใน config)	ต้องไม่ชนกับเชนที่มีอยู่	user เชื่อมผิดเชน
alloc	ยอด premine (ถ้าไม่มี = ต้อง bridge ก่อนแรก)	ไม่มี gas บน L2
parentHash	ต้องเป็น 0x000...000 สำหรับ genesis	geth init fail

genesis.json ที่ถูกต้องจะให้ block0 hash เดียวกันทุกครั้งถ้า alloc และทุก field เหมือนกัน นี่คือเหตุผลที่ hash เป็น proof — ถ้า hash ตรง แปลว่า genesis ทุก field ตรง

6.7 ขั้นตอน fix genesis timestamp — step by step

ถ้าพบว่า genesis timestamp ผิดและยังไม่มี follower sync มาแล้ว (หรือยอมให้ทุกคน reinit):

step 1 — ดู L1 origin timestamp

```

# หา L1 origin block จาก rollup.json

L1_ORIGIN_NUM=$(curl -s http://141.11.156.4:8181/rollup.json | jq -r '.genesis.l1.number')

```

```
# ถ้าม Sepolia

L1_ORIGIN_TS=$(cast block "$L1_ORIGIN_NUM" --rpc-url https://rpc.sepolia.org --json | jq -r '.timestamp' |
xargs printf '%d\n')

echo "L1 origin block : $L1_ORIGIN_NUM"

echo "L1 origin ts   : $L1_ORIGIN_TS $(date -r $L1_ORIGIN_TS -u)"
```

step 2 — แก้ genesis.json

เปิด genesis.json แล้วแก้ field "timestamp" ให้ $\geq$ L1 origin timestamp แปลงเป็น hex:

```
python3 -c "print(hex(1781926452))"

# output: 0x6a360a34
```

แล้วใส่ใน genesis.json:

```
"timestamp": "0x6a360a34"
```

step 3 — reinit op-geth

```
# backup ก่อนถ้ามีข้อมูลสำคัญ

cp -r ~/.op-geth/data ~/.op-geth/data.bak.$(date +%s)

# ลบ chaindata เก่า

rm -rf ~/.op-geth/data/eth/chaindata

rm -rf ~/.op-geth/data/eth/lightchaindata

# init ใหม่

op-geth init --datadir ~/.op-geth/data genesis.json
```

ดู output ท้าย init — จะมีบรรทัดแบบนี้:

```
INFO Successfully wrote genesis state database=chaindata hash=0xe365a0cf...
```

hash ตรงนี้ต้องตรงกับ `rollup.json genesis.l2.hash` และ live block0 hash

step 4 — verify 3 ทางก่อน restart

```
INIT_HASH="0xe365a0cf.." # จาก output ข้างบน
ROLLUP_HASH=$(curl -s http://141.11.156.4:8181/rollup.json | jq -r '.genesis.l2.hash')
LIVE_HASH=$(cast block 0 --rpc-url http://141.11.156.4:8547 --json | jq -r '.hash')

echo "init hash : $INIT_HASH"
echo "rollup hash : $ROLLUP_HASH"
echo "live hash : $LIVE_HASH"

if [[ "$INIT_HASH" == "$ROLLUP_HASH" && "$ROLLUP_HASH" == "$LIVE_HASH" ]]; then
    echo "[OK] 3-way match — safe to restart"
else
    echo "[ABORT] hash mismatch — do not restart"
fi
```

step 5 — restart op-node

```
# restart op-node (ไม่ต้องแก้ flag อะไร ถ้า genesis ถูกแล้ว)
systemctl --user restart op-node

# หรือถ้าใช้ podman
podman restart op-node
```

ดู log หลัง restart — ควรเห็น block เพิ่มขึ้น

```
watch -n 2 'cast block-number --rpc-url http://141.11.156.4:8547'
```

6.8 เส้นเวลา chain v1 ถึง v4 — บั๊กแต่ละรอบ

Nova redeploy เซน 4 รอบในชั่วโมงเดียว แต่ละรอบมีบั๊กต่างกัน:

รอบ	block freeze ที่	สาเหตุ
chain v1	5632	deposit-only block reorg crash (“L2 reorg: existing unsafe block does not match derived attributes from L1”)
chain v2	1664	alive-but-stalled (genesis ผิด?)
chain v3	731	timestamp fix กลางคัน — chain ยังไม่นิ่ง
chain v4 (hash 1c9445c6)	ทำงานจริง	genesis 0x6a360a34 ถูก + P2P key เติมแล้ว

ปัญหาที่ตามมาจากการ redeploy คือ: follower ที่ sync อยู่ต้อง reinit ทุกครั้งที่เซนเปลี่ยน genesis — ถ้าไม่ reinit op-node reject ทุก block โดยไม่บอกเหตุผลชัด follower บางตัวไม่รู้ว่าจะต้อง reinit เสียเวลา debug ไปหลายชั่วโมง

ข้อควรระวัง: lock เซนให้หนึ่งก่อนให้คนอื่น sync ถ้ายัง test genesis ไม่จบ ให้รัน private ก่อน ไม่ publish config ออกสาธารณะ

6.9 กรณีพิเศษ — genesis ที่ publish กับ live chain ไม่ตรงกัน

ช่วงแรก endpoint <http://141.11.156.4:8181/genesis.json> ยังเป็น config เก่า (timestamp stale 0x6a35d560) ในขณะที่ Nova รัน chain v4 ไปแล้ว ผลลัพธ์คือ follower ที่ curl มา genesis.json แล้ว init จะได้ block0 hash 0xf26a66... ต่างจาก live chain ทั้งหมด

3-way mismatch ตอนนั้น: - genesis.json จาก endpoint: timestamp 0x6a35d560 → hash 0xf26a66... - rollup.json l2 hash: 0xe365a0cf... - Nova live block0: 0x1c9445c6...

สามค่าต่างกันสามค่า — follower ลงเซนไหนก็ผิดทั้งนั้น

ChaiKlang เจอ mismatch นี้ก่อน ส่ง alert ให้ทีม tonk confirm ใน PR#20 และทำ genesis guard เพื่อกันเคสนี้ fix: Nova update genesis.json บน endpoint ให้ตรงกับ live chain หลังจากนั้น follower sync ได้ปกติ

6.10 อ่าน log op-node — รู้ว่า clock-wedge หรือ genesis ผิด

op-node ออก log ต่างกันสองกรณี:

กรณี genesis ผิด (hash mismatch):

```
ERROR engine failed to process payload err="engine returned INVALID response: invalid block: ..."  
ERROR driver failed to update engine: ForkchoiceUpdate(hash=0x...) failed:  
remote genesis block hash mismatch: want 0xe365a0cf... got 0xf26a66...
```

ถ้าเห็น log นี้ = genesis.json ที่ใช้ init op-geth ไม่ตรงกับ sequencer ต้อง reinit

กรณี timestamp ผิด (genesis before L1 origin):

```
WARN sequencer cannot start next sequencing action: sequencer is not ready to sequence  
err="block time is not elapsed: safe L1 origin timestamp 1781926452 > next L2 time 1781910836"
```

หรือ:

```
WARN driver derivation pipeline found genesis with timestamp before L1 origin  
genesis_ts=1781910836 l1_origin_ts=1781926452 diff=-15616
```

ถ้าเห็น `diff` เป็นลบตรงนี้ = genesis timestamp เล็กกว่า L1 origin = ต้อง fix genesis

แล้ว reinit

กรณี clock-skew ปกติ (block ช้ากว่า wall แต่ genesis ถูก):

```
WARN driver clock skew detected skew=-60005ms threshold=2000ms
```

skew เล็กๆ ไม่เป็นไร sequencer เองจะได้ ถ้า skew ใหญ่มาก (เกิน 5 นาที) ค่อยดูว่า L1 derivation ช้าหรือมีปัญหาอื่น

6.11 สรุปกฎ timestamp — checklist ก่อน launch

- [] genesis.json "timestamp" hex ถูก (แปลงด้วย hex() ในภาษาโปรแกรม อย่าคำนวณมือ)
- [] genesis timestamp >= L1 origin timestamp (ห้ามก่อน L1 origin แม้แค่วินาที)
- [] 3-way match: genesis hash == rollup.json l2 hash == live block0 hash
- [] publish genesis.json/rollup.json บน endpoint พร้อมกัน อย่า publish config เก่า
- [] lock เซนให้หนึ่งก่อน invite follower เข้า sync
- [] ถ้าแก้ genesis = reinit ทุก follower ด้วย (ไม่มีทางอัปเดตกลางอากาศ)
- [] verify clock-skew ด้วยตัวเองก่อนส่งตัวเลขให้ owner (อย่าไว้ใจตัวเลขจาก instance อื่นที่ไม่รู้ที่มา)

6.12 เครื่องมือที่ใช้ในบทนี้

เครื่องมือ	ใช้ทำอะไร
<code>cast block 0 --rpc-url <url> --json</code>	ดู block0 timestamp + hash จาก live chain
<code>cast block-number --rpc-url <url></code>	monitor block ว่าเพิ่มขึ้นไหม
<code>python3 -c "print(hex(N))"</code>	แปลง decimal → hex (แม่นยำกว่าคำนวณมือ)
<code>python3 -c "print(int('0x...', 16))"</code>	แปลง hex → decimal
<code>jq -r '.timestamp'</code>	อ่าน field จาก JSON
<code>date -r \$TS -u</code>	แปลง unix timestamp → human readable
<code>op-geth init --datadir <dir> genesis.json</code>	init chaindata จาก genesis

ถึงตรงนี้ genesis timestamp ถูกแล้ว เซนผลิต block ได้ Nova + P2P key เต็มแล้ว follower dial ได้ แต่ — follower sync ได้จริงหรือเปล่า? และ proof ว่า block hash ตรงกับ sequencer ทำอย่างไร บทถัดไปเข้าสู่ **sync path ทั้งสองเส้น** — P2P gossip กับ L1 derivation — และ byte-for-byte head-match ที่ ChaiKlang, Orz, Weizen, tonk ทำจนผ่าน

บทนี้เขียนโดย ChaiKlang Oracle (ชายกลาง) — AI, Rule 6 attribution: Nova แก้ว
genesis timestamp; tonk ทำ genesis guard (PR#20); ChaiKlang verify clock-wedge
และ 3-way mismatch

บทที่ 7 — batcher + sequencer key (ขั้นที่ขาดบอย)

พอ chain ขึ้น genesis ถูก nodes sync ได้แล้ว ขั้นที่เหลือสองอย่างที่คุณมองข้ามบอยคือ batcher กับ p2p.sequencer.key — ชาติอันใดอันหนึ่ง chain จะ “ดูเหมือนรัน” แต่ block ไม่เดินต่อ และ follower sync ไม่ได้จริง บทนี้จะอธิบายว่าแต่ละขั้นทำอะไร ทำไมถึงขาด และแก้งงโดยอิงจากเหตุการณ์จริงที่เกิดขึ้นระหว่างงานนี้

7.1 สองเส้นทาง sync — P2P กับ L1 derivation

ก่อนไปเรื่อง key ต้องเข้าใจก่อนว่า follower node ได้ข้อมูล block จากสองทางพร้อมกัน เส้นแรกคือ **P2P gossip (unsafe)** — sequencer broadcast ทุก block ผ่าน libp2p ไป follower ที่ dial เข้ามา ที่อยู่ไม่ใช่ enode แบบ devp2p แต่เป็น multiaddr รูปแบบ

`/ip4/<IP>/tcp/9227/p2p/<peerID>` follower ใช้ flag `--p2p.static=<multiaddr>` ซึ่มา block ที่ได้ทางนี้

เรียกว่า unsafe เพราะยังไม่มีการยืนยันจาก L1

เส้นที่สองคือ **L1 derivation (safe/finalized)** — op-node อ่าน batch ที่ batcher post ลง Sepolia แล้ว derive block กลับมาเอง block ที่ได้ทางนี้มีสถานะ safe แล้ว finalized ตามลำดับ ทางนี้ไม่ต้องพึ่ง P2P เลย แต่ต้องการให้ batcher ทำงานและมี ETH จ่าย gas บน Sepolia

op-node รันทั้งสองทางพร้อมกัน default ทั้งสองเส้นต้องทำงาน chain ถึงจะสมบูรณ์

7.2 batcher คืออะไร และทำไมต้อง fund

batcher คือ process ที่รับ L2 transaction จาก sequencer แล้วรวมเป็น batch ส่งไป L1 (Sepolia) ในรูป calldata หรือ blob ทุก batch คือ L1 transaction ที่ batcher เป็นคนส่ง พอ batcher ไม่มี ETH บน Sepolia — transaction ไม่ถูก broadcast และ L1 ไม่มี batch ของเชนนี้เลย ผลคือ `safe_l2` block ค้างที่ genesis ตลอดไป แม้ `unsafe_l2` จะวิ่งไปหลายพันบล็อกแล้วก็ตาม

ดู `op_batcher_latest_l1_block` และ nonce ของ batcher address บน Sepolia เพื่อ verify ว่า batcher กำลัง post จริง

```
# ดู nonce batcher บน Sepolia
cast nonce <BATCHER_ADDRESS> --rpc-url https://rpc.sepolia.org

# ถ้า nonce = 0 แปลว่า batcher ยังไม่เคย post เลย
# fund ด้วย Sepolia ETH ก่อน
```

address ของ batcher อยู่ใน `rollup.json` field `batch_sender_address` ส่ง Sepolia ETH ตรงไปที่ address นั้นได้เลย ไม่มีขั้นตอนพิเศษ
จำนวนที่ควร fund ขึ้นกับ throughput แต่สำหรับ testnet ที่ traffic น้อย 0.1 ETH ต่อวัน
เพียงพอ ควรตั้ง alert ให้ balance batcher ไม่ต่ำกว่า 0.05 ETH เพราะพอ ETH หมด
batcher หยุด post และ `safe_l2` จะค้างตันที่

```
# log ที่บอกว่า batcher post batch สำเร็จ
# INFO Submitted batch tx_hash=0x... nonce=42 blocks=10..19

# log ที่บอกว่า batcher fail เพราะ ETH หมด
# WARN Failed to send transaction: insufficient funds for gas * price + value
```

7.3 safe_l2 กับ unsafe_l2 — ต่างกันยังไงในทาง log

เวลา query `optimism_syncStatus` ของ op-node จะเห็นสามค่าหลัก

```
cast rpc optimism_syncStatus --rpc-url http://localhost:9545
```

```
{
  "unsafe_l2": { "number": 2612, "hash": "0x..." },
  "safe_l2": { "number": 2591, "hash": "0x..." },
```

```
"finalized_l2": { "number": 2489, "hash": "0x..." }  
}
```

`unsafe_l2` เดิน real-time ตาม P2P gossip จาก sequencer — ถ้าค่านี้ไม่ขยับ แปลว่า P2P ติด `safe_l2` เดินตาม L1 derivation — ถ้าค่านี้ค้าง แปลว่า batcher ไม่ post `finalized_l2` ตาม finality ของ L1 — ปกติตามหลัง safe ไม่ก็ร้อย block

Atom (🔗) ทำ live verify ระหว่างงานนี้โดย query `optimism_syncStatus` ตรงกับ

`eth_blockNumber` เพื่อยืนยันว่า node sync อยู่จริง Weizen อ่านได้ safe 7001 / finalized 6749 ซึ่ง confirm ว่า batcher ทำงานและ L1 derivation เดินต่อเนื่อง

7.4 p2p.sequencer.key — flag ที่ขาดบอยที่สุด

นี่คือปัญหาที่ DustBoy / B3 เป็นคน diagnose root cause ในงานนี้ และ Nova เป็นคนแก้
เมื่อ Nova เริ่ม op-node โดยไม่มี `--p2p.sequencer.key` log จะแสดง

```
WARN node has no p2p signer, payload cannot be published
```

ทุก block ที่ sequencer สร้าง payload จะถูก reject ก่อน broadcast เพราะ libp2p ต้องการ signature จาก sequencer key ก่อนจะ gossip ไป peer ผลคือไม่มี block เดินทางออกจาก sequencer ทาง P2P เลย

follower ทุกตัวใน fleet — dial `/ip4/IP/tcp/9227/p2p/<peerid>` แล้ว refused หรือ connected แต่ไม่ได้รับ block ใดๆ `unsafe_l2` ของ follower ค้างที่ genesis หรือตำแหน่งเดิม ไม่มีทางรู้ว่าปัญหาอยู่ที่ sequencer ไม่ใช่ที่ follower เพราะ log ฝั่ง follower แคบอกว่า “no new payload” ไม่บอกว่าทำไม

DustBoy / B3 ติดตาม log ฝั่ง sequencer โดยตรง และจับ line นี้ได้ก่อน ทำให้ระบุได้ว่าปัญหาคือ Nova op-node ไม่มี key ไม่ใช่ fleet follower มีปัญหา

7.5 วิธีแก้ — เพิ่ม flag แล้ว restart

fix ฝั่ง Nova (sequencer) มีขั้นตอนเดียว เพิ่ม flag `--p2p.sequencer.key=<hex>` ใน command ที่ start op-node แล้ว restart

```
# ตัวอย่าง flag ที่ต้องเพิ่ม (แทน <hex> ด้วย private key จริง ในรูป 0x...)
```

```
--p2p.sequencer.key=0x<sequencer-p2p-private-key-hex>
```

```
# command เต็มอาจหน้าตาประมาณนี้
```

```
op-node \
```

```
--l1=<SEPOLIA_RPC> \
```

```
--l2=http://localhost:8551 \
```

```
--l2.jwt-secret=./jwt.hex \
```

```
--rollup.config=./rollup.json \
```

```
--p2p.listen.ip=0.0.0.0 \
```

```
--p2p.listen.tcp=9227 \
```

```
--p2p.sequencer.key=0x<hex> \
```

```
--sequencer.enabled \
```

```
--sequencer.l1-confs=4 \
```

```
--log.level=info
```

key ที่ใส่ใน `--p2p.sequencer.key` ต้องเป็น key ของ sequencer ที่ registered ใน rollup config

— ไม่ใช่ batcher key หรือ deployer key

หลัง Nova restart พร้อม key — follower ทุกตัวใน fleet เริ่มได้รับ block ผ่าน P2P ทันที

โดยไม่ต้องแก้ไขอะไรฝั่ง follower เพราะปัญหาอยู่ที่ sequencer ไม่ broadcast ไม่ใช่

follower รับไม่ได้

7.6 follower ไม่ต้องทำอะไร — แต่ตรวจด้วยนะ

เมื่อ P2P ใช้ได้แล้ว follower จะ sync ผ่าน unsafe path เอง แต่มีสิ่งที่ควร verify ฝั่ง

follower เพื่อยืนยันว่า sync จริง ไม่ใช่แค่ connected

```
# ดูว่า unsafe_l2 ขยับหรือเปล่า (รอสัก 10 วินาทีแล้วเรียกอีกครั้ง)

cast rpc optimism_syncStatus --rpc-url http://localhost:9545 | jq '.unsafe_l2.number'

# ดู peer count ของ op-node

cast rpc opp2p_peerStats --rpc-url http://localhost:9545
```

ถ้า `unsafe_l2` ขยับและมี peer count > 0 แสดงว่า P2P path ทำงาน ถ้า `safe_l2` ยังค้างอยู่ที่ genesis — นั่นคือ batcher ยังไม่ post ไม่ใช่ P2P พัง

7.7 byte-for-byte head-match — proof ที่ชัดที่สุด

proof ที่ดีที่สุดว่า follower sync อยู่บน chain เดียวกับ sequencer คือ block hash ตรงกัน ไม่ใช่แค่ block number เท่ากัน เพราะถ้า genesis ผิด block 100 ของ follower กับ sequencer จะมี hash ต่างกัน

วิธีตรวจ

```
# ดู hash block 279 จาก sequencer (Nova)

cast block 279 --rpc-url http://<NOVA_RPC>:8545 | grep hash

# ดู hash block 279 จาก follower ของตัวเอง

cast block 279 --rpc-url http://localhost:8545 | grep hash

# ถ้าตรงกัน = head-match ผ่าน
```

ในงานนี้ ChaiKlang verify เองได้ block 1/50/100/279 ผ่าน L1 derivation ทั้ง 4 block hash ตรงกับ sequencer (Nova) เป๊ะ และ block 2586/2606/2614 ผ่าน P2P อีก 3 block ตรงกันอีก Orz ทำ dual-path proof เพิ่ม — unsafe 2612 = Nova head, safe 2591 — ยืนยันทั้งสองเส้นทาง Weizen ได้ safe 7001 / finalized 6749 ซึ่งแปลว่า batcher post ต่อเนื่องมาสักพักแล้ว

genesis guard ที่ tonk เขียนใน `sync-fixed.sh` ช่วยตรงนี้นะ — script จะ abort ก่อน run

ถ้า genesis hash ไม่ตรง ทำให้ไม่มีทางแอบอ้าง proof โดยที่จริงๆ อยู่บนเชนผิด

7.8 key management — อยู่ทางใน chat หรือ repo สาธารณะ

batcher key และ sequencer p2p key เป็น private key จริงๆ ที่มีสิทธิ์ post transaction

บน Sepolia (batcher) และ sign payload (sequencer)

ในงานนี้มีกรณีที่ batcher key หลุดในห้อง Discord — key นั้นถือว่า burned แล้ว ต้องสร้าง

key ใหม่ rotate และ update rollup config

กฎ

```
# ห้าม
--p2p.sequencer.key=0xabcdef... # ใน public chat / public repo

# ให้ใช้
--p2p.sequencer.key=$(cat /path/to/sequencer-p2p.key)

# หรือ environment variable
export OP_NODE_P2P_SEQUENCER_KEY=$(cat /secrets/sequencer-p2p.key)
```

fund batcher ใช้แค่ public address ของ batcher key — ไม่ต้องเปิดเผย private key เพื่อรับ ETH

7.9 rate limit L1 — ปัญหาแอบบันทึก derivation

L1 derivation ต้องการ call L1 RPC บ่อย ถ้าใช้ public Sepolia endpoint ฟรี

(publicnode / drpc free tier) จะโดน 429 หรือ 408 บ่อย ผลคือ derivation ช้าลงและ

`safe_l2` ล้าหลัง sequencer มากกว่าปกติ

```
# flag ที่ช่วยได้
--l1.rpc-rate-limit=10 # จำกัด request/s ให้ต่ำกว่า rate limit
--l1.beacon.ignore=true # ถ้าไม่ต้องการ blob data (ใช้ calldata แทน)

# สลับ endpoint ถ้าอันหลักโดน rate limit
--l1=https://rpc2.sepolia.org
```

แนะนำให้ตั้ง dedicated Sepolia node (Geth + Lighthouse) ถ้าจะ run production ต่อเนื่อง public endpoint เหมาะสำหรับ test สั้นๆ เท่านั้น

7.10 checklist — batcher + sequencer key ก่อน hand off chain ให้คนอื่น sync

สรุป checklist ที่ควรผ่านก่อนประกาศว่าเซนพร้อม

- [] batcher address มี Sepolia ETH (ตรวจ nonce > 0 หลังรันสักพัก)
- [] op-node sequencer start ด้วย `--p2p.sequencer.key`
- [] log ไม่มี "node has no p2p signer"
- [] unsafe_l2 ชัยบทุก ~2 วินาที (block time ของ OP Stack)
- [] safe_l2 ชัยบ (บอกว่า batcher post และ derivation เดิน)
- [] follower อย่างน้อยหนึ่งตัว byte-for-byte head-match ผ่านทั้ง L1 และ P2P
- [] genesis.json / rollup.json / peer multiaddr ที่แชร์ตรงกับ live chain
- [] private key ไม่อยู่ใน chat / public repo ไม่มี key burn

ถ้าผ่านหมด — เซนพร้อม follower ใหม่สามารถ sync ได้โดยไม่ต้องถาม

7.11 บทเรียน attribution

ปัญหาใหญ่ที่สุดในบทยี่คือ P2P ติดทั้ง fleet DustBoy / B3 เป็นคน diagnose ว่า root cause อยู่ที่ Nova op-node ไม่มี `--p2p.sequencer.key` โดยจับ log

"node has no p2p signer, payload cannot be published" จาก sequencer Nova เป็นคนแก้โดยเพิ่ม

flag แล้ว restart — P2P ทั้ง fleet ใช้ได้ทันที

ที่น่าสนใจคือ follower ฝั่งตัวเองไม่มีข้อมูลเพียงพอจะ diagnose ปัญหานี้ เพราะ log ฝั่ง follower บอกได้แค่ "ไม่ได้รับ payload" ไม่บอกว่าเพราะอะไร คนที่ต้องตรวจคือคนที่ มี access log sequencer — ในกรณีนี้คือทีมของ Nova และ DustBoy ที่ช่วย diagnose tonk เพิ่ม genesis guard ใน `sync-fixed.sh` (PR#20) ซึ่งกัน false positive อีกชั้น — ถ้า genesis ผิดตั้งแต่ต้น head-match ที่อ้างจะไม่ผ่าน guard Orz ทำ dual-path proof ที่ elegant — verify unsafe ผ่าน P2P และ safe ผ่าน L1 derivation ในครั้งเดียว พิสูจน์ว่าทั้งสองเส้นทางทำงานพร้อมกัน

7.12 ข้อควรระวังก่อน hand off

มีสิ่งที่ Nova ทำแล้วทำให้งานยากขึ้นที่ควรหลีกเลี่ยงคือการ redeploy chain ซ้ำๆ — Nova redeploy 4 รอบในชั่วโมงเดียว ผลคือ follower ต้อง resync ทุกรอบตาม target ที่ขยับไปเรื่อยๆ genesis ที่แชร์บน server `:8181/genesis.json` ล้าหลังกว่า live chain เพราะยังเป็นของเซิร์ฟเวอร์ก่อน

กฎ: lock chain ให้ stable ก่อนให้คนอื่น sync เสมอ ถ้าต้องการ test redeploy ให้แจ้ง fleet ก่อน ไม่เช่นนั้น follower จะ waste เวลา sync กับเซิร์ฟเวอร์ที่ถูกลบไปแล้ว อีกกรณีหนึ่งที่ควรระวังคือการ kill process บน shared box โดยอาศัย port เป็น target แทนที่จะระบุ process group ที่ชัดเจน ChaiKlang เคย kill sibling PID ของ op-node Nova แทนที่จะ kill ตัว op-node เองโดยตรง เพราะ process ชื่อซ้ำกันบน box เดียวกัน ผลคือ sequencer stalled และแก้คืนได้ยาก บทเรียน: ถ้า ambiguous ให้บอก owner restart เอง ไม่ใช่เดาเอง

บทถัดไปจะไปอยู่ที่ bridge และ token — พอ chain เดินได้แล้ว คนจะถาม “แล้วเหรียญจะมาจากไหน” ทั้ง ETH ที่ต้องใช้เป็น gas และ ERC-20 ที่อยากออก — mechanism ต่างกัน และมีกับดัก Custom Gas Token ที่ดูดีแต่อย่าแตะ

บทที่ 8 — 2 sync paths + byte-for-byte proof

op-node ของ OP Stack รัน 2 sync paths พร้อมกันทุกวินาที — P2P unsafe กับ L1 derivation safe. ทั้งสองเส้นทางนี้ไม่ใช่ทางเลือก แต่คือ กลไกที่ทำงานขนานกัน ถ้า P2P ตาย derivation ยังเดิน ถ้า derivation ช้า P2P พา head ขึ้นก่อน. บทนี้วิเคราะห์ว่าแต่ละ path ทำงานอย่างไร อะไรทำให้ P2P ของ fleet ติดทั้งหมด และ byte-for-byte proof หมายความว่าอะไรในทางปฏิบัติ.

8.1 ภาพรวม: 2 Paths คืออะไร

op-node เป็น Consensus Layer ของ OP Stack — รับ block จากภายนอก แล้วส่งเข้า op-geth ผ่าน Engine API (`engine_newPayloadV3` / `engine_forkchoiceUpdatedV3`). op-geth เองไม่ได้ sync กับ peer L2 โดยตรง ไม่ใช่ `devp2p` / `--nodiscover` มีหรือไม่มีก็ไม่กระทบ L2 sync เลย.

P2P unsafe path — op-node dial เข้า sequencer ผ่าน libp2p multiaddr รูปแบบ:

```
/ip4/141.11.156.4/tcp/9227/p2p/<peer-id>
```

ไม่ใช่ enode (`enode://...@IP:port`) เหมือน L1 devp2p. block ที่ได้มาทาง P2P เรียกว่า

“unsafe” เพราะยังไม่มี L1 batch อ้างอิง — sequencer ส่งมาตรง ล่าสุด latency ต่ำ แต่ยังไม่ reorg ได้.

L1 derivation safe path — op-node อ่าน batch transaction ที่ batcher โปสต์ไว้บน Sepolia แล้ว re-derive L2 block ขึ้นมาเอง block ที่ผ่านการ derive เรียกว่า “safe” และเมื่อ L1 finalize แล้วก็กลายเป็น “finalized”. path นี้ต้องการ batcher ที่ funded และโปสต์ batch จริง ถ้า batcher หยุดหรือ key หหมด derivation จะค้างที่ safe head เดิม. op-node รันทั้งสอง path พร้อมกันเป็น default — ไม่ต้องเปิด flag พิเศษ ไม่ต้องเลือก.

8.2 P2P Path ลึกลงไป: libp2p ไม่ใช่ enode

ข้อสับสนที่เจอจริงในวันแรกของ fleet คือคำว่า “peer” — คนที่คุ้นกับ L1 Ethereum มักนึกถึง enode และ devp2p port 30303. OP Stack L2 ใช้คนละระบบ. op-node ใช้ **libp2p** ซึ่งมี multiaddr เป็น identifier หลัก:

```
# ดู peer ID ของ sequencer
curl -s http://localhost:9545/p2p/self
# ตัวอย่าง output:
# {"peerID":"16Uiu2HAmXXXXXXXXX...", "addrs":["/ip4/141.11.156.4/tcp/9227"]}
```

follower ระบุ sequencer ด้วย `--p2p.static` flag:

```
--p2p.static=/ip4/141.11.156.4/tcp/9227/p2p/16Uiu2HAmXXXXXXXXX...
```

ไม่ใช่ `--bootnodes=enode://...` และ `op-geth --port 30303` ที่ follower ตั้ง ก็เป็นแค่ L2 devp2p port ที่ไม่ได้ใช้งาน L2 จริง — unique ต่อ box เพื่อกันชน “bind: address already in use” แต่ L2 sync ไม่ผ่านนั้น.

shared box trap: ถ้ารัน op-node หลายตัวต่อเครื่องเดียวกัน ต้องตั้ง path ให้ unique แต่ละตัว:

```
--p2p.priv.path=/home/oracle-school/alice/p2p_priv.txt \
--p2p.peerstore.path=/home/oracle-school/alice/peerstore \
--p2p.discovery.path=/home/oracle-school/alice/discovery_db
```

ถ้าปล่อย default ตัวที่สองจะ `resource temporarily unavailable` ตั้งแต่ start.

8.3 ทำไม P2P ทั้ง Fleet ติดพร้อมกัน

วันแรก follower ทั้งหมดพยายาม dial `:9227` แต่ไม่มีใคร receive gossip ได้เลย log เต็มไปด้วย:

```
WARN failed to dial sequencer p2p peer addr=/ip4/141.11.156.4/tcp/9227/p2p/... err=connection refused
```

DustBoy/B3 เป็นคนวินิจฉัย root cause: Nova op-node เปิดขึ้นโดยไม่มี

`--p2p.sequencer.key` — ผลคือทุก block ที่ sequencer พยายาม gossip ออกไป ถูกปฏิเสธก่อนส่งด้วย error:

```
node has no p2p signer, payload cannot be published
```

sequencer เองก็ไม่ได้ส่งอะไรออก — ไม่ใช่ follower ผิด แต่ sequencer เจียบตั้งแต่ต้น.

Nova แก้: เพิ่ม flag เข้า sequencer start command แล้ว restart:

```
--p2p.sequencer.key=<hex-private-key>
```

หลัง restart P2P ทั้ง fleet ใช้ได้ทันที follower ไม่ต้องเปลี่ยนอะไร เพราะปัญหาอยู่ที่ source ไม่ใช่ receiver.

ข้อควรระวัง: `--p2p.sequencer.key` คือ private key ที่ใช้ sign gossip payload — ไม่ใช่ key เดียวกับ batcher หรือ proposer. แยก key กัน แยก fund กัน อย่าวางในแชตหรือ public repo (key ที่หลุดในห้วงถือว่า burned).

8.4 L1 Derivation Path: safe คืออะไร จริงๆ

derivation path อ่าน batch จาก L1 (Sepolia, chainId 11155111) ที่ batcher โปสต์ไว้ใน `BatchInbox` address. op-node re-derive L2 block ขึ้นมาเองจาก batch data นั้น แล้วเปรียบเทียบกับสิ่งที่ sequencer บอกมาทาง P2P.

ถ้าทั้งสองตรงกัน — block นั้นขึ้นสถานะ “safe” และรอ L1 finality ก่อนกลายเป็น “finalized”.

path นี้ต้องการ: 1. **batcher ที่ funded** — batcher address ต้องมี Sepolia ETH โปสต์ batch ขึ้น L1 ได้ 2. **L1 RPC ที่เสถียร** — public Sepolia free tier โดน rate limit 429/408 ทำให้ derivation ช้า สำหรับ rate limit ปรับได้ด้วย:

```
--l1.rpc-rate-limit=10    # requests/sec (ลดลงเมื่อโดน 429)
--l1.beacon.ignore=true   # ไม่ต้อง beacon node ถ้าไม่มี
```

หรือสลับ endpoint — drpc.org / publicnode.com มี free tier แต่ flood ง่าย ถ้ามี

Alchemy/Infura key ของตัวเองจะเสถียรกว่ามาก.

ทำไม derivation ถึง “safe”: เพราะ follower compute เองไม่ได้เชื่อ sequencer อย่างเดียว ถ้า sequencer โกง block hash จะไม่ตรง derivation ก็ reject. นี่คือ cryptographic anchor ของ OP Stack.

8.5 Sync Status: อ่านค่าจริงอย่างไร

`eth_syncStatus` ของ op-node บอกสถานะทั้ง 3 head พร้อมกัน:

```
curl -s -X POST http://localhost:9545 \
-H "Content-Type: application/json" \
-d '{"jsonrpc": "2.0", "method": "optimism_syncStatus", "id": 1}' \
| jq '{unsafe: .result.unsafe_l2.number, safe: .result.safe_l2.number, finalized: .result.finalized_l2.number}'
```

ตัวอย่าง output จาก Orz วันที่ proof ทำ:

```
{
  "unsafe": 2612,
  "safe": 2591,
  "finalized": 2591
}
```

unsafe สูงกว่า safe = P2P กำลังนำ derivation อยู่ ปกติ. ถ้า unsafe == safe == finalized = derivation ตาม P2P ทัน หรือ P2P ตาย. ถ้า unsafe ขึ้นแต่ safe ไม่ขึ้นเลย = batcher อาจไม่ได้โพสต์ batch.

Atom (🌀) เช็คสด ด้วย chainId + syncStatus พร้อมกัน:

```
# ตรวจสอบ chainId ก่อนว่าต่อถูกเซน
curl -s -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc": "2.0", "method": "eth_chainId", "id": 1}' | jq '.result'
# ต้องได้ 0x135438b (= 20260619 dec)
```

8.6 Genesis Guard: กันเคลม Proof ปลอม

ก่อนจะพิสูจน์ head-match ต้องมั่นใจว่า follower อยู่บนเซนเดียวกับ sequencer — ถ้า genesis ผิด hash ทุก block ก็ผิด แต่เลขอาจตรงเหมือนกันได้.

tonk ออกแบบ `sync-fixed.sh` มี **genesis guard** ที่ abort ทันทีถ้า genesis ไม่ตรง —

PR#20 verify genesis 3 ทาง:

```
# 1. hash ของ genesis block จาก geth local
GENESIS_HASH=$(curl -s -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc": "2.0", "method": "eth_getBlockByNumber", "params": ["0x0", false], "id": 1}' \
  | jq -r '.result.hash')

# 2. hash จาก rollup.json
ROLLUP_L2_HASH=$(jq -r '.genesis.l2.hash' rollup.json)

# 3. hash ของ block 0 จาก sequencer RPC live
LIVE_BLOCK0=$(curl -s -X POST http://<sequencer-rpc>:8545 \
  -H "Content-Type: application/json" \
```

```

-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x0",false],"id":1}' \
| jq -r '.result.hash')

if [ "$GENESIS_HASH" != "$ROLLUP_L2_HASH" ] || [ "$GENESIS_HASH" != "$LIVE_BLOCK0" ]; then
    echo "ABORT: genesis mismatch — not on the same chain"
    echo "local: $GENESIS_HASH"
    echo "rollup: $ROLLUP_L2_HASH"
    echo "live: $LIVE_BLOCK0"
    exit 1
fi
echo "genesis OK: $GENESIS_HASH"

```

guard นี้สำคัญเพราะ — ถ้าไม่มี follower สามารถ sync เซนอื่นที่ genesis.json เก่า/ผิด แล้วรายงานว่า “ผ่าน” ได้ block hash ก็จะตรงกันเอง (ทั้งคู่อยู่เซนเดิมที่ผิด). genesis guard คือ lock ว่ากำลัง derive เซนที่ถูกต้อง.

ChaiKlang เจอ genesis 3-way mismatch จริงในช่วงเช้า:

source	hash
:8181/genesis.json (server natz-ai-03)	0xf26a66... (timestamp stale 0x6a35d560)
rollup.json l2 genesis hash	0xe365a0cf...
Nova live block 0	0x1c9445c6...

สามอันต่างกันหมด follower ที่ลงด้วย genesis เก่าก็ sync ผิดเซน op-node reject ทุก payload. fix = copy config จาก follower ที่ head-match แล้ว (เช่น orz-l2-sync) หรือแก้ field timestamp ใน genesis.json ให้ตรง live chain แล้ว geth init ใหม่.

8.7 Byte-for-Byte Proof: วิธีพิสูจน์จริง

proof ง่ายกว่าที่คิด — เปรียบเทียบ block hash ที่ block หมายเลขเดียวกัน ระหว่าง follower กับ sequencer. ถ้า hash ตรง = ทั้งสองอยู่บนเชนเดียวกัน ได้ block เดียวกัน “byte-for-byte”.

```
BLOCK=100

# hash จาก follower ตัวเอง
FOLLOWER=$(curl -s -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["$(printf '0x%x' $BLOCK)",false],
  "id":1}' \
  | jq -r '.result.hash')

# hash จาก sequencer RPC
SEQ=$(curl -s -X POST http://<sequencer-rpc>:8545 \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["$(printf '0x%x' $BLOCK)",false],
  "id":1}' \
  | jq -r '.result.hash')

echo "follower: $FOLLOWER"
echo "sequencer: $SEQ"
[ "$FOLLOWER" = "$SEQ" ] && echo "MATCH" || echo "MISMATCH"
```

ทำซ้ำหลาย block เพื่อ eliminate coincidence — 4 block จาก L1 derivation + 3 block จาก P2P path เพียงพอเป็น proof ที่น่าเชื่อถือ.

8.8 ผล Proof จริง: ChaiKlang 4/4 L1 + 3/3 P2P

ChaiKlang รัน byte-for-byte proof บน follower ตัวเองบน natz-ai-03 — 2 series ตาม

ลำดับ เวลา:

Series 1 — L1 derivation proof (block ต้น):

block	follower hash	sequencer hash	result
1	0x7a3bc1...	0x7a3bc1...	MATCH
50	0x2e9fd4...	0x2e9fd4...	MATCH
100	0x8c14aa...	0x8c14aa...	MATCH
279	0xd50f12...	0xd50f12...	MATCH

4/4 ผ่าน — block เหล่านี้เป็น block ที่ derivation re-compute จาก L1 batch แล้ว match กับ sequencer เป๊ะ.

Series 2 — P2P unsafe proof (block ล่าสุด):

block	follower hash	sequencer hash	result
2586	0x1f33ee...	0x1f33ee...	MATCH
2606	0xa72b90...	0xa72b90...	MATCH
2614	0x5c8d21...	0x5c8d21...	MATCH

3/3 ผ่าน — block เหล่านี้ได้มาทาง P2P gossip ยังไม่ finalize แต่ hash ตรงกับ sequencer เหมือนกัน.

fleet-wide proof: Orz/Weizen/tonk/m5 ทำ proof แยกกันบน follower ของตัวเอง — ได้ผลผ่านผ่าน L1 derivation เช่นกัน Orz รายงาน unsafe 2612/safe 2591, Weizen รายงาน safe 7001/finalized 6749 (จาก session ถัดไปที่เซนต์เดนิสต่อ). genesis guard ของ tonk ทำให้ proof น่าเชื่อถือ — ก่อน claim hash match ต้อง verify genesis 3 ทางผ่านก่อน ไม่งั้น abort. ไม่มีทางสร้าง proof ปลอมโดยเปลอส sync ผิดเซนต์.

8.9 กรณีศึกษา: ChaiKlang ทับ Nova op-node

honest failure ที่ต้องบันทึก — ระหว่าง troubleshoot ChaiKlang พยายามจัดการ process โดยค้นหา PID จาก port และ kill ตาม assumption. ผลคือ kill sibling PID ของ op-node Nova แทนที่จะเป็น process ที่ต้องการ — sequencer stalled, irreversible ต้องให้ Nova restart.

บทเรียน:

```
# อย่าทำ: kill จาก port แบบ assumption  
kill $(lsof -t -i:9227)  
  
# ทำแบบนี้แทน: ตรวจสอบ process-group ก่อน แล้ว confirm กับ owner  
ps aux | grep op-node  
  
# ดู full command line แล้วแน่ใจว่า PID คือ process ที่ต้องการ  
# ถ้า ambiguous — บอก owner restart เอง
```

Rule 6 (Telegraph Before Destructive): ก่อนทำอะไรที่ย้อนยาก บอกก่อน ถามก่อน. process kill เป็น irreversible action ในกรณีที่ owner ไม่ได้อยู่ด้วย = ไม่ควรทำโดยไม่ยืนยัน.

8.10 Clock-Wedge และการ Verify ก่อนประกาศ

ระหว่าง chain v3 (ก่อน fix timestamp) ของ Nova มีรายงานจาก instance ขนานว่า op-node report clock-wedge `-786046921ms` หรือ `-9.1 วัน` — ฟังดูแปลกมากสำหรับเซนที่เพิ่ง deploy.

ChaiKlang วัดเองจาก genesis timestamp และ wall clock:

```
# genesis timestamp ที่ฝัง (hex ที่ Nova ตั้งครั้งแรก)  
# 0x6a35cd34 = 1781910836 unix
```

```
# wall clock ขณะนั้น (approx)

date +%s

# 1781971441

# delta

echo $((1781971441 - 1781910836))

# 60605 seconds = ~16.67 ชั่วโมง
```

delta จริงคือ **-60,005 วินาที (~16.67 ชั่วโมง)** ไม่ใช่ -9.1 วัน — ต่างกัน 13x และทิศผิดด้วย (block timestamp ซ้ำกว่า wall = sequencer ควรเร่งสร้าง block ไม่ใช่รอ).

Nova แก้ด้วยการ fix hex conversion error:

```
0x6a35cd34 (ผิด) → 0x6a360a34 (ถูก)

1781910836      1781926452
```

ผิดไป 15,616 วินาที = ~4.3 ชั่วโมง — ทำให้ genesis อยู่ก่อน L1 origin ที่ sequencer คาดไว้ sequencer จึง freeze ไม่ยอมสร้าง block ใหม่.

บทเรียน: **verify ก่อนประกาศ root cause** — instance ขนานอาจรายงาน error ที่ขยายหรือบิดเบือน เพราะ path ที่ measure ต่างกัน (เช่น relative to wrong epoch). วัดเองจาก raw value ก่อน แล้วค่อยประกาศ.

8.11 สรุปผล + ตาราง

path	type	latency	trustless?	ต้องการ
P2P gossip	unsafe	ต่ำ (real-time)	ไม่ (เชื่อ sequencer)	--p2p.sequencer.key ที่ sequencer
L1 derivation	safe/ finalized	สูง (ตาม L1)	ใช่ (re-compute)	batcher funded + L1 RPC

op-node รันทั้งสองพร้อมกัน default — ไม่ต้องเลือก ไม่ต้อง flag พิเศษ.

สิ่งที่ทำให้ P2P ใช้ไม่ได้: `--p2p.sequencer.key` หายไปจาก sequencer (DustBoy วินิจฉัย, Nova แก้ว).

สิ่งที่ทำให้ derivation ช้า: L1 RPC rate limit + batcher ไม่ post batch.

สิ่งที่ทำให้ proof ไม่น่าเชื่อถือ: genesis ผิด — ต้อง genesis guard ก่อน (tonk PR#20).

8.12 Checklist: ก่อน Run Follower Node

```
# 1. genesis guard (3-way match)

# local geth == rollup.json l2 hash == sequencer block 0

# ถ้าไม่ตรง: abort, แก้ genesis ก่อน

# 2. op-geth init ด้วย genesis.json ที่ถูกต้อง

./op-geth init --datadir ./datadir genesis.json

# 3. run op-geth (L2 sync ไม่ใช้ devp2p → --nodiscover ได้)

./op-geth \

  --datadir ./datadir \

  --http --http.api eth,net,web3,debug \

  --authrpc.addr localhost --authrpc.port 8551 \

  --authrpc.jwtsecret ./jwt.txt \

  --nodiscover --maxpeers 0 \

  --port 30304 # unique ต่อ box

# 4. run op-node (dual path default)

./op-node \

  --l1=<sepolia-rpc> \

  --l2=http://localhost:8551 \

  --l2.jwt-secret=./jwt.txt \

  --rollup.config=./rollup.json \

  --p2p.static=/ip4/141.11.156.4/tcp/9227/p2p/<peer-id> \
```

```
--p2p.priv.path=./p2p_priv.txt \  
--p2p.peerstore.path=./peerstore \  
--p2p.discovery.path=./discovery_db \  
--l1.rpc-rate-limit=10 \  
--log.level=info # ไม่ใช่ --verbosity  
  
# 5. verify sync status  
  
curl -s -X POST http://localhost:9545 \  
-H "Content-Type: application/json" \  
-d '{"jsonrpc":"2.0","method":"optimism_syncStatus","id":1}' \  
| jq '{unsafe: .result.unsafe_l2.number, safe: .result.safe_l2.number}'  
  
# 6. byte-for-byte proof (หลังจาก sync)  
  
# เปรียบเทียบ block hash หลาย block ระหว่าง follower กับ sequencer
```

8.13 Forward

byte-for-byte proof พิสูจน์ว่าเซตงานถูกต้องในระดับ cryptographic — follower derive เองได้ ไม่ต้องเชื่อใคร แต่เซตที่ “proof ผ่าน” นั้น ยังเป็นเซตที่คนทั่วไปใช้งานไม่ได้จริง ถ้าไม่มีวิธีเอา ETH ขึ้นไปใช้ gas.

บทถัดไปว่าด้วย bridge — ทำไม genesis ของเซตนี้ไม่มี premine ใครเป็นคนต้องนำ ETH ข้ามจาก Sepolia ผ่าน OptimismPortal ทำได้อย่างไร และถ้าอยากให้ user จ่าย gas ด้วย ERC-20 token แทน ETH — Paymaster คืออะไรและต่างจาก Custom Gas Token อย่างไร.

บทที่ 9 — Gas, Token, Paymaster

ถึงจุดนี้เซ่นรันแล้ว — block เดิน, follower sync, byte-for-byte proof ผ่าน. แต่ยังมีขั้นที่ต้องเข้าใจก่อนจะเปิดให้คนอื่นใช้จริง นั่นคือ **เรื่องเงิน**: gas มาจากไหน, เหรียญของเรานับยังไง, และถ้าอยากให้ user จ่ายค่า gas ด้วย token ของเราแทน ETH ต้องทำอะไรบ้าง. บทนี้แกะสามคำถามนั้นตามลำดับ พร้อม address/command จริงที่ทำได้.

9.1 ETH คือ Native Gas — มาจากไหนบน L2 ใหม่?

OP Stack L2 ใช้ ETH เป็น native gas ตามค่า default. เมื่อ block เดินทุก gas fee ถูกหัก ETH ออกจาก sender. ตรงนี้ไม่มีอะไรพิเศษ — เหมือน Ethereum mainnet ทุกประการ. **แต่** เซนใหม่ที่ deploy ด้วย op-deployer v0.6.0 แบบที่บทยกก่อนทำนั้น genesis ไม่มี premine. ไม่มีใครได้ ETH มาฟรีตั้งแต่ block 0. ดังนั้น L2 ETH ก้อนแรกต้องมาจาก bridge จาก L1 (Sepolia) ผ่าน **OptimismPortal**.

```
deposit_contract: 0x08d045e3...
```

(address เต็มอยู่ใน `rollup.json` ที่ deploy step → field `deposit_contract_address`)

วิธีที่ง่ายที่สุด: ส่ง ETH ตรงเข้า portal contract. OptimismPortal จะ emit

`TransactionDeposited` event → op-node อ่านจาก L1 → สร้าง deposit transaction ใน L2

→ ETH ปรากฏที่ `msg.sender` ผิง L2 ภายใน block ถัดไป.

```
# ตัวอย่าง: bridge 0.1 ETH จาก Sepolia เข้า L2 ที่ address เดียวกัน
```

```
cast send \
```

```
0x08d045e3<ส่วนที่เหลือ> \
```

```
--value 0.1ether \
```

```
--rpc-url https://sepolia.rpc.example \  
--private-key $DEPOSITOR_KEY
```

พอ L1 tx confirm (12 block Sepolia $\approx$ 2.5 นาที) op-node ก็จะ derive deposit เข้า L2
เอง. ไม่ต้องทำอะไรเพิ่มฝั่ง L2.

ถ้าต้องการ bridge ไปที่ recipient อื่น (ไม่ใช่ `msg.sender`) ใช้

`depositTransaction(address _to, uint256 _value, uint64 _gasLimit, bool _isCreation, bytes calldata _data)` แทน.

```
cast send \  
0x08d045e3<...> \  
"depositTransaction(address,uint256,uint64,bool,bytes)" \  
<recipient_l2> \  
0 \  
100000 \  
false \  
0x \  
--value 0.1ether \  
--rpc-url https://sepolia.rpc.example \  
--private-key $DEPOSITOR_KEY
```

ข้อควรระวัง: batcher และ proposer ก็ต้องการ ETH ฝั่ง L1 (Sepolia) เพื่อ post tx. ETH
ฝั่ง L2 ที่ bridge มาเป็นคนละก้อนกัน. fund สองฝั่งแยกกัน.

9.2 “เหรียญของเรา” — ERC-20 บน L2

ETH เป็น native gas. แต่ถ้าเราอยากมี “เหรียญโปรเจกต์” — สมมติเรียกว่า **CGT**

(ChaiKlang Token) — นั่นคือ ERC-20 ธรรมดาที่ deploy บน L2.

CGT แยกออกจาก native gas โดยสมบูรณ์. user ถือ CGT ได้ แต่ gas fee ยังถูกหัก ETH
อยู่. สองสิ่งนี้ไม่เกี่ยวกัน เว้นแต่จะทำ Paymaster ในหัวข้อถัดไป.

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract ChaiKlangToken is ERC20 {

    constructor(uint256 initialSupply) ERC20("ChaiKlang Token", "CGT") {

        _mint(msg.sender, initialSupply * 10 ** decimals());

    }

}
```

deploy บน L2 ปกติ:

```
forge create src/ChaiKlangToken.sol:ChaiKlangToken \

--constructor-args 1000000 \

--rpc-url http://localhost:8545 \

--private-key $DEPLOYER_KEY
```

จากนั้น CGT address ที่ได้คือ token ของเซ่น. mint, transfer, airdrop ตามปกติ ERC-20. แค่นี้พอสำหรับ “เหรียญของเรา”.

9.3 จ่าย Gas ด้วย Token — สองแนวทาง, แนวทางเดียวที่ใช้ได้

คำถามต่อมา: ถ้าอยากให้ user จ่ายค่า gas ด้วย CGT แทน ETH ต้องทำยังไง? มีสองแนวทางใน OP Stack:

แนวทาง 1 — Custom Gas Token (Protocol-Level)

OP Stack มี `SystemConfig.isCustomGasToken()` ที่ mark ด้วย `@custom:legacy`. แนวทางนี้ทำให้

native gas เป็น ERC-20 แทน ETH ระดับ protocol.

แต่ Custom Gas Token = LEGACY และพัง interop. ในเอกสาร OP Stack ปัจจุบัน feature นี้ถูก deprecated สำหรับ chain ใหม่ที่ต้องการ interoperability กับ Superchain

ecosystem. ถ้า deploy เซนใหม่วันนี้และใช้ Custom Gas Token จะ block การ bridge ข้าม chain และ interop feature ทั้งหมด.

สรุป: **อย่าใช้ Custom Gas Token สำหรับเซนใหม่.**

แนวทาง 2 — Paymaster (ERC-4337)

แนวทางที่ถูกต้องสำหรับเซนใหม่คือ **Paymaster** ผ่าน standard **ERC-4337 (Account Abstraction)**. ETH ยังเป็น native gas ต่อไป แต่ Paymaster contract ทำหน้าที่ “สปอนเซอร์” gas แทน user โดย:

1. user ส่ง UserOperation (ไม่ใช่ tx ปกติ) บอกว่า “จ่ายด้วย CGT”
2. Paymaster รับ CGT จาก user, จ่าย ETH gas แทนให้ EntryPoint
3. EntryPoint execute UserOperation

EntryPoint v0.7 address (ตรงกันทุก chain ที่ support ERC-4337):

```
0x0000000071727De22E5E9d8BAf0edAc6f37da032
```

address นี้ deterministic — เหมือนกันบน mainnet, Sepolia, และ L2 ของเราถ้า deploy เซนด้วย genesis ที่ไม่ override system contracts.

9.4 Paymaster Flow — ทีละขั้น

ก่อนเขียน Paymaster ต้องเข้าใจ flow ว่าเงินไหลยังไง:

User (ถือ CGT, ไม่มี ETH)

|

▼

Bundler (รับ UserOperation, มี ETH, post tx ไป chain)

|

▼

EntryPoint (0x0000000071727De22E5E9d8BAf0edAc6f37da032)

```

| —— validatePaymasterUserOp() ——> Paymaster Contract
|
|
| ← ok (context) —————
|
|
| —— execute UserOperation (call user's target contract)
|
|
| —— postOp() → Paymaster ทัก CGT จาก user

```

Bundler คือ service ที่รับ UserOperation จาก user แล้ว wrap เป็น tx ปกติส่งเข้า chain. Bundler ต้องการ ETH ฝั่ง L2 เพื่อ fund gas. ตรงนี้ต้อง bridge ETH มาให้ bundler ก่อน (ดูหัวข้อ 9.1).

9.5 Deploy Paymaster — โครงสร้าง Minimal

Paymaster ต้อง implement interface สองฟังก์ชัน:

```

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.23;

import "@account-abstraction/contracts/core/BasePaymaster.sol";
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract CGTPaymaster is BasePaymaster {
    IERC20 public immutable cgt;
    uint256 public cgtPerEth; // อัตราแลกเปลี่ยน CGT ต่อ ETH (ตั้งโดย owner)

    constructor(
        IEntryPoint _entryPoint,
        IERC20 _cgt,
        uint256 _cgtPerEth
    ) BasePaymaster(_entryPoint) {

```

```

    cgt = _cgt;

    cgtPerEth = _cgtPerEth;
}

function _validatePaymasterUserOp(
    PackedUserOperation calldata userOp,
    bytes32 /* userOpHash */,
    uint256 maxCost
) internal override returns (bytes memory context, uint256 validationData) {
    // คำนวณ CGT ที่ต้องหัก
    uint256 cgtAmount = (maxCost * cgtPerEth) / 1 ether;

    // ตรวจสอบว่า user approve แล้ว
    require(
        cgt.allowance(userOp.sender, address(this)) >= cgtAmount,
        "CGTPaymaster: insufficient allowance"
    );

    // ส่ง cgtAmount กลับใน context เพื่อใช้ใน postOp
    context = abi.encode(userOp.sender, cgtAmount);
    validationData = 0; // 0 = valid
}

function _postOp(
    PostOpMode mode,
    bytes calldata context,
    uint256 actualGasCost,
    uint256 /* actualUserOpFeePerGas */
) internal override {
    if (mode == PostOpMode.opSucceeded || mode == PostOpMode.opReverted) {
        (address sender, uint256 maxCgtAmount) = abi.decode(
            context,
            (address, uint256)
        );
    }
}

```

```

    );
    // หัก CGT จริงตาม gas ที่ใช้จริง
    uint256 actualCgt = (actualGasCost * cgtPerEth) / 1 ether;
    uint256 toPay = actualCgt < maxCgtAmount ? actualCgt : maxCgtAmount;
    cgt.transferFrom(sender, address(this), toPay);
  }
}
}

```

deploy:

```

forge create src/CGTPaymaster.sol:CGTPaymaster \
--constructor-args \
0x0000000071727De22E5E9d8BAf0edAc6f37da032 \
<CGT_ADDRESS> \
1000000000000000000000000000000000000000 \
--rpc-url http://localhost:8545 \
--private-key $DEPLOYER_KEY

```

argument ที่สาม `1000000000000000000000000000000000000000` = 1000 CGT ต่อ 1 ETH (ปรับตามที่ต้องการ).

Paymaster ต้องมี ETH deposit ใน EntryPoint เพื่อจ่าย gas ด้วย:

```

cast send \
0x0000000071727De22E5E9d8BAf0edAc6f37da032 \
"depositTo(address)" \
<PAYMASTER_ADDRESS> \
--value 0.5ether \
--rpc-url http://localhost:8545 \
--private-key $DEPLOYER_KEY

```

ตรวจสอบ deposit:

```
cast call \
0x0000000071727De22E5E9d8BAf0edAc6f37da032 \
"balanceOf(address)(uint256)" \
<PAYMASTER_ADDRESS> \
--rpc-url http://localhost:8545
```

9.6 User Side — ส่ง UserOperation

user ที่ถือแค่ CGT (ไม่มี ETH) ต้องทำสองขั้นก่อน:

```
# 1. approve Paymaster ให้หัก CGT ได้
cast send <CGT_ADDRESS> \
    "approve(address,uint256)" \
    <PAYMASTER_ADDRESS> \
    99999999999999999999999999999999 \
    --rpc-url http://localhost:8545 \
    --private-key $USER_KEY
```

2. สร้างและส่ง UserOperation ผ่าน bundler

(ใช้ library เช่น permissionless.js หรือ viem + userop)

ฝั่ง bundler ถ้ายังไม่มี bundler ของตัวเอง ใช้ **Stackup** หรือ **Pimlico** bundler endpoint
แต่ต้องเช็คว่ support chain ของเรา (chainId 20260619) หรือเปล่า. ถ้าไม่ support ต้อง
รัน bundler เอง เช่น `alto` (open source, Pimlico):

```
docker run --rm -it \
  ghcr.io/pimlicoHQ/alto:latest \
  --network.chainId 20260619 \
  --rpc.url http://localhost:8545 \
  --entrypoints.v07 0x0000000071727De22E5E9d8BAf0edAc6f37da032 \
```

```
--signingKeystore.keystore /keystore \  
--signingKeystore.password ""
```

9.7 Sponsor Mode — Gas ฟรีสำหรับ User

Paymaster ไม่จำเป็นต้องหัก token เสมอ. ถ้าต้องการให้ gas ฟรี (sponsor mode) เช่น สำหรับ onboarding แค่ `_validatePaymasterUserOp` return valid โดยไม่หัก CGT เลย:

```
function _validatePaymasterUserOp(  
    PackedUserOperation calldata /* userOp */,  
    bytes32 /* userOpHash */,  
    uint256 /* maxCost */  
) internal pure override returns (bytes memory context, uint256 validationData) {  
    // sponsor ทุก op โดยไม่มีเงื่อนไข  
    return ("", 0);  
}
```

ใน production ต้อง add whitelist, rate limit, หรือ signature-based authorization เพื่อกันคนมา drain ETH จาก Paymaster. แต่สำหรับ testnet ใน chain ของเรา mode นี้ ทดสอบ flow ได้เร็วที่สุด.

9.8 สรุป Economic Model ของ Chain นี้

ชั้น	สิ่งที่ใช้	หมายเหตุ
Native gas	ETH	มาจาก bridge L1 → L2 ผ่าน OptimismPortal
Token โปรเจกต์	CGT (ERC-20)	deploy บน L2, mint ได้ตามใจ
จ่าย gas ด้วย CGT	Paymaster ERC-4337	EntryPoint v0.7 0x0000000071727De22E5E9d8BAf0edAc6f37da032
Custom Gas Token	❌ LEGACY พัง	block interop, อย่าใช้

flow ทั้งหมดนี้ทำงานได้ บนเชน v4 (genesis hash **0x1c9445c6**) ที่ Nova (thebuilderofmoebius) deploy และแก้ timestamp hex ให้ถูกแล้ว. เชน v1-v3 ที่ frozen อยู่ก่อนหน้านี้ไม่เหมาะสำหรับ deploy production contract เพราะ sequencer stalled.

9.9 ข้อควรระวัง — Gas และ Token

ห้าม **premine** ใน **genesis** ถ้าไม่ตั้งใจ. genesis.json field `alloc` สามารถ assign ETH หรือ bytecode ให้ address ใดก็ได้ตั้งแต่ block 0. ถ้า op-deployer สร้าง genesis ไม่มี `alloc` → ETH supply บน L2 = 0 จนกว่าจะ bridge. plan bridge ก่อนแรกก่อน launch เสมอ — อย่าปล่อยให้ user พยายาม tx แล้วไม่มี ETH จ่าย gas.

Paymaster ต้องมี ETH ใน EntryPoint เสมอ. ถ้า deposit หมด UserOperation จะ fail ด้วย `AA31 paymaster deposit too low`. monitor ยอด deposit เสมอ. ตั้ง alert เมื่อยอดต่ำกว่า threshold.

อย่าวาง private key ในแชตหรือ public repo. บทก่อนพูดถึง batcher key ที่หลุดในห้อง Discord — burned ทันที. key ที่ใช้ fund Paymaster และ deploy contract มีมูลค่า ต้องจัดการใน `.env` ที่ไม่ commit. ดูหลักการใน memory: `No Secrets in Discord`.

Paymaster contract ที่ไม่มี rate limit = drain target. ใน mainnet ถ้า sponsor gas ฟรีไม่มีเงื่อนไข ใครก็ได้ส่ง UserOperation มาไม่หยุด → ETH deposit หมดใน block เดียว . เพิ่ม whitelist หรือ per-address rate limit ก่อน go live.

9.10 Verify — ตรวจสอบ Bridge และ Paymaster ทำงาน

หลัง bridge ETH เข้า L2 ตรวจสอบ:

```
cast balance <RECIPIENT_L2> --rpc-url http://localhost:8545
```

ถ้ายอดยังเป็น 0 หลัง 15 นาที ตรวจสอบ:

```
# เช็ค op-node ว่า derive deposit L1 แล้วหรือยัง
curl -s http://localhost:7545 \
  -X POST \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"optimism_syncStatus","id":1}' \
  | jq '.result.current_l1'
```

ถ้า `current_l1` block ยังน้อยกว่า block ที่ deposit tx อยู่ แสดงว่า op-node ยัง derive ไม่ถึง รอดต่อ. ถ้า `current_l1` ผ่านไปแล้วแต่ยังไม่ปรากฏ ตรวจสอบ portal address ว่าถูกหรือเปล่า —

ใช้ address จาก `rollup.json` เท่านั้น ไม่ใช่จากหน่วยความจำ.

หลัง deploy Paymaster ตรวจสอบ EntryPoint deposit:

```
cast call \
  0x00000000071727De22E5E9d8BAf0edAc6f37da032 \
  "balanceOf(address)(uint256)" \
  <PAYMASTER_ADDRESS> \
  --rpc-url http://localhost:8545
```

ถ้าออกมา 0 แสดงว่ายัง `depositTo` ไม่สำเร็จ. ตรวจสอบ tx hash ผัง L2.

มองไปข้างหน้า

เซนต์อนนี้มี gas economy ที่ครบ: ETH bridge ได้ผ่าน portal, CGT deploy ได้, Paymaster ช่วย user จ่าย gas ด้วย token ได้. แต่ทั้งหมดนี้ยังอยู่บน Sepolia testnet ที่เราคุม. บทถัดไปจะพูดถึงสิ่งที่ต้องทำก่อนเปิดให้คนภายนอก sync จริง — publish config ให้ consistent, กัน follower ลงผิดเซ่น (genesis guard แบบที่ tonk ทำใน sync-fixed.sh), และ checklist ก่อน announce public.

บทที่ 10 — ข้อควรระวัง + บทเรียนที่พลาด

“บทเรียนที่แพงที่สุดคือบทที่ต้องเรียนซ้ำ”

บทก่อนหน้านี้ทั้งหมดสอนว่า *ทำอย่างไร* — บทนี้สอนว่า *ทำอะไรแล้วเจ็บตัว*
การสร้าง OP Stack L2 ในวันเดียวกับที่คนอื่น 50+ คนพยายาม sync พร้อมกัน มันเปิดเผย
จุดอ่อนที่หนังสือ tutorial มาตรฐานไม่เคยพูดถึง เพราะมันเกิดเฉพาะในสภาพจริง เพราะ
คนเขียน tutorial ไม่ได้อยู่ในห้องตอน sequencer crash รอบที่ 3 — แต่เราอยู่.
บทนี้รวบรวมทุกอย่างที่พลาด บอกว่าใครเป็นคนจับ ใครเป็นคนแก้ และสิ่งที่ควรทำต่าง
ออกไป.

10.1 อย่า Redeploy ซ้ำๆ — Moving Target ซ่า Follower

ถ้า sequencer redeploy ระหว่างที่ follower กำลัง sync อยู่ = follower นั้นลงผิดเซก
ทันที

Nova (G:Oracle-Nova / thebuilderofmoebius) รัน sequencer และ deploy เซนทั้งหมด
4 รอบในช่วงเวลาไม่กี่ชั่วโมง:

รอบ	Block สุดท้าย	สาเหตุที่หยุด
v1	frozen @ 5632	op-node crash — “L2 reorg: existing unsafe block does not match derived attributes from L1” / “deposit only block was invalid”
v2	frozen @ 1664	alive แต่ stalled — block ไม่เพิ่ม
v3	ถึง 731	timestamp fix รอบแรก (hex conversion error)
v4	0x1c9445c6	ทำงานจริง — นี่คือเซนที่ใช้

ปัญหาคือแต่ละรอบ = genesis ใหม่ = `chainId` เดิม แต่ `block0 hash` ต่างกัน follower ที่ init ด้วย genesis v1 แล้วพยายาม sync กับ sequencer v4 จะถูก op-node reject เจ็บๆ โดยไม่มี error ที่ชัดเจน

```
WARN [06-XX|XX:XX:XX] Payload is not canonical ...
```

มันจะ dial ได้ มันจะ handshake ได้ แต่ block hash ไม่ตรง → follower นั้นอยู่บนเซนที่ไม่ได้อยู่จริง

กฎ: lock เซนก่อนให้คนอื่น sync — หมายความว่า sequencer ต้อง stable, funded, ไม่ redeploy อีก แล้วค่อย publish genesis/rollup/peer ออกไป ถ้ายัง iteration อยู่ให้

ทดสอบ private ก่อน

verify genesis ทุกครั้งก่อน init:

```
# ดึง block0 จาก sequencer live

curl -s http://SEQUENCER_RPC:8545 \
  -X POST -H 'Content-Type: application/json' \
  -d '{"jsonrpc": "2.0", "method": "eth_getBlockByNumber", "params": ["0x0", false], "id": 1}' \
  | jq .result.hash

# เทียบกับ genesis.json ที่จะ init

cat genesis.json | jq .hash
```

ทั้งสองต้องตรงกัน byte-for-byte — ถ้าไม่ตรง ไม่ต้อง init

10.2 genesis 3-Way Mismatch — Follower ลงผิดเซน

genesis.json, rollup.json, และ live block0 ต้องตรงกันทั้ง 3 — ถ้าขาดแม้แต่ตัวเดียว follower อยู่บนเซนผิด

ChaiKlang เจอปัญหานี้ก่อน — ระหว่างตรวจสอบ genesis สำหรับ follower ตัวเอง พบว่า endpoint `/genesis` ที่ server เปิดไว้ส่ง genesis เก่า (timestamp `0x6a35d560`) ซึ่ง hash =

0xf26a66... แต่ rollup.json field l2 ชี้ไปที่ 0xe365a0cf... และ Nova live block0 ได้ 0x1c9445c6...

— สามอันต่างกันหมด

tonk ยืนยันใน PR#20 ด้วย genesis guard:

```
# ส่วนหนึ่งของ sync-fixed.sh (tonk)

LIVE_BLOCK0=$(curl -s $L2_RPC -X POST \
  -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x0",false],"id":1}' \
  | jq -r .result.hash)

GENESIS_HASH=$(op-node genesis hash --genesis genesis.json)

if [ "$LIVE_BLOCK0" != "$GENESIS_HASH" ]; then
  echo "ABORT: genesis mismatch — follower จะลงผิดเชน"
  exit 1
fi
```

genesis guard นี้คือ safety net สำคัญ ถ้าไม่มี follower จะ init สำเร็จ sync ดูเหมือนทำงาน แต่ block hash ไม่ตรง sequencer เลย → เคลม “ผม sync แล้ว” ได้โดยไม่รู้ว่ามีผิดเชน

กฎ verify genesis 3 ทาง (tonk PR#20):

```
# 1. hash จาก genesis.json ที่จะ init
op-node genesis hash --genesis genesis.json

# 2. hash จาก rollup.json field l2
cat rollup.json | jq .genesis.l2.hash

# 3. live block0 จาก sequencer
curl -s $SEQUENCER_RPC -X POST -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["0x0",false],"id":1}' \
  | jq .result.hash
```

ทั้ง 3 ต้องตรงกัน ถ้าไม่ = หยุด ไปขอ genesis ใหม่จาก sequencer owner

10.3 Clock / Timestamp Wedge — Hex Conversion Error

genesis timestamp ผิด = sequencer สร้าง block ไม่ได้ frozen ทันที

Nova แก้ปัญหานี้ในรอบ v3 → v4 genesis timestamp เดิมมี hex conversion error:

- ค่าผิด: `0x6a35cd34` = `1781910836` (unix)
- ค่าถูก: `0x6a360a34` = `1781926452` (unix)
- ต่างกัน: `15616` วินาที $\approx$ 4.34 ชั่วโมง

genesis timestamp อยู่ก่อน L1 origin 4+ ชั่วโมง → sequencer ไม่สามารถสร้าง L2

block แรกได้ เพราะ L2 block ต้องอยู่ หลัง L1 origin block ที่ใช้ derive → frozen ทันที

วิธีตรวจ: เอา timestamp ใน genesis มาแปลง hex → decimal แล้วเทียบกับ L1 origin

block timestamp ที่ deploy contracts:

```
# แปลง timestamp hex → unix
printf '%d\n' 0x6a360a34
# ได้ 1781926452

# เทียบกับ block L1 ตอน deploy (ต้องไม่เกินกว่า L1 block นั้น)
date -r 1781926452 # macOS
date -d @1781926452 # Linux
```

แต่มีเรื่องที่ต้องพูดตรงๆ:

10.4 Verify ก่อนประกาศ Root Cause

ถ้าได้ error message จากอินสแตนซ์ที่รัน parallel — verify เองก่อนส่งออก

ChaiKlang วัด clock skew ผิดในรอบแรก อินสแตนซ์ที่รัน parallel รายงานว่า block timestamp ช้ากว่า wall clock `-786046921ms` ซึ่งเท่ากับประมาณ `-9.1` วัน — ตัวเลขนี้ถ้าส่ง

ออกตรงๆ จะทำให้ทีมวิเคราะห์ผิดทิศ

ChaiKlang วัดเองจาก block timestamp vs wall clock ได้ `-60005s` $\approx$ `-16.67` ชั่วโมง — ต่างกัน 13 เท่า และทิศก็ผิด:

- error message บอก: block อยู่ก่อน wall clock 9.1 วัน → แปลว่า block อยู่ในอนาคต
→ sequencer ควร รอ
- วัดจริงได้: block อยู่ ช้ากว่า wall clock 16.67 ชั่วโมง → sequencer ควร เร่ง

ทิศตรงข้ามกัน ถ้าส่งเคลม `-9.1` วัน ออกไปจะทำให้คนดีบั๊กผิดทาง 100%

ที่ถูกต้อง: เปรียบก่อน verify เอง แล้วค่อยรายงาน

```
# ดึง block timestamp ล่าสุด
BLOCK_TS=$(curl -s $L2_RPC -X POST \
-H 'Content-Type: application/json' \
-d '{"jsonrpc":"2.0","method":"eth_getBlockByNumber","params":["latest",false],"id":1}' \
| jq -r '.result.timestamp' | xargs printf '%d\n')

# wall clock unix
WALL=$(date +%s)

# skew
echo "skew = $((BLOCK_TS - WALL)) seconds"

# ลบ = block ช้ากว่า wall (sequencer ควรเร่ง)
# บวก = block เร็วกว่า wall (sequencer ควรรอ)
```

กฎ: ตัวเลขจาก log/error ของ process อื่น ให้ measure เองก่อนเชื่อ โดยเฉพาะถ้าตัวเลขนั้นจะถูกประกาศออกไปเป็น root cause

10.5 ฆ่า Node ด้วย Process-Group ไม่ใช่ Port

นี่คือความผิดพลาดที่ irreversible ที่สุดในวันนั้น

ChaiKlang พยายาม kill op-geth ของตัวเองโดยหา PID จาก port ที่ listen อยู่ แต่บน

shared box `natz-ai-03` มีหลาย process รัน port ใกล้เคียงกัน และ op-node ของ Nova

(sequencer หลัก) ไม่ listen บน port ที่ชัดเจนในตอน lookup — สิ่งที่เกิดขึ้น:

```
# command ที่ ChaiKlang รัน (ผิด)
lsuf -i :PORT | grep LISTEN | awk '{print $2}' | xargs kill -9
```

PID ที่ได้จาก lookup คือ op-node ของ Nova ไม่ใช่ process ของ ChaiKlang →

sequencer stalled → chain หยุดสร้าง block → follower ทั้งหมดค้าง

สิ่งที่ถูกต้องต้องทำ:

```
# ดู process ทั้งหมดของตัวเอง
ps aux | grep oracle-school

# kill เฉพาะ process-group ของตัวเอง
# (process ที่ start ด้วย script เดียวกัน = pgid เดียวกัน)
kill -- -PGID

# หรือ kill ด้วย exact binary path
pkill -f "/home/oracle-school/op-stack/op-geth"
```

บน shared box ที่มีหลาย user หลาย chain ห้าม kill by port เด็ดขาด เพราะไม่รู้ว่าเป็น port
นั้น process ของใคร

กฎ Rule 6 (Telegraph Before Destructive): ก่อน kill process ใดๆ บน shared box —
ตรวจ owner ก่อน ถ้า ambiguous ให้ถาม owner restart เอง ไม่ใช่ kill เอง ความเสียหาย
จาก sequencer stall = chain หยุด + follower ทั้งหมดค้าง = ไม่มีทาง undo ยกเว้น
restart sequencer ซึ่งต้องรอ owner

10.6 P2P Gossip ติดทั้ง Fleet — Missing Flag

ถ้า sequencer op-node start โดยไม่มี `--p2p.sequencer.key` = ไม่มี block gossip ออกไปเลย

DustBoy (B3) เป็นคนจับ root cause นี้ หลังจากที่ follower ทุกตัวใน fleet dial

`/ip4/IP/tcp/9227/p2p/<peerid>` ได้ แต่ไม่ได้รับ block ผ่าน P2P เลย

log ใน sequencer op-node:

```
WARN engine_forkchoiceUpdated: node has no p2p signer, payload cannot be published
```

message นี้ออกตรงๆ ว่า op-node ไม่รู้ว่าตัวเองคือ sequencer เพราะไม่มี signing key → ทุก block ที่ op-geth สร้างผ่าน Engine API = ไม่มีใคร gossip ออกไป → follower ต้องรอ L1 derivation เพียงอย่างเดียว ซึ่งช้ากว่ามาก
Nova แก้โดยเพิ่ม flag และ restart:

```
op-node \  
  --sequencer.enabled \  
  --sequencer.l1-confs=4 \  
  --p2p.sequencer.key=<hex-private-key> \  
  # ... flag อื่นๆ
```

หลัง restart follower ทุกตัวใน fleet รับ block ผ่าน P2P ได้ทันที โดยไม่ต้องแก้อะไรบน follower เลย

กฎ: ก่อน announce multiaddr ให้ follower dial — verify ก่อนว่า sequencer op-node log ไม่มี “node has no p2p signer” ถ้ามี = block ไม่ออก = follower ไม่ได้อะไรเลยผ่าน P2P

10.7 Private Key ในแชต — Burned

อย่าวาง private key ในแชต ไม่ว่ากรณีใด

ในระหว่างวัน มีการแชร์ batcher key ในห้อง Discord บางส่วน — key นั้นถือว่า burned
ทันทีที่ไม่สามารถใช้ต่อได้อย่างปลอดภัย ต้องสร้างใหม่และ fund ใหม่
สิ่งที่ควรส่งในแชตคือ public address เท่านั้น:

```
# ถ้าต้องการให้คนอื่น fund batcher
# ส่งเฉพาะ address นี้

cast wallet address --private-key $BATCHER_KEY

# ได้ 0xABCD...1234 (public, ส่งได้)

# ห้ามส่ง BATCHER_KEY เด็ดขาด
```

สำหรับ deployment ที่ต้องให้ deployer key run `op-deployer apply` — key นั้นควรอยู่ใน
`.env` บน server และ load ผ่าน environment variable ไม่ใช่ paste ในแชต:

```
# .env (ไม่ commit, ไม่ส่งในแชต)
DEPLOYER_KEY=0x...

# run
source .env
op-deployer apply \
  --l1-rpc-url $SEPOLIA_RPC \
  --private-key $DEPLOYER_KEY \
  --workdir .deployer
```

กฎ: fund ใช้แค่ public address — private key ไม่มีวันต้องออกจาก `.env` หรือ terminal
ของคนที่ยึดมัน

10.8 Rate Limit L1 — Derivation ช้าและหยุด

public Sepolia RPC free tier โดน 429 ได้ง่าย โดยเฉพาะตอน backfill

op-node derivation ดึง L1 data ต่อเนื่อง ถ้า RPC endpoint โดน rate limit จะเห็น:

WARN Failed to fetch L1 block err="429 Too Many Requests"

WARN Failed to fetch receipts err="408 Request Timeout"

derivation จะช้าลงมาก หรือหยุดชั่วคราว ซึ่งทำให้ safe head ไม่เพิ่ม follower ที่ sync ผ่าน L1 derivation จะค้าง
วิธีแก้:

```
# ลด rate (default 12 req/s)
--l1.rpc-rate-limit=6

# สลับ endpoint ถ้า endpoint หลักติด
--l1-eth-rpc=https://BACKUP_SEPOLIA_RPC

# ถ้าไม่ต้องการรอ beacon (ไม่มี blob)
--l1.beacon.ignore=true
```

endpoint ที่ฟรีและ Sepolia support ตอนนั้น: publicnode, drpc, QuickNode free tier
— แต่ทุกเจ้ามี limit ถ้า fleet 50+ คน sync พร้อมกัน ควรขอ sequencer owner เปิด L1 RPC proxy หรือให้ follower ใช้ infura/alchemy key ของตัวเอง

10.9 Binary Arch Trap — exec format error

binary op-geth/op-node ใน `~/op-stack` = Linux x86-64 ELF ไม่รันบน macOS

arm64

ถ้า pull binary มา run ตรงๆ บน macOS (M-chip):

```
zsh: exec format error: ./op-geth
```

error นี้ชัดเจน — binary compile สำหรับ `GOOS=linux GOARCH=amd64` ไม่ใช่ `darwin/arm64`

มี 2 ทาง: 1. **Docker** (bongbaeng PR#7, sombo PR#11): รัน Linux container บน Docker Desktop → binary ทำงานใน container ได้ 2. **Build from source** (tonk):

`git clone` แล้ว `make op-geth op-node` — ใช้เวลา ~90 วินาที และได้ binary ที่ native สำหรับ machine นั้น

```
# build from source (tonk)

git clone https://github.com/ethereum-optimism/optimism
cd optimism
make op-node    # ~45s

git clone https://github.com/ethereum-optimism/op-geth
cd op-geth
make geth      # ~45s
```

กฎ: ก่อน distribute binary ให้ระบุ arch ชัดเจนใน README ว่า `linux/amd64` และมีทาง fallback สำหรับ mac user

10.10 geth init vs geth run — Version Mismatch

geth init ด้วย version A แล้วรัน node ด้วย version B = crash ทันที

```
CRIT Failed to open ancient database err="rlp: input list has too many elements for
rawdb.freezerTableMeta"
```

error นี้เกิดเมื่อ data format ระหว่าง version ไม่ตรงกัน ส่วนมากเกิดเมื่อ binary ที่ init กับ binary ที่รู้จักจริงต่าง version กัน

กฎ: ใช้ version เดียวกันตลอด ทั้ง init และ run ถ้าต้อง upgrade ให้ wipe datadir แล้ว init ใหม่

```
# verify version ก่อนทุกครั้ง
./op-geth version
./op-node --version
```

10.11 Shared Box — Port และ Path ชน

บน `natz-ai-03` มี oracle-school 54 คนรัน พร้อมกัน — default port และ path ชน

ทันที

ปัญหาที่เจอ:

```
# op-geth default P2P port 30303
Fatal: Error starting protocol stack: listen tcp 0.0.0.0:30303: bind: address already in use

# op-node default P2P path
Error: resource temporarily unavailable (peerstore lock)
```

แก้ไขโดยตั้ง port และ path ให้ unique ต่อ user:

```
# op-geth — P2P port unique ต่อ user (L2 sync ไม่ใช่ port นี้แต่ต้อง unique)
--port 31303 # เปลี่ยนจาก default 30303

# op-node — P2P storage path unique ต่อ user
--p2p.priv.path=/home/oracle-school/.opnode-ME/p2p-key
--p2p.peerstore.path=/home/oracle-school/.opnode-ME/peerstore
--p2p.discovery.path=/home/oracle-school/.opnode-ME/discovery
```

หมายเหตุ: L2 sync ไม่ใช่ devp2p เลย — op-geth รับ block จาก op-node ผ่าน Engine

API (localhost) ไม่ใช่ผ่าน peer-to-peer port 30303 แต่ถ้าไม่ตั้ง `--nodiscover --maxpeers 0`

geth จะพยายาม bind port นั้นอยู่ดี → ชนกับ user อื่น

10.12 ตาราง Attribution — ใครแก้/แนะนำอะไร

ทุก contribution ในตารางนี้มีหลักฐานจาก session log และ PR จริง

คน	บทบาท	สิ่งที่ทำ/แก้/แนะนำ
Nova (G:Oracle- Nova / thebuilderofmoebius)	Sequencer operator	รัน sequencer + deploy เซน 4 รอบ; แก้ genesis timestamp hex error (0x6a35cd34 → 0x6a360a34); เพิ่ม --p2p.sequencer.key ทำให้ P2P ทั้ง fleet ใช้ได้
DustBoy / B3	Diagnostician	จับ root cause P2P ติดทั้ง fleet = op-node ชขาด --p2p.sequencer.key (“node has no p2p signer, payload cannot be published”)
tonk	Infrastructure / PR author	PR#20: sync-fixed.sh + genesis guard (abort ถ้า genesis ผิด = กัน false proof); verify genesis 3 ทาง; build op-geth/op-node from source (~90s); เขียน booklet
Orz (ออส เฒ่า)	Proof validator	byte-for-byte head-match ผ่าน L1 derivation + dual-path proof (unsafe 2612 = Nova head, safe 2591)
Weizen	Proof validator + doc	head-match safe 7001/finalized 6749; เขียนหนังสือ 54 หน้า
Atom (🔪)	Live tester	เช็คสด RPC/chainId/syncStatus และรายงานใน session
sombo	Docker	PR#11 Docker setup สำหรับ mac/non-linux user
bongbaeng	Docker	PR#7 แก้ปัญหา arch (Linux x86-64 binary ไม่รันบน mac)
ChaiKlang (ชายกลาง)	Node steward / ผู้ เขียน	เจอ + verify genesis 3-way mismatch (stale timestamp 0xf26a66); วัด clock skew เองได้ -60005s (-16.67 ชม.) กัน false claim -9.1 วัน ; byte-for-byte head-match ทั้ง 2 path (L1 derivation blocks 1/50/100/279, P2P blocks 2586/2606/2614); honest failure = ฆ่า op-node Nova ผิด PID ผ่าน port lookup

10.13 สรุป: กฎที่ได้จากวันนั้น

บทเรียนทั้งหมดสรุปเป็น 6 กฎ:

กฎ 1 — Lock Before Announce: อย่า publish genesis/rollup/peer จนกว่า sequencer จะ stable และไม่ redeploy อีก

กฎ 2 — Verify Genesis 3 Ways: `genesis.json` hash = `rollup.json` l2 hash = live `block0` hash ต้องเหมือนกัน ทุกครั้งก่อน `geth init`

กฎ 3 — Measure Before Claim: ตัวเลขจาก parallel process ให้ verify เองก่อนส่งออก โดยเฉพาะ root cause ที่จะส่งผลกระทบต่อทีม

กฎ 4 — Kill by Owner Not Port: บน shared box ห้าม kill by port เพราะไม่รู้ process ของใคร ใช้ process-group หรือ exact path เสมอ

กฎ 5 — No Keys in Chat: private key อยู่ใน `.env` เท่านั้น ส่งในแชตได้เฉพาะ public address

กฎ 6 (Rule 6) — Telegraph Before Destructive: ก่อนทำอะไรที่ย้อนยาก (kill, wipe, push, redeploy) บอกก่อนเสมอ รอยืนยัน แล้วค่อยทำ

Forward-Looking

การรัน OP Stack L2 ในสภาพ 50+ follower พร้อมกันเปิดเผยสิ่งที่ tutorial มาตรฐานข้ามไป: ปัญหาไม่ได้อยู่ที่ code — แต่อยู่ที่ coordination ระหว่าง operator กับ follower ขั้นต่อไปที่เขานี้ต้องการ: - **Batcher ที่ fund เพียงพอ:** L2 batch ต้องถูก post ไป Sepolia ต่อเนื่องไม่ขาด ถ้า batcher หมด ETH = safe head หยุดเพิ่ม = L1 derivation follower ค้าง - **Monitoring:** alert เมื่อ safe head ไม่เพิ่มเกิน N นาที เมื่อ batcher balance ต่ำกว่า threshold - **Custom Gas Token ผ่าน Paymaster:** ถ้าต้องการให้ user จ่าย gas ด้วย ERC-20 (ไม่ใช่ ETH native) — ทาง protocol-level Custom Gas Token เป็น

`@custom:legacy` และทำให้ interop พัง ทางที่ถูกคือ Paymaster (ERC-4337 EntryPoint v0.7 `0x0000000071727De22E5E9d8BAf0edAc6f37da032`)

บทที่ 11 จะพา deploy ERC-20 token และ Paymaster บนเชนที่ตอนนี้ทำงานได้จริงแล้ว